

Week4 算法复杂度分析与线性结构

Updated 2026-03-23 16:33 GMT+8

Compiled by Hongfei Yan (2025 Spring)

核心内容: 复杂度分析 (Big-O)、基础排序、链表 (Linked List)、栈 (Stack)、队列 (Queue)

调度场算法 Shunting Yard 是重点。

数据结构是在计算机中存储、组织数据的方式。小到变量、数组，大到线段树、平衡树，都是数据结构。

程序运行离不开数据结构，不同的数据结构又各有优劣，能够处理的问题各不相同，而根据具体问题选取合适的数据结构，可以大大提升程序的效率。所以，学习各种各样的数据结构是很有必要的。

数据结构部分简介，<https://oi-wiki.org/ds/>

0 内容预览

0.1 算法分析与时间复杂度 (Analysis & Big-O)

在研究具体数据结构前，必须掌握衡量效率的度量衡。

1 渐近记号 (Asymptotic Notation)

我们使用 **RAM (Random Access Machine)** 模型，假设基本操作（算术、存取、控制流）耗费常数时间。

- $O(g(n))$ (**Upper Bound**): 渐近上界，最坏情况。
- $\Omega(g(n))$ (**Lower Bound**): 渐近下界，最好情况。
- $\Theta(g(n))$ (**Tight Bound**): 紧确界，平均情况。

2 常用复杂度级别 (Common Orders of Growth)

复杂度	名称	典型例子
$O(1)$	Constant / 常数级	数组索引、哈希表查找
$O(\log n)$	Logarithmic / 对数级	二分查找
$O(n)$	Linear / 线性级	遍历列表、线性查找
$O(n \log n)$	Linearithmic	归并排序、快速排序平均情况
$O(n^2)$	Quadratic / 平方级	冒泡排序、嵌套循环
$O(2^n)$	Exponential / 指数级	递归求斐波那契、汉诺塔

Tip: 在 Python 中, `list.append()` 是 $O(1)$, 但 `list.insert(0, v)` 是 $O(n)$ 。

0.2 线性数据结构 (Linear Structures)

线性结构的特征是：元素之间存在一对一的线性关系，具有“前驱”和“后继”的概念。

1 顺序存储 vs 链式存储

特性	顺序表 (Sequential List)	链表 (Linked List)
底层实现	连续内存块 (Array)	离散节点 + 指针 (Pointers)
随机访问 $O(1)$	支持	不支持 (需 $O(n)$ 遍历)
插入/删除	$O(n)$ (涉及元素移动)	$O(1)$ (已知位置时修改指针)
空间开销	预分配空间, 可能浪费	每个节点需额外存指针空间
Python对应	<code>list</code> (动态数组)	需手动实现 <code>Node</code> 类

2 链表的三种形态

1. 单向链表 (Singly Linked List): 每个节点仅指向下一个。
2. 双向链表 (Doubly Linked List): 节点含 `prev` 和 `next`, 支持双向遍历。
3. 循环链表 (Circular Linked List): 尾节点指向头节点。

经典技巧: 快慢指针 (Fast-Slow Pointers)

- 找中点: 快指针走两步, 慢指针走一步。
- 判环: 若快慢指针相遇, 则链表有环。

0.3 栈 (Stack) - 后进先出 (LIFO)

栈是一种限制在单端进行插入和删除的操作受限线性表。

1 核心操作

- `push(item)`: 入栈
- `pop()`: 出栈 (弹出顶部元素)
- `peek()`: 查看栈顶元素但不弹出

2 典型应用实战

1. **括号匹配 (Parentheses Matching)**: 遇到左括号入栈, 遇到右括号弹出匹配。
2. **进制转换 (Base Conversion)**: 除 N 取余, 余数入栈, 最后弹出。
3. **表达式转换 (Infix to Postfix)**: 调度场算法 (Shunting Yard)。
 - 数字直接输出。
 - 优先级高的运算符先出栈。
 - 括号强制改变优先级。

0.4 队列 (Queue) - 先进先出 (FIFO)

1 普通队列

- `enqueue`: 队尾入队
- `dequeue`: 队首出队
- **Python 实践**: 避免使用 `list.pop(0)` ($O(n)$), 应使用 `collections.deque` 的 `popleft()` ($O(1)$)。

2 双端队列 (Deque)

允许在两端同时进行 `add` 和 `remove` 操作。

- **应用**: 回文检查 (Palindrome Checker)。

3 模拟系统 (Simulation)

通过队列模拟真实世界 (如打印机任务、约瑟夫环), 通过平均等待时间评估系统效率。

0.5 排序算法 (Sorting Algorithms)

1 $O(n^2)$ 基础排序

- **冒泡排序 (Bubble Sort)**: 邻居交换, 每趟把最大的“浮”到最后。
- **选择排序 (Selection Sort)**: 寻找剩余部分最小值, 每趟只交换一次 (效率略优于冒泡)。
- **插入排序 (Insertion Sort)**: 构建有序序列, 对未排序数据在已排序序列中从后向前扫描并插入 (在数据近乎有序时极快)。

2 $O(n \log n)$ 高级排序

- **归并排序 (Merge Sort)**: 分治法, 稳定排序, 需要 $O(n)$ 额外空间。
- **快速排序 (Quick Sort)**: 分治法, 不稳定, 原地排序。最坏情况 $O(n^2)$ (取决于 Pivot 选择)。
- **希尔排序 (Shell Sort)**: 插入排序的改进版, 通过步长 (Gap) 减少移动次数。

3 排序算法对比表

算法	平均时间复杂度	空间复杂度	稳定性
Bubble	$O(n^2)$	$O(1)$	Stable
Selection	$O(n^2)$	$O(1)$	Unstable
Insertion	$O(n^2)$	$O(1)$	Stable
Merge	$O(n \log n)$	$O(n)$	Stable
Quick	$O(n \log n)$	$O(\log n)$	Unstable

0.6 代码模板 (Python Implementation)

1 快速排序 (双指针法)

```
1 def quick_sort(arr, low, high):
2     """
3     快速排序主函数 (递归入口)
4     arr: 待排序的列表, low: 当前处理子数组的起始索引, high: 当前处理子数组的结束索引
5     """
6     # 递归终止条件: 当子数组长度为 0 或 1 时 (low >= high), 无需排序
7     if low < high:
8         # 1. 分区操作: 将数组分为两部分, 左边 <= pivot, 右边 >= pivot
9         # 返回 pivot 最终所在的索引位置
10        pivot_idx = partition(arr, low, high)
11
12        # 2. 递归排序 pivot 左边的子数组
13        quick_sort(arr, low, pivot_idx - 1)
14
15        # 3. 递归排序 pivot 右边的子数组
16        quick_sort(arr, pivot_idx + 1, high)
17
18 def partition(arr, low, high):
19     """
20     分区函数 (双指针法 / 双向扫描)。逻辑:
21     1. 选取第一个元素作为基准值 (pivot)。
22     2. left 指针从左向右找大于 pivot 的数。
23     3. right 指针从右向左找小于 pivot 的数。
24     4. 交换这两个数, 直到指针相遇。
25     5. 最后将 pivot 放到相遇位置 (right), 完成分区。
26
27     返回: right: pivot 最终所在的索引
28     """
29     # 选取子数组的第一个元素作为基准值
30     pivot = arr[low]
31
32     # 初始化双指针
33     # left 从基准值的下一个位置开始, 向右扫描
34     left = low + 1
35     # right 从子数组末尾开始, 向左扫描
```

```

36     right = high
37
38     while True:
39         # 只要 left 没越界 且 当前元素 <= pivot, 就继续向右移
40         # 目的: 找到第一个 > pivot 的元素
41         while left <= right and arr[left] <= pivot:
42             left += 1
43
44         # 只要 right 没越界 且 当前元素 >= pivot, 就继续向左移
45         # 目的: 找到第一个 < pivot 的元素
46         while left <= right and arr[right] >= pivot:
47             right -= 1
48
49         # --- 判断是否交换 ---
50         if left <= right:
51             # 如果 left 和 right 没有交错, 说明找到了逆序对
52             # 交换 arr[left] (大于pivot) 和 arr[right] (小于pivot)
53             arr[left], arr[right] = arr[right], arr[left]
54         else:
55             # 如果 left > right, 说明指针已交错, 扫描结束
56             break
57
58         # --- 基准值归位 ---
59         # 循环结束后, right 指向的是最后一个 <= pivot 的位置
60         # 将基准值 (arr[low]) 与 arr[right] 交换
61         # 此时, right 左侧都 <= pivot, 右侧都 >= pivot
62         arr[low], arr[right] = arr[right], arr[low]
63
64         # 返回基准值的最终索引, 供递归使用
65         return right

```

2 归并排序 (分治法)

```

1  def merge_sort(arr):
2      if len(arr) <= 1:
3          return arr
4      mid = len(arr) // 2
5      left = merge_sort(arr[:mid])
6      right = merge_sort(arr[mid:])
7      return merge(left, right)
8
9  def merge(left, right):
10     res = []
11     i = j = 0
12     while i < len(left) and j < len(right):
13         if left[i] < right[j]:
14             res.append(left[i]); i += 1
15         else:
16             res.append(right[j]); j += 1
17     res.extend(left[i:])
18     res.extend(right[j:])
19     return res

```

0.7 常见笔试面试 (Q&A)

Q: 为什么 QuickSort 的平均情况是 $O(n \log n)$?

A: 因为平均每次划分能平衡, 递归树深度为 $\log n$, 每层处理 $O(n)$ 个元素。

Q: 链表和数组哪个随机插入快?

A: 若已知位置指针, 链表快 ($O(1)$); 若需先寻找第 i 个位置, 两者理论都是 $O(n)$, 但数组由于内存连续性对 CPU 缓存更友好, 实际通常更快。

Q: Python 的 `list` 扩容机制?

A: 当空间不足时, Python 会申请更大的空间 (通常倍增), 并搬运旧数据。这使得 `append` 的摊还复杂度仍为 $O(1)$ 。

1 算法分析与时间复杂度

使用数据结构与算法 (DSA) 的主要目的是为了有效地和高效地解决问题。你如何决定自己编写的程序是否高效呢? 这通过复杂度来衡量。复杂度分为两种类型:

1. 时间复杂度: 时间复杂度用于衡量执行代码所需的时间。
2. 空间复杂度: 空间复杂度指的是成功执行代码功能所需的存储空间量。在数据结构与算法中, 会经常遇到辅助空间这个术语, 它指的是程序中除了输入数据结构外使用的额外空间。

简单LC234.回文链表, <https://leetcode.cn/problems/palindrome-linked-list/>

进阶: 能否用 $O(n)$ 时间复杂度和 $O(1)$ 空间复杂度解决此题?

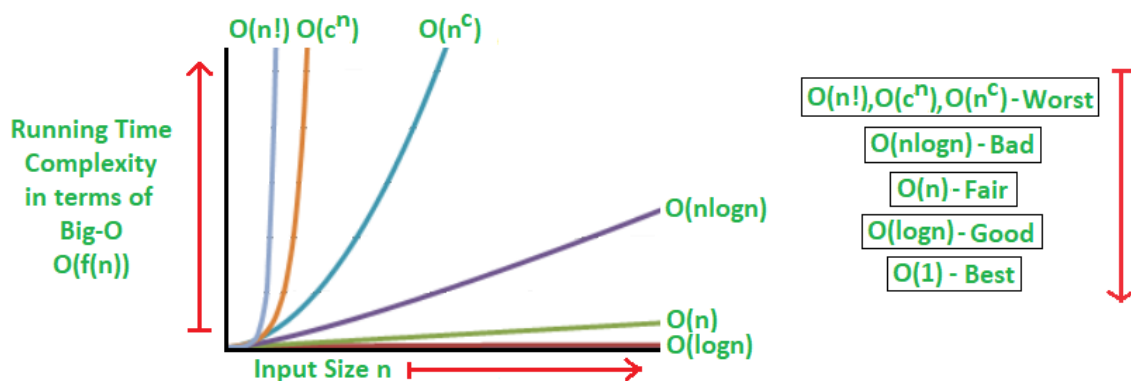
上述两种复杂度都是相对于输入参数来衡量的。但这里出现了一个问题。执行一段代码所需的时间取决于多个因素, 例如:

- 程序中执行的操作数量, 设备的速度, 以及如果是在在线平台上执行的话, 数据传输的速度。

那么如何确定哪一个更高效呢? 答案是使用渐近符号。**渐近符号**是一种数学工具, 它根据输入大小计算所需时间, 并不需要实际执行代码。

它忽略了依赖于系统的常数, 并且只与整个程序中执行的模块化操作的数量有关。以下三种渐近符号最常用以表示算法的时间复杂度:

- **大O符号 (O)** - **大O符号特别描述了最坏情况下情形**。 Θ 记号渐进地给出算法的平均复杂度, 即一个函数的上界和下界。当只有一个渐进上界时, 即当 n 足够大时, 使用 O 记号。
- **欧米伽符号 (Ω)** - 欧米伽(Ω)符号特别描述了最好情况下情形。
- **西塔符号 (Θ)** - 这个符号代表了算法的平均复杂度。



算法的增长率

在代码分析中最常用的符号是**大O符号**，它给出了代码运行时间的上界（或者说是输入规模大小对应的内存使用量）。大O符号帮助理解当输入数据量增加时，算法的执行时间或空间需求将以怎样的速度增长。

8.1 分析算法

分析算法意味着预测该算法所需的资源。虽然有时我们主要关心像内存、通信带宽或计算机硬件这类资源，但是通常想要度量的是计算时间。一般来说，通过分析某个问题的几种候选算法，可以识别出最高效的那一个。这样的分析可能会指出不止一个可行的候选算法，但在这一过程中，通常可以淘汰几个较差的算法。

在分析一个算法之前，必须有一个要使用的实现技术的模型，包括该技术的资源模型及其成本。我们将假设一种通用的**单处理器计算模型——随机存取机（random-access machine, RAM）**来作为实现技术，算法可以用计算机程序来实现。在RAM模型中，指令是顺序执行的，没有并发操作。

严格来说，应该精确地定义RAM模型的指令及其成本。然而这样做会很繁琐，并且不会对算法设计和分析提供太多的洞察力。但必须小心不要滥用RAM模型。例如，如果RAM有一个排序指令，那么就可以只用一条指令完成排序。这样的RAM将是不现实的，因为实际的计算机没有这样的指令。因此，**指导原则是实际计算机的设计方式。RAM模型包含了在实际计算机中常见的指令：算术运算（如加法、减法、乘法、除法、求余数、取底、取顶），数据移动（加载、存储、复制），以及控制（条件分支和无条件分支、子程序调用和返回）。每条这样的指令都需要固定的时间量。**

RAM模型中的数据类型有整型和浮点实数型。虽然在此处通常不关心精度问题，但在某些应用中精度是至关重要的。也假设每个数据字的大小有限制。例如，在处理大小为 n 的输入时，我们通常假设对某个常数 $c \geq 1$ ，整数由 $c \lg n$ 位表示。我们要求 $c \geq 1$ 是为了确保每个字能够容纳 n 的值，从而使我们能够索引各个输入元素，并且我们将 c 限制为常数以防止字长无限增长。（如果字长可以无限增长，我们就可以在一个字中存储大量数据并在恒定时间内对其进行操作——这显然是一种不切实际的情况。）

计算机字长（Computer Word Length）是指计算机中用于存储和处理数据的基本单位的位数。它表示计算机能够一次性处理的二进制数据的位数。

字长的大小对计算机的性能和数据处理能力有重要影响。较大的字长通常意味着计算机能够处理更大范围的数据或执行更复杂的操作。常见的字长包括 8 位、16 位、32 位和 64 位。

较小的字长可以节省存储空间和资源，适用于简单的计算任务和资源有限的设备。较大的字长通常用于处理更大量级的数据、进行复杂的计算和支持高性能计算需求。

需要注意的是，字长并不是唯一衡量计算机性能的指标，还有其他因素如处理器速度、缓存大小、操作系统等也会影响计算机的整体性能。

实际计算机包含未在上述列表中的指令，这些指令在RAM模型中代表了一个灰色地带。例如，幂运算是否是常数时间指令？在一般情况下，并不是；当 x 和 y 是实数时，计算 x^y 需要多条指令。然而，在某些受限的情况下，幂运算是一个常数时间操作。许多计算机有一个“左移”指令，它可以在常数时间内将一个整数的位向左移动 k 个位置。在大多数计算机中，将一个整数的位向左移动一个位置等价于乘以 2，因此将位向左移动 k 个位置就等价于乘以 2^k 。因此，只要 k 不超过计算机字的位数，这样的计算机可以通过将整数 1 左移 k 个位置来在一个常数时间内计算出 2^k 。我们将努力避免在RAM模型中出现这样的灰色地带，但在 k 是一个足够小的正整数时，我们会把 2^k 的计算视为一个常数时间操作。

在RAM模型中，并不试图模拟现代计算机中常见的内存层次结构。也就是说，不模拟缓存或虚拟内存。一些计算模型尝试考虑内存层次结构效应，这在实际程序运行在真实机器上时有时是非常显著的。但总体而言，分析不会考虑它们。包括内存层次结构的模型比RAM模型复杂得多，所以可能难于使用。此外，基于RAM模型的分析通常是实际机器性能的良好预测指标。

即使是在RAM模型中分析一个简单的算法也可能是一项挑战。所需的数学工具可能包括组合数学、概率论、代数技巧以及识别公式中最重要的项的能力。由于一个算法的行为可能对每个可能的输入都是不同的，需要一种方式来用简单且易于理解的公式概括这种行为。

尽管通常只选择一种机器模型来分析给定的算法，但在决定如何表达分析时，仍然面临许多选择。希望有一种方法可以简单地书写和操作，能够展示算法资源需求的重要特征，并抑制繁琐的细节。

1 插入排序的算法分析

在算法分析中有两个关键概念：输入规模（input size）和运行时间（running time），并以插入排序（INSERTION-SORT）为例来说明这些概念。

输入规模是衡量一个算法输入大小的标准，它取决于具体的问题。对于一些问题，比如排序或计算离散傅里叶变换，最自然的度量方式是输入中项目的数量——例如，排序时的数组长度 n 。对于其他问题，如整数乘法，输入规模的最佳度量可能是表示输入所需的总位数。有时，用两个数字描述输入规模比用一个更合适，例如，如果算法的输入是一个图，则可以用顶点和边的数量来描述输入规模。

运行时间是指特定输入上算法执行的基本操作或“步骤”的数量。为了使这个概念尽可能与机器无关，通常假设伪代码中的每一行需要常量时间来执行。也就是说，每一行可能需要不同的时间，但每执行第 i 行所需的时间为 c_i ，其中 c_i 是一个常数。这种观点符合RAM模型，并且反映了大多数实际计算机上如何实现伪代码。

随着讨论的深入，对插入排序运行时间的表达将从使用所有语句成本 c_i 的复杂公式演变成为一种更简单、更简洁、更易处理的符号。这种简化后的符号也将使得比较不同算法的效率变得更加容易。

接下来，使用插入排序过程来展示每个语句的时间“成本”以及每个语句被执行的次数。对于每一个 $j = 2, 3, \dots, n$ （其中 $n = A.length$ ），令 t_j 表示当 j 取该值时，第 7 行的 while 循环测试被执行的次数。当 for 或 while 循环以常规方式退出（即由于循环头中的测试）时，测试被执行的次数比循环体多一次。另外，这里假定注释不是可执行语句，因此它们不需要任何时间。

Implementation of Insertion Sort Algorithm

插入排序是一种简单的排序算法，其工作原理类似于你在手中整理扑克牌的方式。数组被虚拟地分成已排序和未排序两部分。从未排序部分选取值，并将其放置到已排序部分的正确位置上。

$\sum_{j=1}^{n-1} t_j$ 是 $\sum_{j=1}^{n-1} t_j$ 的LaTeX表示，

$\sum_{j=1}^{n-1} (t_j - 1)$ 是 $\sum_{j=1}^{n-1} (t_j - 1)$ 的LaTeX表示。

```
1 def insertion_sort(arr):                                # cost times
```

```

2     for i in range(1, len(arr)):           # c1      n
3         j = i                             # c2      n - 1
4
5         # Insert arr[j] into the
6         # sorted sequence arr[0..j-1]      # 0        n - 1
7         while arr[j - 1] > arr[j] and j > 0: # c4      \sum_{j=1}^{n-1}
t_j
8             arr[j - 1], arr[j] = arr[j], arr[j - 1] # c5      \sum_{j=1}^{n-1}
(t_j - 1)
9             j -= 1                         # c6      \sum_{j=1}^{n-1}
(t_j - 1)
10
11
12 arr = [2, 6, 5, 1, 3, 4]
13 insertion_sort(arr)
14 print(arr)
15
16 # [1, 2, 3, 4, 5, 6]

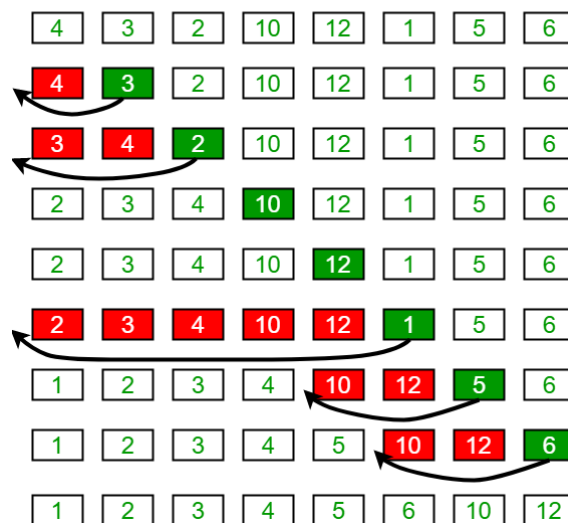
```

<https://www.geeksforgeeks.org/insertion-sort/>

Insertion sort is a simple sorting algorithm that works similarly to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed in the correct position in the sorted part.

要以升序对大小为N的数组进行排序，需要遍历数组并将当前元素（称为“关键字”）与它的前一个元素进行比较；如果关键字比它的前一个元素小，则将它与前面的元素进行比较。将较大的元素移动一个位置以腾出空间给被交换的元素。

Insertion Sort Execution Example



笔试题目示例

Q: Suppose you have the following list of numbers to sort: [15, 5, 4, 18, 12, 19, 14, 10, 8, 20] which list represents the partially sorted list after three complete passes of insertion sort? (C)

- A. [4, 5, 12, 15, 14, 10, 8, 18, 19, 20]
- B. [15, 5, 4, 10, 12, 8, 14, 18, 19, 20]
- C. [4, 5, 15, 18, 12, 19, 14, 10, 8, 20]**
- D. [15, 5, 4, 18, 12, 19, 14, 8, 10, 20]

算法的运行时间是每个被执行语句的运行时间之和；一个执行一次需要 c_i 步骤并且总共执行 n 次的语句将对总运行时间贡献 $c_i \times n$ 。为了计算 $T(n)$ ，即在 n 个值的输入上 INSERTION-SORT 的运行时间，我们将成本列和次数列的乘积相加，得到

$$T(n) = \sum (c_i \times t_i)$$

这里，每个 c_i 代表伪代码中第 i 行执行一次所需的时间（常量），而 t_i 则表示该行被执行的次数。对于插入排序，需要考虑每一行代码被执行的具体情况，特别是内层循环的执行次数会依赖于数组中元素的初始排列。通过这种方式，可以得出一个关于输入规模 n 的函数表达式，用来描述插入排序的运行时间。

$$T(n) = c_1 n + c_2 (n - 1) + c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)$$

即使对于给定大小的输入，**算法的运行时间也可能取决于给出的是哪个具体输入**。例如，在插入排序 (INSERTION-SORT) 中，最佳情况发生在数组已经是有序的时候。对于每个 $i = 1, 2, 3, \dots, n-1$ ，当 j 在其初始值 i 时，在第 7 行发现 $arr[j - 1] \leq arr[j]$ 。因此，对于 $i = 1, 2, 3, \dots, n-1$ ，有 $t_j = 1$ ，这时最佳情况下的运行时间是

$$\begin{aligned} T_{\text{best}}(n) &= c_1 n + c_2 (n - 1) + c_4 (n - 1) \\ &= (c_1 + c_2 + c_4) n - (c_2 + c_4) \end{aligned}$$

可以将这个运行时间表示为 $an + b$ ，其中常量 a 和 b 取决于语句成本 c_i ；因此，它是 n 的**线性函数**。

如果数组是以逆序排序的——也就是说，以递减顺序排列——那么就会出现最坏情况。我们必须将每个元素 $A[j]$ 与整个已排序子数组 $A[0 \dots j-1]$ 中的每个元素进行比较，因此 $t_j = j$ 对于 $j = 1, 2, \dots, n-1$ 。注意到这一点，

$$\begin{aligned} \sum_{j=1}^{n-1} j &= \frac{n(n-1)}{2} \\ \sum_{j=1}^{n-1} (j - 1) &= \frac{(n-1)(n-2)}{2} \end{aligned}$$

we find that in the worst case, the running time of INSERTION-SORT is

$$\begin{aligned} T_{\text{worst}}(n) &= c_1 n + c_2 (n - 1) + c_4 \left(\frac{n(n-1)}{2} - 1 \right) + c_5 \left(\frac{(n-1)(n-2)}{2} \right) + c_6 \left(\frac{(n-1)(n-2)}{2} \right) \\ &= \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2} \right) n^2 + \left(c_1 + c_2 - \frac{c_4}{2} - \frac{3c_5}{2} - \frac{2c_6}{2} \right) n + (-c_2 - c_4 + c_5 + c_6) \end{aligned}$$

可以将这种最坏情况下的运行时间表示为 $an^2 + bn + c$ ，其中常量 a 、 b 和 c 再次取决于语句成本 c_i ；因此，它是 n 的**二次函数**。

通常情况下，就像在插入排序中一样，对于给定的输入，算法的运行时间是固定的，尽管一些有趣的“随机化”算法，即使对于固定的输入，它们的行为也可能有所不同。

2 最坏情况与平均情况分析

在对插入排序的分析中，既考虑了最佳情况，即输入数组已经排序的情况，也考虑了最坏情况，即输入数组是逆序排列的情况。然而，通常专注于寻找只有最坏情况下的运行时间，也就是对于任何大小为 n 的输入最长的运行时间。给出关注最坏情况的三个理由：

- 一个算法的最坏情况下的运行时间为任何输入提供了一个运行时间的上限。了解它提供了算法永远不会超过这个时间的保证。
- 对于某些算法，最坏情况出现得相当频繁。例如，在数据库中搜索特定信息时，当信息不在数据库中时，搜索算法的最坏情况经常发生。在某些应用中，可能经常会进行不存在的信息搜索。
- “平均情况”通常几乎和最坏情况一样糟糕。假设我们随机选择 n 个数字并应用插入排序。确定元素 $A[j]$ 应该插入到子数组 $A[0 \dots j-1]$ 中的哪个位置需要多长时间？平均来说， $A[0 \dots j-1]$ 中的一半元素小于 $A[j]$ ，另一半大于 $A[j]$ 。因此，平均而言，我们需要检查子数组 $A[0 \dots j-1]$ 的一半，所以 t_j 大约是 $j/2$ 。结果得到的平均情况下的运行时间最终是输入规模的二次函数，就像最坏情况下的运行时间一样。

在某些特殊情况下，我们将对算法的平均情况运行时间感兴趣；将看到概率分析技术应用于各种算法。平均情况分析的范围是有限的，因为可能不清楚什么构成特定问题的“平均”输入。经常假设所有给定大小的输入都是等可能的。实际上，这一假设可能会被违反，但有时可以使用一种随机化算法，它会做出随机选择，从而使概率分析成为可能，并得出一个期望的运行时间。

笔试题目示例

对长度为 3 的顺序表进行查找，若查找第一个元素的概率为 $1/2$ ，查找第二个元素的概率为 $1/4$ ，查找第三个元素的概率为 $1/8$ ，则执行任意查找需要比较元素的平均个数为 ____。

$\#1 * (1/2) + 2 * (1/4) + 3 * (1/8) + 3 * (1/8) = 1.75$ ，还有 $1/8$ 的失败查询概率。

3 增长量级

我们使用某些简化的抽象来使过程 INSERTION-SORT 的分析更加容易。首先，通过使用常数 c_i 来表示这些代价来忽略每条语句的实际成本。其次，注意到这些常数也提供了比我们真正需要的更多细节：把最坏情况运行时间表达为 $an^2 + bn + c$ ，其中 a 、 b 和 c 是依赖于语句代价 c_i 的常数。这样，我们不但忽略实际的语句代价，而且也忽略抽象的代价 c_i 。

现在我们做出一种更简化的抽象：即我们真正兴趣的运行时间的增长率，或增长量级。所以，只考虑公式中主要项（例如， an^2 ），因为对于 n 的值很大时，低阶项相对来说不太重要。我们也忽略主要项的常系数，因为对大的输入，在确定计算效率时常量因子不如增长率重要。对于插入排序，当忽略低阶项和主要项的常系数后，剩下的是来自主要项的 n^2 因子。记插入排序具有最坏运行时间 $\Theta(n^2)$ （读作“theta of n-squared”）。

如果一个算法的最坏情况具有比另一个算法更低的增长量级，则它比另一个算法更高效。由于常数因子和低阶项的影响，一个运行时间增长阶较高的算法在小输入的情况下可能会比一个增长阶较低的算法花费的时间更少。但对于足够大的输入，例如，在最坏情况下，一个 $\Theta(n^2)$ 的算法将比一个 $\Theta(n^3)$ 的算法运行得更快。

这里提到的 Θ 符号是用来描述算法运行时间的增长阶的紧确界，意味着算法的运行时间在渐近情况下既不会快于也不会慢于 n^2 的某个常数倍。这是关于算法复杂度分析中的渐近记法的一种表述方式，用来概括地说明算法性能随输入规模变化的趋势。

4 O记号

通用的记号应该是， O 表示上界， Ω 表示下界， Θ 表示渐进阶，就是既上界又下界。

Θ -记号从渐近上界和下界两个方面约束一个函数。当我们只有渐近上界时，我们使用 O -记号。对于给定的函数 $g(n)$ ，用 $O(g(n))$ （读作“大O of g of n ”或简称“O of g of n ”）来表示满足以下条件的函数集合：

$$O(g(n)) = \{f(n) : \text{存在正的常数 } c \text{ 和 } n_0, \text{ 使得对所有 } n \geq n_0, \text{ 有 } 0 \leq f(n) \leq cg(n)\}$$

若存在正常量 c_1 和 c_2 ，使得对于足够大的 n ，函数 $f(n)$ 能“夹入” $c_1g(n)$ 与 $c_2g(n)$ 之间，则 $f(n)$ 属于集合 $\Theta(g(n))$ 。因为 $\Theta(g(n))$ 是一个集合，所以可以记“ $f(n) \in \Theta(g(n))$ ”，以指出 $f(n)$ 是 $\Theta(g(n))$ 的成员。作为替代，我们通常记“ $f(n) = \Theta(g(n))$ ”以表达相同的概念。因为我们按这种方式活用了等式，所以你可能感到困惑，但是在本节的后面我们将看到这样做有其好处。

图3-1(a)给出了函数 $f(n)$ 与 $g(n)$ 的一幅直观画面，其中 $f(n) = \Theta(g(n))$ 。对在 n_0 及其右边 n 的所有值， $f(n)$ 的值位于或高于 $c_1g(n)$ 且位于或低于 $c_2g(n)$ 。换句话说，对所有 $n \geq n_0$ ，函数 $f(n)$ 在一个常量因子内等于 $g(n)$ 。我们称 $g(n)$ 是 $f(n)$ 的一个渐近紧确界(asymptotically tight bound)。

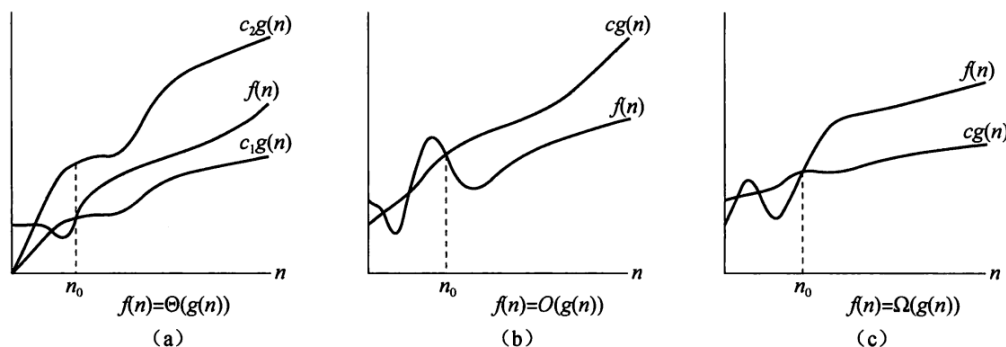


图3-1 Θ 、 O 和 Ω 记号的图例。在每个部分，标出的 n_0 的值是最小的可能值，任何更大的值也将有效。
(a) Θ 记号限制一个函数在常量因子内。如果存在正常量 n_0 、 c_1 和 c_2 ，使得在 n_0 及其右边， $f(n)$ 的值总位于 $c_1g(n)$ 与 $c_2g(n)$ 之间或等于它们，那么记 $f(n) = \Theta(g(n))$ 。(b) O 记号为函数给出一个在常量因子内的上界。如果存在正常量 n_0 和 c ，使得在 n_0 及其右边， $f(n)$ 的值总小于或等于 $cg(n)$ ，那么记 $f(n) = O(g(n))$ 。(c) Ω 记号为函数给出一个在常量因子内的下界。如果存在正常量 n_0 和 c ，使得在 n_0 及其右边， $f(n)$ 的值总大于或等于 $cg(n)$ ，那么记 $f(n) = \Omega(g(n))$ 。

我们使用 O 记号来给出一个函数的一个在常量因子内的上界。图3-1(b)展示了 O 记号背后的直觉知识。对在 n_0 及其右边， $f(n)$ 的值总小于或等于 $cg(n)$ 。

利用 O 记号，通常可以通过检查算法的整体结构来简单描述算法的运行时间。例如，插入排序算法中的双重嵌套循环结构立即给出了最坏情况下运行时间为 $O(n^2)$ 的上界。

由于 O 记号描述的是上界，当用它来约束算法的最坏情况运行时间时，就得到了该算法在任何输入上的运行时间的上限。这意味着，在最坏情况下，算法不会比这个上界更慢，无论输入是什么。

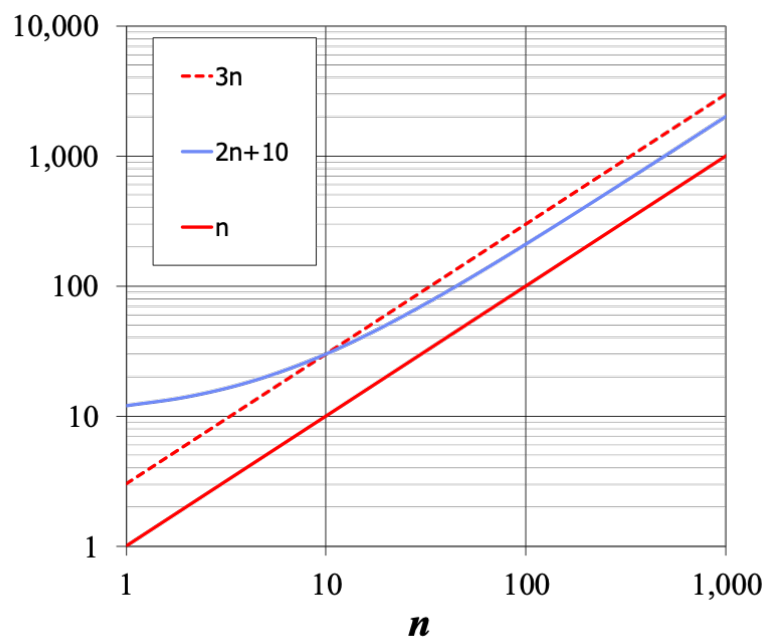
举例： $2n + 10$ is $O(n)$

$$2n + 10 \leq cn$$

$$(c - 2)n \geq 10$$

$$n \geq 10/(c - 2)$$

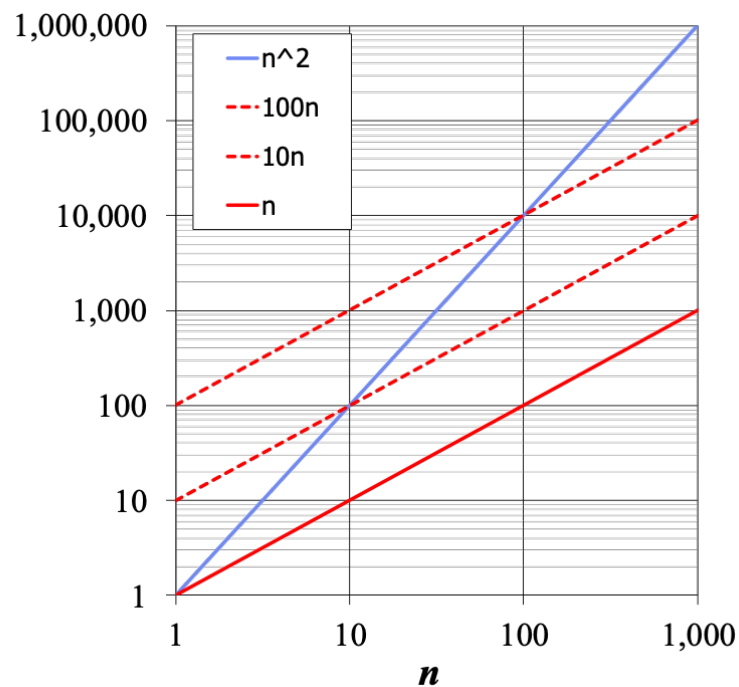
Pick $c = 3$ and $n_0 = 10$



举例: the function n^2 is not $O(n)$

$$n^2 \leq cn$$

$n \leq c$, the inequality cannot be satisfied since c must be a constant



More Big-Oh Examples

$7n - 2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n - 2 \leq cn$ for $n \geq n_0$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq cn^3$ for $n \geq n_0$

This is true for $c = 4$ and $n_0 = 21$

$3\log n + 5$ is $O(\log n)$

need $c > 0$ and $n_0 > 1$ such that $3\log n + 5 \leq c\log n$ for $n \geq n_0$

this is true for $c = 8$ and $n_0 = 2$

大O记号给出了函数增长率的上界。陈述 $f(n)$ 是 $O(g(n))$ 意味着 $f(n)$ 的增长率不超过 $g(n)$ 的增长率。

可以使用大O记号根据它们的增长率来对函数进行排序。

换句话说，如果一个函数 $f(n)$ 是 $O(g(n))$ ，那么对于足够大的输入 n ， $f(n)$ 的值不会超过 $g(n)$ 的某个常数倍。这提供了一种方式来描述和比较不同算法随着输入规模增加而表现出的效率差异，通过将算法的运行时间或空间需求的增长率与一些基准函数（如线性、平方、立方、指数等）进行对比。在算法分析中，经常使用大O记号来简化表达，并专注于算法性能的关键趋势，忽略掉那些对于大规模输入影响较小的细节。

Big-Oh Rules

If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,

Drop lower-order terms 忽略低阶项

Drop constant factors 忽略常数因子

Use the smallest possible class of functions 使用尽可能小的函数类别

Say $2n$ is $O(n)$ instead of $2n$ is $O(n^2)$

Use the simplest expression of the class 使用该类别中最简单的表达方式

Say $3n + 5$ is $O(n)$ instead of $3n + 5$ is $O(3n)$

在 Python 中，`list.append()` 是 $O(1)$ ，但 `list.insert(0, v)` 是 $O(n)$ 。

<https://www.ics.uci.edu/~pattis/ICS-33/lectures/complexitypython.txt>

Lists:				Sets:		
Operation	Example	Complexity Class	Notes	Operation	Example	Complexity Class
Index	<code>l[i]</code>	$O(1)$		Length	<code>len(s)</code>	$O(1)$
Store	<code>l[i] = 0</code>	$O(1)$		Add	<code>s.add(5)</code>	$O(1)$
Length	<code>len(l)</code>	$O(1)$		Containment	<code>x in/not in s</code>	$O(1)$
Append	<code>l.append(5)</code>	$O(1)$	mostly: ICS-46 covers details	Remove	<code>s.remove(..)</code>	$O(1)$
Pop	<code>l.pop()</code>	$O(1)$	same as <code>l.pop(-1)</code> , popping at end	Discard	<code>s.discard(..)</code>	$O(1)$
Clear	<code>l.clear()</code>	$O(1)$	similar to <code>l = []</code>	Pop	<code>s.pop()</code>	$O(1)$
				Clear	<code>s.clear()</code>	$O(1)$
Slice	<code>l[a:b]</code>	$O(b-a)$	<code>l[1:5]:O(1)/l[:]:O(len(l)-0)=O(N)</code>	Dictionaries: dict and defaultdict		
Extend	<code>l.extend(...)</code>	$O(len(...))$	depends only on len of extension			
Construction	<code>list(...)</code>	$O(len(...))$	depends on length of ... iterable			
check ==, !=	<code>l1 == l2</code>	$O(N)$		Dictionaries: dict and defaultdict		
Insert	<code>l[a:b] = ...</code>	$O(N)$				
Delete	<code>del l[i]</code>	$O(N)$	depends on i; $O(N)$ in worst	Operation	Example	Complexity Class
Containment	<code>x in/not in l</code>	$O(N)$	linearly searches list	Index	<code>d[k]</code>	$O(1)$
Copy	<code>l.copy()</code>	$O(N)$	Same as <code>l[:]</code> which is $O(N)$	Store	<code>d[k] = v</code>	$O(1)$
Remove	<code>l.remove(...)</code>	$O(N)$		Length	<code>len(d)</code>	$O(1)$
Pop	<code>l.pop(i)</code>	$O(N)$	$O(N-i)$: <code>l.pop(0):O(N)</code> (see a)	Delete	<code>del d[k]</code>	$O(1)$
Extreme value	<code>min(l)/max(l)</code>	$O(N)$	linearly searches list for v	get/setdefault	<code>d.get(k)</code>	$O(1)$
Reverse	<code>l.reverse()</code>	$O(N)$		Pop	<code>d.pop(k)</code>	$O(1)$
Iteration	<code>for v in l:</code>	$O(N)$	Worst: no return/break in lo	Pop item	<code>d.popitem()</code>	$O(1)$
Sort	<code>l.sort()</code>	$O(N \log N)$	key/reverse mostly doesn't c	Clear	<code>d.clear()</code>	$O(1)$
Multiply	<code>k*l</code>	$O(k \cdot N)$	<code>5*l</code> is $O(N)$; <code>len(l)*l</code> is $O(N^2)$	View	<code>d.keys()</code>	$O(1)$

8.2 Sorting Algorithm

Sorting Algorithm is used to rearrange a given array or list elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of element in the respective data structure.

There are a lot of different types of sorting algorithms. Some widely used algorithms are:

- [Bubble Sort](#)
- [Selection Sort](#)
- [Insertion Sort](#)
- [Quick Sort](#)
- [Merge Sort](#)
- [ShellSort](#)

There are several other sorting algorithms also and they are beneficial in different cases. You can learn about them and more in our dedicated article on [Sorting algorithms](#).

https://github.com/GMyhf/2024spring-cs201/blob/main/code/ten_sort_algorithms.md

包括：冒泡排序（Bubble Sort），插入排序（Insertion Sort），选择排序（Selection Sort），希尔排序（Shell Sort），归并排序（Merge Sort），快速排序（Quick Sort），堆排序（Heap Sort），计数排序（Counting Sort），桶排序（Bucket Sort），基数排序（Radix Sort）

1 Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

Algorithm

In Bubble Sort algorithm,

- traverse from left and compare adjacent elements and the higher one is placed at right side.
- In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

```
1  # Optimized Python program for implementation of Bubble Sort
2  def bubbleSort(arr):
3      n = len(arr)
4
5      # Traverse through all array elements
6      for i in range(n):
7          swapped = False
8
9          # Last i elements are already in place
10         for j in range(0, n - i - 1):
11
12             # Traverse the array from 0 to n-i-1
13             # Swap if the element found is greater
14             # than the next element
15             if arr[j] > arr[j + 1]:
16                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
17                 swapped = True
18         if (swapped == False):
19             break
20
21
22 # Driver code to test above
23 if __name__ == "__main__":
24     arr = [64, 34, 25, 12, 22, 11, 90]
25
26     bubbleSort(arr)
27     print(' '.join(map(str, arr)))
28
```

Complexity Analysis of Bubble Sort:

Time Complexity: $O(N^2)$

Auxiliary Space: $O(1)$

Advantages of Bubble Sort:

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

- Bubble sort has a time complexity of $O(N^2)$ which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

Some FAQs related to Bubble Sort:

Q1. What is the Boundary Case for Bubble sort?

Bubble sort takes minimum time (Order of n) when elements are already sorted. Hence it is best to check if the array is already sorted or not beforehand, to avoid $O(N^2)$ time complexity.

Q2. Does sorting happen in place in Bubble sort?

Yes, Bubble sort performs the swapping of adjacent pairs without the use of any major data structure. Hence Bubble sort algorithm is an **in-place** algorithm.

Q3. Is the Bubble sort algorithm stable?

Yes, the bubble sort algorithm is **stable**.

Q4. Where is the Bubble sort algorithm used?

Due to its simplicity, bubble sort is often used to introduce the concept of a sorting algorithm.

Q: Suppose you have the following list of numbers to sort: [19, 1, 9, 7, 3, 10, 13, 15, 8, 12] which list represents the partially sorted list after three complete passes of bubble sort?? (B)

A: [1, 9, 19, 7, 3, 10, 13, 15, 8, 12] B: **[1, 3, 7, 9, 10, 8, 12, 13, 15, 19]**

C: [1, 7, 3, 9, 10, 13, 8, 12, 15, 19] D: [1, 9, 19, 7, 3, 10, 13, 15, 8, 12]

2 Selection Sort

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

The algorithm repeatedly **selects the smallest (or largest)** element from the unsorted portion of the list and swaps it with the first element of the unsorted part. This process is repeated for the remaining unsorted portion until the entire list is sorted.

```
1  A = [64, 25, 12, 22, 11]
2
3  # Traverse through all array elements
```

```

4  for i in range(len(A)):
5
6  # Find the minimum element in remaining
7  # unsorted array
8  min_idx = i
9  for j in range(i + 1, len(A)):
10     if A[min_idx] > A[j]:
11         min_idx = j
12
13     # Swap the found minimum element with
14     # the first element
15     A[i], A[min_idx] = A[min_idx], A[i]
16
17 # Driver code to test above
18 print(' '.join(map(str, A)))
19
20 # Output: 11 12 22 25 64

```

The **selection sort** improves on the bubble sort by making only one exchange for every pass through the list. In order to do this, a selection sort looks for the largest value as it makes a pass and, after completing the pass, places it in the proper location. As with a bubble sort, after the first pass, the largest item is in the correct place. After the second pass, the next largest is in place. This process continues and requires $n-1$ passes to sort n items, since the final item must be in place after the $(n-1)$ st pass.

Figure 3 shows the entire sorting process. On each pass, the largest remaining item is selected and then placed in its proper location. The first pass places 93, the second pass places 77, the third places 55, and so on.

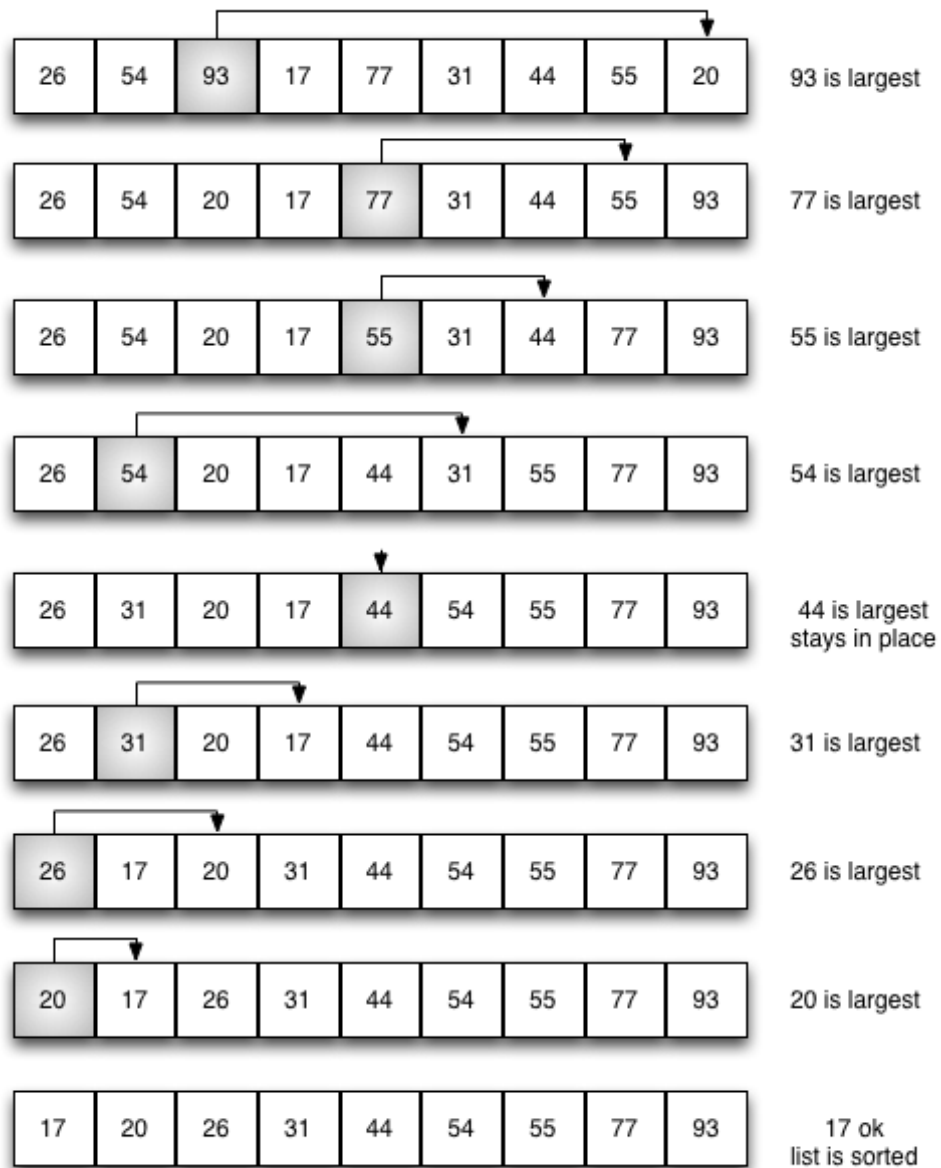


Figure 3: selectionSort

```

1  def selectionSort(alist):
2      for fillslot in range(len(alist)-1, 0, -1):
3          positionOfMax = 0
4          for location in range(1, fillslot+1):
5              if alist[location] > alist[positionOfMax]:
6                  positionOfMax = location
7
8          if positionOfMax != fillslot:
9              alist[fillslot], alist[positionOfMax] = alist[positionOfMax],
alist[fillslot]
10
11  alist = [54, 26, 93, 17, 77, 31, 44, 55, 20]
12  selectionSort(alist)
13  print(alist)
14
15  # [17, 20, 26, 31, 44, 54, 55, 77, 93]

```

You may see that the selection sort makes the same number of comparisons as the bubble sort and is therefore also O^2). However, **due to the reduction in the number of exchanges, the selection sort typically executes faster in benchmark studies**. In fact, for our list, the bubble sort makes 20 exchanges, while the selection sort makes only 8.

Complexity Analysis of Selection Sort

Time Complexity: The time complexity of Selection Sort is $O(N^2)$ as there are two nested loops:

- One loop to select an element of Array one by one = $O(N)$
- Another loop to compare that element with every other Array element = $O(N)$
- Therefore overall complexity = $O(N) * O(N) = O(N*N) = O(N^2)$

Auxiliary Space: $O(1)$ as the only extra memory used is for temporary variables while swapping two values in Array. The selection sort never makes more than $O(N)$ swaps and can be useful when memory writing is costly.

Advantages of Selection Sort Algorithm

- Simple and easy to understand.
- Works well with small datasets.

Disadvantages of the Selection Sort Algorithm

- Selection sort has a time complexity of $O(n^2)$ in the worst and average case.
- Does not work well on large datasets.
- Does not preserve the relative order of items with equal keys which means it is not stable.

Frequently Asked Questions on Selection Sort

Q1. Is Selection Sort Algorithm stable?

The default implementation of the Selection Sort Algorithm is **not stable**. However, it can be made stable. Please see the [stable Selection Sort](#) for details.

Q2. Is Selection Sort Algorithm in-place?

Yes, Selection Sort Algorithm is an **in-place** algorithm, as it does not require extra space.

Q: Suppose you have the following list of numbers to sort: [11, 7, 12, 14, 19, 1, 6, 18, 8, 20] which list represents the partially sorted list after three complete passes of selection sort?
(D)

- A. [7, 11, 12, 1, 6, 14, 8, 18, 19, 20]
- B. [7, 11, 12, 14, 19, 1, 6, 18, 8, 20]
- C. [11, 7, 12, 14, 1, 6, 8, 18, 19, 20]
- D. [11, 7, 12, 14, 8, 1, 6, 18, 19, 20]

3 Quick Sort

quickSort is a sorting algorithm based on the [Divide and Conquer algorithm](#) that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

How does QuickSort work?

The key process in quickSort is a partition(). The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.

Partition is done **recursively** on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.

<https://www.geeksforgeeks.org/introduction-to-divide-and-conquer-algorithm-data-structure-and-algorithm-tutorials/>

Divide And Conquer

This technique can be divided into the following three parts:

1. **Divide:** This involves dividing the problem into smaller sub-problems.
2. **Conquer:** Solve sub-problems by calling recursively until solved.
3. **Combine:** Combine the sub-problems to get the final solution of the whole problem.

The following are some standard algorithms that follow Divide and Conquer algorithm.

1. **Quicksort** is a sorting algorithm. The algorithm picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.
2. **Merge Sort** is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.

What does not qualifies as Divide and Conquer:

Binary Search is a searching algorithm. In each step, the algorithm compares the input element x with the value of the middle element in the array. If the values match, return the index of the middle. Otherwise, if x is less than the middle element, then the algorithm recurs for the left side of the middle element, else recurs for the right side of the middle element. Contrary to popular belief, this is not an example of Divide and Conquer because there is only one sub-problem in each step (**Divide and conquer requires that there must be two or more sub-problems**) and hence this is a case of **Decrease and Conquer**.

在partition函数中两个指针 `i` 和 `j` 的方式实现。快排中的partition可以用双指针或者单指针实现，前者更容易理解，也是机考中喜欢出的题目类型。

```
1 def quicksort(arr, left, right):
2     if left < right:
3         partition_pos = partition(arr, left, right)
4         quicksort(arr, left, partition_pos - 1)
5         quicksort(arr, partition_pos + 1, right)
6
7
8 def partition(arr, left, right):
9     i = left
10    j = right - 1
11    pivot = arr[right]
12    while i <= j:
13        while i <= right and arr[i] < pivot:
14            i += 1
15        while j >= left and arr[j] >= pivot:
16            j -= 1
17        if i < j:
18            arr[i], arr[j] = arr[j], arr[i]
19    if arr[i] > pivot:
20        arr[i], arr[right] = arr[right], arr[i]
21    return i
22
23
24 arr = [22, 11, 88, 66, 55, 77, 33, 44]
25 quicksort(arr, 0, len(arr) - 1)
26 print(arr)
27
28 # [11, 22, 33, 44, 55, 66, 77, 88]
```

要分析 `quicksort` 函数，注意对于长度为 n 的列表，如果分区总是发生在列表中间，则将再次有 $\log n$ 次划分。为了找到分割点，需要检查每个 n 项与枢轴值（pivot value）。结果是 $n \log n$ 。此外，不需要额外的内存，不像归并排序过程那样。

不幸的是，在最坏的情况下，分割点可能不在中间，并且可能会极度偏向左侧或右侧，导致非常不均匀的划分。在这种情况下，对 n 项的列表进行排序会被分为对 0 项和 $n-1$ 项的两个列表进行排序。然后，对 $n-1$ 项的列表排序又被分为大小为 0 和大小为 $n-2$ 的列表，依此类推。结果是一个 $O(n^2)$ 的排序，伴随着递归所需的所有开销。

有多种方式来选择枢轴值。特别地，可以尝试通过一种称为“三数取中”（median of three）的技术来缓解因划分不均带来的问题。为了选择枢轴值，我们会考虑列表中的第一个、中间的和最后一个元素。在我们的例子中，它们是 54、77 和 20。现在选取中位数值，在我们的情况中是 54，并用它作为枢轴值。这个想法是在列表中的第一项不属于列表中间位置时，使用三数取中能选择一个更好的“中间”值。当原始列表初始时已部分排序的情况下，这种方法将特别有用。

Complexity Analysis of Quick Sort:

Time Complexity:

- Best Case: $\Omega(N \log N)$
The best-case scenario for quicksort occurs when the pivot chosen at each step divides the array into roughly equal halves.
In this case, the algorithm will make balanced partitions, leading to efficient sorting.
- Average Case: $\Theta(N \log N)$
Quicksort's average-case performance is usually very good in practice, making it one of the fastest sorting algorithms.
- Worst Case: $O(N^2)$
The worst-case scenario for quicksort occurs when the pivot at each step consistently results in highly unbalanced partitions. When the array is already sorted and the pivot is always chosen as the smallest or largest element. To mitigate the worst-case scenario, various techniques are used such as choosing a good pivot (e.g., median of three) and using Randomized algorithm (Randomized Quicksort) to shuffle the element before sorting.

Auxiliary Space: $O(1)$, if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Advantages of Quick Sort:

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort:

- It has a worst-case time complexity of $O(N^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Q. Choose the leftmost element as pivot, given the following list of numbers [14, 17, 13, 15, 19, 10, 3, 16, 9, 12] which answer shows the contents of the list after the second partitioning according to the quicksort algorithm? (D)

- A. [9, 3, 10, 13, 12]
- B. [9, 3, 10, 13, 12, 14]
- C. [9, 3, 10, 13, 12, 14, 17, 16, 15, 19]
- D. [9, 3, 10, 13, 12, 14, 19, 16, 15, 17]**

The first partitioning works on the entire list, and the second partitioning works on the left partition not the right. It's important to remember that quicksort works on the entire list and sorts it in place.

Q: Given the following list of numbers [1, 20, 11, 5, 2, 9, 16, 14, 13, 19] what would be the first pivot value using the median of 3 method? (B)

- A. 1
- B. 9**
- C. 16
- D. 19

although 16 would be the median of 1, 16, 19 the middle is at `len(list) // 2`.

Q: Which of the following sort algorithms are guaranteed to be $O(n \log n)$ even in the worst case? (C)

- A. Shell Sort
- B. Quick Sort
- C. Merge Sort**
- D. Insertion Sort

Merge Sort is the only guaranteed $O(n \log n)$ even in the worst case. The cost is that merge sort uses more memory.

4 Merge Sort

Merge sort is defined as a sorting algorithm that works by dividing an array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays back together to form the final sorted array.

In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

How does Merge Sort work?

Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted). Then the sorted subarrays are merged into one sorted array.

```
1 def mergeSort(arr):
2     if len(arr) > 1:
3         mid = len(arr)//2
4
5         L = arr[:mid] # Dividing the array elements
6         R = arr[mid:] # Into 2 halves
7
8         mergeSort(L) # Sorting the first half
9         mergeSort(R) # Sorting the second half
10
11     i = j = k = 0
12     # Copy data to temp arrays L[] and R[]
```

```

13     while i < len(L) and j < len(R):
14         if L[i] <= R[j]:
15             arr[k] = L[i]
16             i += 1
17         else:
18             arr[k] = R[j]
19             j += 1
20             k += 1
21
22     # Checking if any element was left
23     while i < len(L):
24         arr[k] = L[i]
25         i += 1
26         k += 1
27
28     while j < len(R):
29         arr[k] = R[j]
30         j += 1
31         k += 1
32
33
34 if __name__ == '__main__':
35     arr = [12, 11, 13, 5, 6, 7]
36     mergeSort(arr)
37     print(' '.join(map(str, arr)))
38 # Output: 5 6 7 11 12 13

```

Complexity Analysis of Merge Sort

Time Complexity: $O(N \log N)$, Merge Sort is a **recursive** algorithm and time complexity can be expressed as following recurrence relation.

$$T(n) = 2T(n/2) + \theta(n)$$

The above recurrence can be solved either using the Recurrence Tree method or the Master method. It falls in case II of the Master Method and the solution of the recurrence is $\theta(N \log N)$. The time complexity of Merge Sort is $\theta(N \log N)$ in all 3 cases (worst, average, and best) as merge sort always divides the array into two halves and takes linear time to merge two halves.

Auxiliary Space: $O(N)$, In merge sort all elements are copied into an auxiliary array. So **N auxiliary space is required for merge sort.**

Applications of Merge Sort:

- Sorting large datasets: Merge sort is particularly well-suited for sorting large datasets due to its guaranteed worst-case time complexity of $O(n \log n)$.
- **External sorting**: Merge sort is commonly used in external sorting, where the data to be sorted is too large to fit into memory.
- Custom sorting: Merge sort can be adapted to handle different input distributions, such as partially sorted, nearly sorted, or completely unsorted data.
- **Inversion Count Problem**: **Inversion Count** for an array indicates – how far (or close) the array is from being sorted. If the array is already sorted, then the inversion count is 0, but if

the array is sorted in reverse order, the inversion count is the maximum.

Advantages of Merge Sort:

- Stability: Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- Guaranteed worst-case performance: Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
- Parallelizable: Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

Drawbacks of Merge Sort:

- Space complexity: Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- Not in-place: Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Not always optimal for small datasets: For small datasets, Merge sort has a higher time complexity than some other sorting algorithms, such as insertion sort. This can result in slower performance for very small datasets.

Q: 给定排序列表 [21,1,26,45,29,28,2,9,16,49,39,27,43,34,46,40], 在归并排序的第 3 次递归调用时, 排序的是哪个子表? (B)

A: [16, 49, 39, 27, 43, 34, 46, 40] B: **[21, 1]** C: [21, 1, 26, 45] D: [21]

Remember mergesort doesn't work on the right half of the list until the left half is completely sorted.

Q: 排序数据同上, 归并排序中, 哪两个子表是最先归并的? (C)

A: [21, 1] and [26, 45] B: [1, 2, 9, 21, 26, 28, 29, 45] and [16, 27, 34, 39, 40, 43, 46, 49]

C: **[21] and [1]** D: [9] and [16]

The lists [21] and [1] are the first two base cases encountered by mergesort and will therefore be the first two lists merged.

归并排序如果自上而下是递归, 自下而上是迭代?

是的, 归并排序有两种常见的实现方式, 对应不同的思维方式:

自上而下 (递归实现)

- 思路: 分治法。把数组不断二分直到每个子数组长度为1, 然后两两合并。
- 过程: 递归地将数组拆分为两半; 合并排序好的左右子数组。
- 特点:
 - 写法直观, 易理解;

- 使用系统栈递归，空间复杂度为 $O(\log n)$ 。

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        result.append(left[i] if left[i] <= right[j] else right[j])
14        if left[i] <= right[j]:
15            i += 1
16        else:
17            j += 1
18    result.extend(left[i:])
19    result.extend(right[j:])
20    return result
```

自下而上（迭代实现）

- 思路：从最小粒度（每个元素是一个已排序段）开始，两两合并，不断扩大子数组的大小。
- 过程：
 1. 初始时，每个长度为1的元素看作已排序；
 2. 进行第一轮合并：长度为1的段合并成长度为2；
 3. 再进行第二轮：长度为2合并成长度为4，以此类推；
 4. 最后整段合并完成。
- 特点：无需递归；空间复杂度相同，但可以避免系统栈溢出。

```
1 def merge_sort_iterative(arr):
2     n = len(arr)
3     size = 1
4     while size < n:
5         for start in range(0, n, 2 * size):
6             mid = min(start + size, n)
7             end = min(start + 2 * size, n)
8             arr[start:end] = merge(arr[start:mid], arr[mid:end])
9         size *= 2
10    return arr
```

 总结比较：

实现方式	分治顺序	栈/循环	空间复杂度	特点
自上而下递归	先分后合	递归栈	$O(n + \log n)$	写法直观、易理解
自下而上迭代	先合后归	循环	$O(n)$	避免递归、适合大数组

如果你要分析归并排序的“一趟归并”效果，那通常是基于 **自下而上迭代实现**，因为那里面每一轮循环就是“归并一趟”。

5 Shell Sort

希尔排序主要是**插入排序**的一种变体。在插入排序中，我们只能将元素向前移动一个位置。当一个元素需要被移动很远时，会涉及到很多次移动。希尔排序的思想是允许远距离的元素交换。在希尔排序中，我们使数组对于较大的 h 值成为 h -sorted。我们不断减少 h 的值直到它变为1。如果一个数组的所有每隔 h 个元素的子列表都是排序的，则称该数组为 h 排序的。

Algorithm:

Step 1 – Start

Step 2 – Initialize the value of gap size. Example: h

Step 3 – Divide the list into smaller sub-part. Each must have equal intervals to h

Step 4 – **Sort these sub-lists using insertion sort**

Step 5 – Repeat this step 2 until the list is sorted.

Step 6 – Print a sorted list.

Step 7 – Stop.

```

1  def shellSort(arr, n):
2      # code here
3      gap = n // 2
4
5      while gap > 0:
6          j = gap
7          # Check the array in from left to right
8          # Till the last possible index of j
9          while j < n:
10             i = j - gap # This will keep help in maintain gap value
11
12             while i >= 0:
13                 # If value on right side is already greater than left side
value
14                 # We don't do swap else we swap
15                 if arr[i + gap] > arr[i]:
16                     break
17                 else:
18                     arr[i + gap], arr[i] = arr[i], arr[i + gap]
19
20                 i = i - gap # To check left side also
21                 # If the element present is greater than current element
22                 j += 1
23             gap = gap // 2

```

```

24
25
26 # driver to check the code
27 arr2 = [12, 34, 54, 2, 3]
28
29 shellSort(arr2, len(arr2))
30 print(' '.join(map(str, arr2)))
31
32 # Output: 2 3 12 34 54

```

Time Complexity: Time complexity of the above implementation of Shell sort is $O(n^2)$. In the above implementation, the gap is reduced by half in every iteration. There are many other ways to reduce gaps which leads to better time complexity. See [this](#) for more details.

Worst Case Complexity

The worst-case complexity for shell sort is $O(n^2)$

Shell Sort Applications

1. Replacement for insertion sort, where it takes a long time to complete a given task.
2. To call stack overhead we use shell sort.
3. when recursion exceeds a particular limit we use shell sort.
4. For medium to large-sized datasets.
5. In insertion sort to reduce the number of operations.

<https://en.wikipedia.org/wiki/Shellsort>

The running time of Shellsort is heavily dependent on the gap sequence it uses. For many practical variants, determining their time complexity remains an open problem.

Unlike **insertion sort**, **Shellsort is not a stable sort** since gapped insertions transport equal elements past one another and thus lose their original order. It is an **adaptive sorting algorithm** in that it executes faster when the input is partially sorted.

Stable sort algorithms sort equal elements in the same order that they appear in the input.

Q: Given the following list of numbers: [5, 16, 20, 12, 3, 8, 9, 17, 19, 7] Which answer illustrates the contents of the list after all swapping is complete for a gap size of 3? (A)

- A. [5, 3, 8, 7, 16, 19, 9, 17, 20, 12]
- B. [3, 7, 5, 8, 9, 12, 19, 16, 20, 17]
- C. [3, 5, 7, 8, 9, 12, 16, 17, 19, 20]
- D. [5, 16, 20, 3, 8, 12, 9, 17, 20, 7]

Each group of numbers represented by index positions 3 apart are sorted correctly.

6 Comparison sorts

在排序算法中，**稳定性是指相等元素的相对顺序是否在排序后保持不变**。换句话说，如果排序算法在排序过程中保持了相等元素的相对顺序，则称该算法是稳定的，否则是不稳定的。

对于判断一个排序算法是否稳定，一种常见的方法是观察交换操作。挨着交换（相邻元素交换）是稳定的，而隔着交换（跳跃式交换）可能会导致不稳定性。

Below is a table of [comparison sorts](#). A comparison sort cannot perform better than $O(n \log n)$ on average.

Name	Best	Average	Worst	Memory	Stable	Method	Other notes
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarian's Algorithm)
Timsort	n	$n \log n$	$n \log n$	n	Yes	Insertion & Merging	Makes $n-1$ comparisons when the data is already sorted or reverse sorted.
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	No	Partitioning	Quicksort is usually done in-place with $O(\log n)$ stack space.
Shellsort	$n \log n$	$n^{4/3}$	$n^{3/2}$	1	No	Insertion	Small code size.
Insertion sort	n	n^2	n^2	1	Yes	Insertion	$O(n + d)$, in the worst case over sequences that have d inversions.
Bubble sort	n	n^2	n^2	1	Yes	Exchanging	Tiny code size.
Selection sort	n^2	n^2	n^2	1	No	Selection	Stable with $O(n)$ extra space, when using linked lists, or when made as a variant of Insertion Sort instead of swapping the two items.

Highly tuned implementations use more sophisticated variants, such as [Timsort](#) (merge sort, insertion sort, and additional logic), used in [Android](#), [Java](#), and [Python](#), and [introsort](#) (quicksort and heapsort), used (in variant forms) in some [C++ sort](#) implementations and in [.NET](#).

编程题目

02299:Ultra-QuickSort

merge sort, <http://cs101.openjudge.cn/practice/02299/>

2 线性结构概论

What Are Linear Structures? 线性结构

线性结构（Linear Structures）是项的有序集合。一旦加入元素，它相对于前后元素的位置就固定了。

- 特征：具有两个端点（左/右、前/后、顶/底）。
- 分类依据：区分不同线性结构的关键在于项的添加和移除方式，尤其是操作发生的位置。

这些变化产生了计算机科学中一些最有用的数据结构。它们出现在许多算法中，并且可用于解决各种重要的问题。

线性结构，是一种逻辑结构，描述了元素按线性顺序排列的规则。常见的线性表存储方式有**数组**和**链表**，它们在不同场景下具有各自的优势和劣势。

数组是一种连续存储结构，它将线性表的元素按照一定的顺序依次存储在内存中的连续地址空间上。数组需要预先分配一定的内存空间，每个元素占用相同大小的内存空间，并可以通过索引来进行快速访问和操作元素。访问元素的时间复杂度为 $O(1)$ ，因为可以直接计算元素的内存地址。然而，插入和删除元素的时间复杂度较高，平均为 $O(n)$ ，因为需要移动其他元素来保持连续存储的特性。

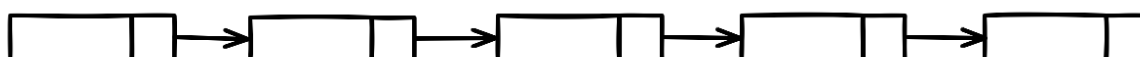
在Python中，`list` 更接近于数组的存储结构。是使用动态数组（Dynamic Array）实现的，而不是链表。**动态数组是一种连续的、固定大小的内存块**，可以在需要时自动调整大小。这使得 `list` 支持快速的随机访问和高效的尾部操作，例如附加（append）和弹出（pop）。

与链表不同，动态数组中的元素在内存中是连续存储的。这允许通过索引在 `list` 中的任何位置进行常数时间 $O(1)$ 的访问。此外，动态数组还具有较小的内存开销，因为它们不需要为每个元素存储额外的指针。

当需要在 `list` 的中间进行插入或删除操作时，动态数组需要进行元素的移动，因此这些操作的时间复杂度是线性的 $O(n)$ 。如果频繁地插入或删除元素，而不仅仅是在尾部进行操作，那么链表可能更适合，因为链表的插入和删除操作在平均情况下具有常数时间复杂度。

总结起来，Python 中的 `list` 是使用动态数组实现的，具有支持快速随机访问和高效尾部操作的优点。但是，如果需要频繁进行插入和删除操作，可能需要考虑使用链表或其他数据结构。

Python 中的 `list` 和 C++ 中的 STL（Standard Template Library）中的 `vector` 具有相似的实现和用法。`vector` 也是使用动态数组实现的，提供了类似于 `list` 的功能，包括随机访问、尾部插入和删除等操作。



链表是一种存储结构，它是线性表的链式存储方式。链表通过节点的相互链接来实现元素的存储。每个节点包含元素本身以及指向下一个节点的指针。链表的插入和删除操作非常高效，时间复杂度为 $O(1)$ ，因为只需要调整节点的指针。然而，访问元素的时间复杂度较高，平均为 $O(n)$ ，因为必须从头节点开始遍历链表直到找到目标元素。

选择使用数组还是链表作为存储方式取决于具体问题的需求和限制。**如果需要频繁进行随机访问操作，数组是更好的选择**。如果需要频繁进行插入和删除操作，链表更适合。通过了解它们的特点和性能，可以根据实际情况做出选择。

存储方式对比：数组 vs 链表

线性表有两种主要的物理存储方式：

特性	顺序存储 (顺序表/数组)	链式存储 (链表)
底层实现	连续地址空间	离散节点 + 指针引用
访问效率	$O(1)$ (支持随机访问)	$O(n)$ (需从头遍历)
增删效率	$O(n)$ (涉及元素移动)	$O(1)$ (仅需修改指针)
空间开销	预分配, 可能浪费或需扩容	动态分配, 每个节点需额外存储指针
Python对应	内置 <code>list</code> (基于动态数组)	需手动实现 <code>Node</code> 类

What is a Stack? 栈

栈（有时被称为“压入栈”）是一种有序的项目集合，其中新项目的添加和现有项目的移除总是在同一端进行。这一端通常被称为“顶端”。与顶端相对的另一端被称为“基底”。

栈的基底是重要的，因为存储在栈中靠近基底的项目代表了在栈中最久的项目。最近添加的项目是准备首先被移除的那个。这种排序原则有时被称为**LIFO（后进先出）**。它提供了一种基于项目在集合中停留时间长短的排序。较新的项目接近顶端，而较旧的项目接近基底。

日常生活中有许多栈的例子。几乎任何自助餐厅都有一个托盘或盘子的堆栈，你可以从顶部取走一个，为下一位排队的顾客露出一个新的托盘或盘子。

What Is a Queue? 队列

队列是一种有序的项目集合，其中新项目的添加发生在一端，这端被称为“尾部”，而现有项目的移除则发生在另一端，通常称为“前端”。当元素进入队列时，它从尾部开始，向前端移动，直到成为下一个要被移除的元素为止。

队列中最新增加的项目必须在集合的末尾等待。在集合中最久的项目位于前端。这种排序原则有时被称为**FIFO（先进先出）**，也称为“先来先服务”。

线性结构操作复杂度表

结构	操作	复杂度 (平均)	推荐 Python 实现
栈 (Stack)	Push / Pop	$O(1)$	<code>list</code> (使用 <code>append/pop</code>)
队列 (Queue)	Enqueue / Dequeue	$O(1)$	<code>collections.deque</code>
单链表	在头部增删	$O(1)$	手动 <code>Node</code> 类
双向链表	在两端增删	$O(1)$	<code>collections.deque</code>
顺序表	中间插入/删除	$O(n)$	<code>list</code>

学习建议：掌握栈和队列的关键不在于写出它们的类定义，而在于能够敏锐察觉问题中的 **LIFO** (回溯、倒序处理) 或 **FIFO** (排队、按序处理) 特征，并选择合适的 Python 工具库实现。

3 链表 (Linked List)

链表 (Linked List) 是一种常见的数据结构，用于存储和组织数据。它由一系列节点组成，每个节点包含一个数据元素和一个指向下一个节点（或前一个节点）的指针。它的特点是插入与删除数据十分方便，但寻找与读取数据的表现欠佳。



在链表中，每个节点都包含两部分：

1. 数据元素（或数据项）：这是节点存储的实际数据。可以是任何数据类型，例如整数、字符串、对象等。
2. 指针（或引用）：该指针指向链表中的下一个节点（或前一个节点）。它们用于建立节点之间的连接关系，从而形成链表的结构。

根据指针的类型和连接方式，链表可以分为不同类型，包括：

1. 单向链表：每个节点只有一个指针，指向下一个节点。链表的头部指针指向第一个节点，而最后一个节点的指针为空（指向 `None`）。
2. 双向链表：每个节点有两个指针，一个指向前一个节点，一个指向后一个节点。双向链表可以从头部或尾部开始遍历，并且可以在任意位置插入或删除节点。
3. 循环链表：最后一个节点的指针指向链表的头部，形成一个环形结构。循环链表可以从任意节点开始遍历，并且可以无限地循环下去。

与数组的区别

链表和数组都可用于存储数据。与链表不同，数组将所有元素按次序依次存储。不同的存储结构令它们有了不同的优势：

链表因其链状的结构，能方便地删除、插入数据，操作次数是 $O(1)$ 。但也因为这样，寻找、读取数据的效率不如数组高，在随机访问数据中的操作次数是 $O(n)$ 。

数组可以方便地寻找并读取数据，在随机访问中操作次数是 $O(1)$ 。但删除、插入的操作次数是 $O(n)$ 次。

```
1 # Definition for singly-linked list.
2 class ListNode:
3     def __init__(self, x):
4         self.val = x
5         self.next = None
```

```

1 class DLinkedNode:
2     """双向链表的节点类"""
3     def __init__(self, key=0, value=0):
4         self.key = key
5         self.value = value
6         self.prev = None
7         self.next = None

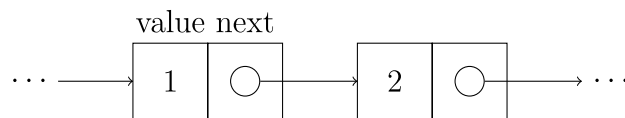
```

链表相对于数组的一个重要特点是，链表的大小可以动态地增长或缩小，而不需要预先定义固定的大小。这使得链表在需要频繁插入和删除元素的场景中更加灵活。

然而，链表的访问和搜索操作相对较慢，因为需要遍历整个链表才能找到目标节点。与数组相比，链表的优势在于插入和删除操作的效率较高，尤其是在操作头部或尾部节点时。因此，链表在需要频繁插入和删除元素而不关心随机访问的情况下，是一种常用的数据结构。

3.1 单向链表 (Singly Linked List)

单向链表中包含数据域和指针域，其中数据域用于存放数据，指针域用来连接当前结点和下一节点。



常见操作

- 插入操作：可以在链表的头部、尾部或中间插入新节点。
- 删除操作：可以删除链表中的某个节点。
- 遍历操作：从头部开始，逐一访问链表中的每个节点。
- 查找操作：根据节点数据查找对应的节点。

单向链表实现1: 尾插法

```

1 class Node:
2     def __init__(self, value):
3         self.value = value
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9
10    def insert(self, value):
11        new_node = Node(value)
12        if self.head is None:
13            self.head = new_node
14        else:
15            current = self.head

```

```

16         while current.next:
17             current = current.next
18             current.next = new_node
19
20     def delete(self, value):
21         if self.head is None:
22             return
23
24         if self.head.value == value:
25             self.head = self.head.next
26         else:
27             current = self.head
28             while current.next:
29                 if current.next.value == value:
30                     current.next = current.next.next
31                     break
32             current = current.next
33
34     def display(self):
35         current = self.head
36         while current:
37             print(current.value, end=" ")
38             current = current.next
39         print()
40
41     # 使用示例
42     linked_list = LinkedList()
43     linked_list.insert(1)
44     linked_list.insert(2)
45     linked_list.insert(3)
46     linked_list.display() # 输出: 1 2 3
47     linked_list.delete(2)
48     linked_list.display() # 输出: 1 3

```

单向链表实现2, 保存了链表的长度

```

1 class LinkedList:
2     class Node:
3         def __init__(self, data, next=None):
4             self.data = data # Store data
5             self.next = next # Point to the next node
6
7     def __init__(self):
8         self.head = None # Initialize head as None
9         self.tail = None # Initialize tail as None
10        self.size = 0 # Initialize size to 0
11
12    def print(self):
13        ptr = self.head
14        while ptr is not None:
15            if ptr != self.head: # Avoid printing a comma before the first
element
16                print(', ', end='')

```

```

17         print(ptr.data, end='')
18         ptr = ptr.next
19     print() # Move to the next line after printing all elements
20
21     def insert_after(self, p, data):
22         nd = LinkList.Node(data)
23         if p is None: # If p is None, insert at the beginning
24             self.pushFront(data)
25         else:
26             nd.next = p.next
27             p.next = nd
28             if p == self.tail: # Update tail if necessary
29                 self.tail = nd
30             self.size += 1
31
32     def delete_after(self, p):
33         if p is None or p.next is None:
34             return # Nothing to delete
35         if self.tail is p.next: # Update tail if necessary
36             self.tail = p
37         p.next = p.next.next
38         self.size -= 1
39
40     def popFront(self):
41         if self.head is None:
42             raise Exception("Popping front from empty link list.")
43         else:
44             data = self.head.data
45             self.head = self.head.next
46             self.size -= 1
47             if self.size == 0:
48                 self.tail = None
49             return data
50
51     def pushFront(self, data):
52         nd = LinkList.Node(data, self.head)
53         self.head = nd
54         if self.size == 0:
55             self.tail = nd
56         self.size += 1
57
58     def pushBack(self, data):
59         if self.size == 0:
60             self.pushFront(data)
61         else:
62             self.insert_after(self.tail, data)
63
64     def clear(self):
65         self.head = None
66         self.tail = None
67         self.size = 0
68
69     def __iter__(self):
70         self.ptr = self.head
71         return self

```

```

72
73     def __next__(self):
74         if self.ptr is None:
75             raise StopIteration()
76         else:
77             data = self.ptr.data
78             self.ptr = self.ptr.next
79             return data
80
81 # 示例用法
82 if __name__ == "__main__":
83     ll = LinkedList()
84     ll.pushFront(1)
85     ll.pushFront(2)
86     ll.pushBack(3)
87     ll.print() # 应该输出: 2,1,3
88     ll.delete_after(ll.head) # 删除第二个元素 (1)
89     ll.print() # 应该输出: 2,3
90     print(f"Pop Front: {ll.popFront()}") # 应该输出: Pop Front: 2
91     ll.print() # 应该输出: 3

```

单链表的应用

- 动态内存管理：链表可以灵活地分配内存空间，特别适用于内存空间不固定的场景。
- 实现队列和栈：链表能够有效地支持栈（LIFO）和队列（FIFO）的实现，因为其在插入和删除操作上有优势。

Q：数组和链表哪个随机位置插入快？

在随机位置插入元素时，数组（Array）和链表（Linked List）的时间复杂度在理论上都是 $O(n)$ ，但在实际应用中，数组通常比链表快。

这个结论可能违背很多人的直觉（直觉上认为链表插入是 $O(1)$ ），我们需要从“查找”和“操作”两个步骤来拆解分析：

1. 过程拆解

在一个随机位置（假设是第 i 个位置）插入元素，分为两步：

第一步：找到位置（查找）

- 数组：支持随机访问，通过下标计算偏移量，耗时 $O(1)$ 。
- 链表：不支持随机访问，必须从头节点开始逐个跳转，耗时 $O(n)$ 。

第二步：执行插入（操作）

- 数组：为了给新元素腾出空间，必须将插入点之后的所有元素向后移动一位，耗时 $O(n)$ 。
- 链表：只要找到了节点，只需修改 2-3 个指针的指向即可，耗时 $O(1)$ 。

综合耗时

- 数组 = $O(1)$ (找) + $O(n)$ (搬运) = $O(n)$
- 链表 = $O(n)$ (找) + $O(1)$ (改指针) = $O(n)$

2. 为什么实际开发中数组通常更快？

虽然理论复杂度相同，但由于底层物理结构的差异，数组在现代计算机体系下表现更好：

1. CPU 缓存友好性（空间局部性）：

- 数组在内存中是连续存储的。CPU 预取数据时会把相邻的内存块一起加载进高速缓存（Cache）。在移动元素时，由于数据连续，处理速度极快。
- 链表的节点在内存中是离散分布的。每访问一个节点都可能导致“缓存未命中”（Cache Miss），需要去慢速的内存（RAM）中读取，性能差距可达几十倍。

2. 内存分配开销：

- 数组通常是预先分配好的一整块空间。
- 链表每插入一个新元素，通常需要调用系统指令进行一次动态内存分配（如 `malloc` 或 `new`），这是一个相对昂贵的操作。

3. 批量移动的优化：

- 现代 CPU 指令集（如 SIMD）和编程语言底层（如 C 语言的 `memmove`）对连续内存的拷贝做了极致优化，移动一小段数组的速度非常惊人。

3. 什么时候链表更快？

只有在以下特定场景下，链表才会体现出优势：

1. 你已经拿到了插入位置的指针/迭代器：

- 如果你不需要“查找”，而是直接在当前光标处插入（比如实现一个文本编辑器），链表是真正的 $O(1)$ ，而数组依然要 $O(n)$ 搬运。

2. 元素对象非常庞大且不使用指针存储：

- 如果数组中存的是非常大的结构体（不是指针），移动元素的开销会变得巨大。而链表只需要交换指针。

3. 频繁在头部插入：

- 对于链表，头部插入是 $O(1)$ 。
- 对于数组，头部插入每次都要移动整个数组。

总结

- 如果你是通过索引（第 i 个位置）插入：数组通常更快（得益于缓存和连续内存）。
- 如果你已经定位到了节点：链表快得多（只需改指针）。
- 如果你是在头部插入：链表快。
- 如果你是在尾部插入：两者都快（数组在不触发扩容时是 $O(1)$ ）。

Q：给出python代码，实际跑起来看看。

为了对比数组（Python 中的 `list`）和链表在随机位置插入的实际性能，我们可以编写一个脚本进行测试。

1. 准备工作

在 Python 中：

- 数组：直接使用内置的 `list`。它是基于动态数组实现的。
- 链表：Python 没有内置的链表，我们需要手动实现一个简单的单链表。

2. Python 性能测试代码

```
1  import time
2  import random
3
4  # --- 1. 定义链表节点 ---
5  class Node:
6      def __init__(self, data):
7          self.data = data
8          self.next = None
9
10 class LinkedList:
11     def __init__(self, initial_data):
12         self.head = Node(None)
13         curr = self.head
14         for d in initial_data:
15             curr.next = Node(d)
16             curr = curr.next
17         self.size = len(initial_data)
18
19     def insert_at(self, index, data):
20         if index < 0 or index > self.size:
21             return
22
23         curr = self.head
24         # 链表必须从头开始找，耗时 O(n)
25         for _ in range(index):
26             curr = curr.next
27
28         # 插入操作本身是 O(1)
29         new_node = Node(data)
30         new_node.next = curr.next
31         curr.next = new_node
32         self.size += 1
33
34 # --- 2. 测试函数 ---
35 def test_performance():
36     # 数据规模：10万个元素
37     N = 100000
38     # 插入次数：1000次（不宜过多，因为链表太慢）
39     insert_count = 1000
40
41     initial_data = list(range(N))
42
43     # --- 测试数组 (Python List) ---
44     arr = list(initial_data)
45     start_time = time.time()
46     for _ in range(insert_count):
```

```

47     pos = random.randint(0, len(arr) - 1)
48     arr.insert(pos, 999) # 数组插入: 查找O(1) + 搬运O(n)
49     array_time = time.time() - start_time
50     print(f"数组 (List) 插入 {insert_count} 次总耗时: {array_time:.4f}
    秒")
51
52     # --- 测试链表 (LinkedList) ---
53     ll = LinkedList(initial_data)
54     start_time = time.time()
55     for _ in range(insert_count):
56         pos = random.randint(0, ll.size - 1)
57         ll.insert_at(pos, 999) # 链表插入: 查找O(n) + 更改指针O(1)
58     ll_time = time.time() - start_time
59     print(f"链表 (LinkedList) 插入 {insert_count} 次总耗时: {ll_time:.4f}
    秒")
60
61     print(f"\n结论: 在这个测试中, 数组比链表快 {ll_time / array_time:.2f}
    倍")
62
63 if __name__ == "__main__":
64     test_performance()

```

3. 运行结果示例

在普通笔记本电脑上运行, 结果通常如下:

```

1  数组 (List) 插入 1000 次总耗时: 0.0450 秒
2  链表 (LinkedList) 插入 1000 次总耗时: 1.2500 秒
3
4  结论: 在这个测试中, 数组比链表快 27.78 倍

```

4. 深度分析: 为什么数组反而快这么多?

虽然理论上两者的复杂度都是 $O(n)$, 但数组在实际运行中几乎总是碾压链表, 原因有三:

1. C 语言级别的优化:

Python 的 `list.insert` 是用 **C 语言** 编写的。在数组内部移动元素 (搬运内存块) 时, 它使用的是极其高效的 `memmove` 操作。而我们手写的链表是用 **Python 解释器** 运行的, 每一步 `for` 循环找节点都要经过 Python 的对象寻址, 效率极低。

2. CPU 缓存 (Cache) 命中率:

- **数组**: 所有元素在内存里挨在一起。当 CPU 读取 `arr[100]` 时, 它会顺便把 `arr[101], arr[102]...` 都加载进缓存。
- **链表**: 节点是散落在内存各处的。CPU 每找下一个节点, 很大率产生 **Cache Miss** (缓存未命中), 必须去慢速的内存 (RAM) 里取数据。

3. 内存开销:

- 数组只存数据。
- 链表每个节点还要额外存一个 `next` 指针, 并且每个节点都是一个独立的 Python 对象, 对象头的额外开销 (Overhead) 非常巨大。

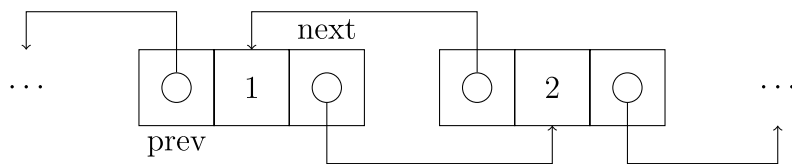
5. 什么时候链表才会变快?

如果你想看到链表比数组快, 必须满足以下极端的条件:

1. 不计算查找时间：比如你已经手握某个节点的指针，直接在它后面插。
2. 巨大的元素对象：如果每个元素是一个 100MB 的大对象（且不是指针引用），数组移动起来会非常吃力，而链表只需要改指针。
3. 在头部插入：你可以试着把代码中的 `random.randint` 改成 `pos = 0`。你会发现数组的速度会大幅下降，而链表（如果维护了头指针）会变快。

3.2 双向链表 (Doubly Linked List)

双向链表中同样有数据域和指针域。不同之处在于，指针域有左右（或上一个、下一个）之分，用来连接上一个结点、当前结点、下一个结点。



常见操作

- 插入操作：可以在链表的头部、尾部或中间插入新节点，插入操作需要同时调整 `next` 和 `prev` 指针。
- 删除操作：可以删除链表中的某个节点，删除操作需要更新前后节点的指针。
- 遍历操作：可以从头到尾或从尾到头进行遍历。

双向链表代码实现

```
1 class Node:
2     def __init__(self, data):
3         self.data = data # 节点数据
4         self.next = None # 指向下一个节点
5         self.prev = None # 指向前一个节点
6
7 class DoublyLinkedList:
8     def __init__(self):
9         self.head = None # 链表头部
10        self.tail = None # 链表尾部
11
12    # 在链表尾部添加节点
13    def append(self, data):
14        new_node = Node(data)
15        if not self.head: # 如果链表为空
16            self.head = new_node
17            self.tail = new_node
18        else:
19            self.tail.next = new_node
20            new_node.prev = self.tail
21            self.tail = new_node
```

```

22
23     # 在链表头部添加节点
24     def prepend(self, data):
25         new_node = Node(data)
26         if not self.head: # 如果链表为空
27             self.head = new_node
28             self.tail = new_node
29         else:
30             new_node.next = self.head
31             self.head.prev = new_node
32             self.head = new_node
33
34     # 删除链表中的指定节点
35     def delete(self, node):
36         if not self.head: # 链表为空
37             return
38
39         if node == self.head: # 删除头部节点
40             self.head = node.next
41             if self.head: # 如果链表非空
42                 self.head.prev = None
43         elif node == self.tail: # 删除尾部节点
44             self.tail = node.prev
45             if self.tail: # 如果链表非空
46                 self.tail.next = None
47         else: # 删除中间节点
48             node.prev.next = node.next
49             node.next.prev = node.prev
50
51         node = None # 删除节点
52
53     # 打印链表中的所有元素，从头到尾
54     def print_list(self):
55         current = self.head
56         while current:
57             print(current.data, end=" <-> ")
58             current = current.next
59         print("None")
60
61     # 打印链表中的所有元素，从尾到头
62     def print_reverse(self):
63         current = self.tail
64         while current:
65             print(current.data, end=" <-> ")
66             current = current.prev
67         print("None")
68
69     # 创建双向链表对象
70     dll = DoublyLinkedList()
71
72     # 添加节点
73     dll.append(10)
74     dll.append(20)
75     dll.append(30)
76

```

```

77 # 在头部添加节点
78 dll.prepend(5)
79
80 # 打印链表
81 print("从头到尾打印: ")
82 dll.print_list() # 5 <--> 10 <--> 20 <--> 30 <--> None
83
84 # 打印链表（逆序）
85 print("从尾到头打印: ")
86 dll.print_reverse() # 30 <--> 20 <--> 10 <--> 5 <--> None
87
88 # 删除节点
89 dll.delete(dll.head.next) # 删除第二个节点（数据为10）
90
91 # 打印链表
92 print("删除一个节点后，链表为: ")
93 dll.print_list() # 5 <--> 20 <--> 30 <--> None
94

```

- **append**：将新节点添加到链表的尾部。
- **prepend**：将新节点添加到链表的头部。
- **delete**：删除链表中的指定节点（无论是头节点、尾节点还是中间节点）。
- **print_list**：从头到尾打印链表中的所有节点。
- **print_reverse**：从尾到头打印链表中的所有节点。

双向链表相对于单向链表的优势在于它能实现双向遍历，使得在某些操作上（例如反向遍历、删除特定节点等）更加高效。

双链表的应用

- 双向遍历：由于双链表可以从头到尾或从尾到头遍历，因此在某些需要双向遍历的数据结构（如浏览器历史记录、操作系统任务调度等）中非常有用。
- 实现双端队列（Deque）：双链表非常适合用于双端队列的实现，可以在队头和队尾都进行快速的插入和删除。
- 内存管理和垃圾回收：双链表用于管理动态内存块，常见于操作系统的内存管理和垃圾回收机制中。

示例M1472.设计浏览器历史记录

Doubly-linked list, <https://leetcode.cn/problems/design-browser-history/>

你有一个只支持单个标签页的浏览器，最开始你浏览的网页是 `homepage`，你可以访问其他的网站 `url`，也可以在浏览历史中后退 `steps` 步或前进 `steps` 步。

请你实现 `BrowserHistory` 类：

- `BrowserHistory(string homepage)`，用 `homepage` 初始化浏览器类。
- `void visit(string url)` 从当前页跳转访问 `url` 对应的页面。执行此操作会把浏览历史

史前进的记录全部删除。

- `string back(int steps)` 在浏览历史中后退 `steps` 步。如果你只能在浏览历史中后退至多 `x` 步且 `steps > x` , 那么你只后退 `x` 步。请返回后退 至多 `steps` 步以后的 `url` 。
- `string forward(int steps)` 在浏览历史中前进 `steps` 步。如果你只能在浏览历史中前进至多 `x` 步且 `steps > x` , 那么你只前进 `x` 步。请返回前进 至多 `steps` 步以后的 `url` 。

```
1 class ListNode:
2     def __init__(self, url: str):
3         self.url = url
4         self.prev = None
5         self.next = None
6
7 class BrowserHistory:
8     def __init__(self, homepage: str):
9         self.current = ListNode(homepage)
10
11     def visit(self, url: str) -> None:
12         new_node = ListNode(url)
13         self.current.next = new_node
14         new_node.prev = self.current
15         self.current = new_node
16
17     def back(self, steps: int) -> str:
18         while steps > 0 and self.current.prev is not None:
19             self.current = self.current.prev
20             steps -= 1
21         return self.current.url
22
23     def forward(self, steps: int) -> str:
24         while steps > 0 and self.current.next is not None:
25             self.current = self.current.next
26             steps -= 1
27         return self.current.url
28
29 if __name__ == "__main__":
30     browserHistory = BrowserHistory("leetcode.com")
31     browserHistory.visit("google.com")
32     browserHistory.visit("facebook.com")
33     browserHistory.visit("youtube.com")
34     print(browserHistory.back(1)) # facebook.com
35     print(browserHistory.back(1)) # google.com
36     print(browserHistory.forward(1)) # facebook.com
37     browserHistory.visit("linkedin.com")
38     print(browserHistory.forward(2)) # linkedin.com
39     print(browserHistory.back(2)) # google.com
40     print(browserHistory.back(7)) # leetcode.com
41
```

单链表与双链表的对比

特性	单链表	双链表
指针数量	每个节点一个指针，指向下一个节点	每个节点两个指针，分别指向前后节点
访问方向	只能从头到尾访问	可以从头到尾或从尾到头访问
内存开销	较低，仅需存储一个指针	较高，需要存储两个指针
插入/删除效率	在头部插入删除高效，但中间插入删除较慢	在任意位置插入删除较高效
操作复杂度	操作简单，适合轻量级应用	操作复杂，适用于双向操作场景
应用场景	动态内存管理，队列、栈实现	双端队列实现，双向遍历等

- 单链表 适用于动态内存管理和需要简单数据操作的场景，其操作效率相对较低，特别是在中间插入和删除时。
- 双链表 通过提供双向指针，增强了操作的灵活性，适用于需要双向遍历和高效插入删除的场景，如双端队列、浏览器历史记录等。
- 两者的选择应根据具体应用场景而定。如果需要简单的线性遍历和动态插入，单链表即可满足需求；而如果涉及到双向操作和复杂的内存管理，双链表则更加合适。

链表结构是基础数据结构之一，理解其操作和算法对于深入学习更复杂的算法和数据结构具有重要意义。

3.3 循环链表 (Circular Linked List)

将链表的头尾连接起来，链表就变成了循环链表。由于链表首尾相连，在插入数据时需要判断原链表是否为空：为空则自身循环，不为空则正常插入数据。

然而这样会导致访问最后一个结点时需要 $O(n)$ 的时间，所以我们可以写出仅设尾指针的循环链表。

```
1 class CircleLinkedList:
2     class Node:
3         def __init__(self, data, next=None):
4             self.data = data
5             self.next = next
6
7     def __init__(self):
8         self.tail = None # 尾指针，指向最后一个节点
9         self.size = 0 # 链表大小
10
11    def is_empty(self):
12        """检查链表是否为空"""
13        return self.size == 0
14
15    def pushFront(self, data):
```

```

16         """在链表头部插入元素"""
17         nd = CircleLinkedList.Node(data)
18         if self.is_empty():
19             self.tail = nd
20             nd.next = self.tail # 自己指向自己形成环
21         else:
22             nd.next = self.tail.next # 新节点指向当前头节点
23             self.tail.next = nd # 当前尾节点指向新节点
24         self.size += 1
25
26     def pushBack(self, data):
27         """在链表尾部插入元素"""
28         nd = CircleLinkedList.Node(data)
29         if self.is_empty():
30             self.tail = nd
31             nd.next = self.tail # 自己指向自己形成环
32         else:
33             nd.next = self.tail.next # 新节点指向当前头节点
34             self.tail.next = nd # 当前尾节点指向新节点
35             self.tail = nd # 更新尾指针
36         self.size += 1
37
38     def popFront(self):
39         """移除并返回链表头部元素"""
40         if self.is_empty():
41             return None
42         else:
43             old_head = self.tail.next
44             if self.size == 1:
45                 self.tail = None # 如果只有一个元素，更新尾指针为None
46             else:
47                 self.tail.next = old_head.next # 跳过旧头节点
48             self.size -= 1
49             return old_head.data
50
51     def popBack(self):
52         """移除并返回链表尾部元素"""
53         if self.is_empty():
54             return None
55         elif self.size == 1:
56             data = self.tail.data
57             self.tail = None
58             self.size -= 1
59             return data
60         else:
61             prev = self.tail
62             while prev.next != self.tail: # 找到倒数第二个节点
63                 prev = prev.next
64             data = self.tail.data
65             prev.next = self.tail.next # 跳过尾节点
66             self.tail = prev # 更新尾指针
67             self.size -= 1
68             return data
69
70     def printList(self):

```

```

71         """打印链表中的所有元素"""
72         if self.is_empty():
73             print('Empty!')
74         else:
75             ptr = self.tail.next
76             while True:
77                 print(ptr.data, end=', ' if ptr != self.tail else '\n')
78                 if ptr == self.tail:
79                     break
80                 ptr = ptr.next
81
82     # 示例用法
83     if __name__ == "__main__":
84         clist = CircleLinkedList()
85
86         print("Pushing elements to front:")
87         for i in range(3):
88             clist.pushFront(i)
89             clist.printList() # 应该依次输出: 0, 1,0, 2,1,0,
90
91         print("Pushing elements to back:")
92         for i in range(3, 6):
93             clist.pushBack(i)
94             clist.printList() # 应该依次输出: 2,1,0,3, 2,1,0,3,4, 2,1,0,3,4,5,
95
96         print("Popping from front:")
97         for _ in range(3):
98             print(f"Popped: {clist.popFront()}")
99             clist.printList() # 应该依次输出: 2,1,0,3,4,5, 1,0,3,4,5, 0,3,4,5,
100
101         print("Popping from back:")
102         for _ in range(3):
103             print(f"Popped: {clist.popBack()}")
104             clist.printList() # 应该依次输出: 5, 3,4, 5, 4, 3, Empty!

```

3.4 核心算法技巧

1 链表反转 (Reverse Linked List)

链表反转是一个经典的算法，它将链表中的节点顺序反转，使得原本指向下一个节点的指针指向前一个节点。该操作在处理栈或队列时非常有用。

单链表反转算法

```

1     # Definition for singly-linked list.
2     class ListNode:
3         def __init__(self, val=0, next=None):
4             self.val = val
5             self.next = next
6
7     def reverse_linked_list(head: ListNode) -> ListNode:
8         prev = None

```

```

9     curr = head
10    while curr is not None:
11        next_node = curr.next # 暂存当前节点的下一个节点
12        curr.next = prev     # 将当前节点的下一个节点指向前一个节点
13        prev = curr          # 前一个节点变为当前节点
14        curr = next_node     # 当前节点变更为原先的下一个节点
15    return prev
16

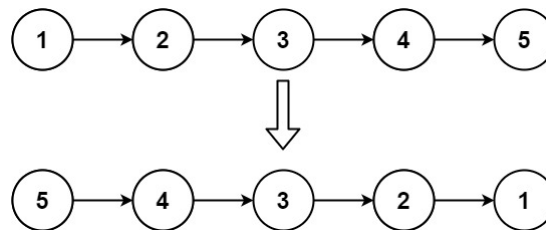
```

示例E206.反转链表

linked list, <https://leetcode.cn/problems/reverse-linked-list/>

给你单链表的头节点 `head`，请你反转链表，并返回反转后的链表。

示例 1:

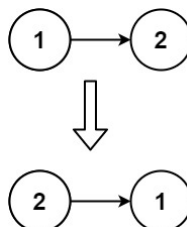


```

1  输入: head = [1,2,3,4,5]
2  输出: [5,4,3,2,1]

```

示例 2:



```

1  输入: head = [1,2]
2  输出: [2,1]

```

示例 3:

```

1  输入: head = []
2  输出: []

```

提示:

- 链表中节点的数目范围是 `[0, 5000]`
- `-5000 <= Node.val <= 5000`

```

1  # Definition for singly-linked list.
2  # class ListNode:
3  #     def __init__(self, val=0, next=None):
4  #         self.val = val
5  #         self.next = next
6  class Solution:
7      def reverseList(self, head: Optional[ListNode]) ->
Optional[ListNode]:
8          pre = None
9          current = head
10         while current:
11             next_node = current.next
12             current.next = pre
13             pre = current
14             current = next_node
15
16         return pre
17

```

2 合并两个排序的链表

合并两个已经排序的链表是一种常见的操作，特别是在归并排序中。

合并两个排序链表

```

1  def merge_sorted_lists(l1, l2):
2      dummy = Node(0)
3      tail = dummy
4      while l1 and l2:
5          if l1.data < l2.data:
6              tail.next = l1
7              l1 = l1.next
8          else:
9              tail.next = l2
10             l2 = l2.next
11         tail = tail.next
12     if l1:
13         tail.next = l1
14     else:
15         tail.next = l2
16     return dummy.next

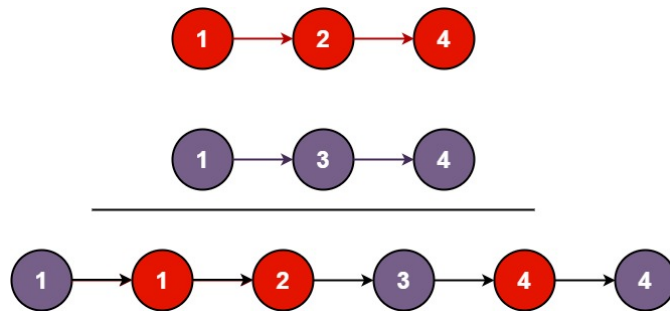
```

示例E21.合并两个有序链表

<https://leetcode.cn/problems/merge-two-sorted-lists/>

将两个升序链表合并为一个新的 升序 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1:



```
1 输入: l1 = [1,2,4], l2 = [1,3,4]
2 输出: [1,1,2,3,4,4]
```

示例 2:

```
1 输入: l1 = [], l2 = []
2 输出: []
```

示例 3:

```
1 输入: l1 = [], l2 = [0]
2 输出: [0]
```

提示:

- 两个链表的节点数目范围是 `[0, 50]`
- `-100 <= Node.val <= 100`
- `l1` 和 `l2` 均按 非递减顺序 排列

```
1  # Definition for singly-linked list.
2  # class ListNode:
3  #     def __init__(self, val=0, next=None):
4  #         self.val = val
5  #         self.next = next
6
7  class Solution:
8      def mergeTwoLists(self, list1: Optional[ListNode], list2:
Optional[ListNode]) -> Optional[ListNode]:
9          # 创建一个哨兵节点 (dummy node), 简化边界条件处理
10         prehead = ListNode(-200)
11         prev = prehead
12
13         # 遍历两个链表直到其中一个为空
```

```

14         while list1 and list2:
15             if list1.val <= list2.val:
16                 prev.next = list1
17                 list1 = list1.next
18             else:
19                 prev.next = list2
20                 list2 = list2.next
21             prev = prev.next
22
23         # 连接还未遍历完的那个链表
24         prev.next = list1 if list1 is not None else list2
25
26         # 返回合并后的链表，跳过哨兵节点
27         return prehead.next

```

3 查找链表的中间节点

通过**快慢指针**的方法，可以在 $O(n)$ 的时间复杂度内找到链表的中间节点。

查找中间节点

```

1 def find_middle_node(head):
2     slow = fast = head
3     while fast and fast.next:
4         slow = slow.next
5         fast = fast.next.next
6     return slow

```

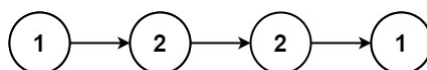
示例E234.回文链表

linked-list, <https://leetcode.cn/problems/palindrome-linked-list/>

给你一个单链表的头节点 `head`，请你判断该链表是否为

回文链表（回文 序列是向前和向后读都相同的序列。如果是，返回 `true`；否则，返回 `false`）。

示例 1:

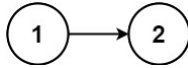


```

1 输入: head = [1,2,2,1]
2 输出: true

```

示例 2:



```
1 输入: head = [1,2]
2 输出: false
```

提示:

- 链表中节点数目在范围 $[1, 10^5]$ 内
- $0 \leq \text{Node.val} \leq 9$

进阶: 你能否用 $O(n)$ 时间复杂度和 $O(1)$ 空间复杂度解决此题?

快慢指针查找链表的中间节点

```
1  # Definition for singly-linked list.
2  class ListNode:
3      def __init__(self, val=0, next=None):
4          self.val = val
5          self.next = next
6  class Solution:
7      def isPalindrome(self, head: Optional[ListNode]) -> bool:
8          if not head or not head.next:
9              return True
10
11         # 1. 使用快慢指针找到链表的中点
12         slow, fast = head, head
13         while fast and fast.next:
14             slow = slow.next
15             fast = fast.next.next
16
17         # 2. 反转链表的后半部分
18         prev = None
19         while slow:
20             next_node = slow.next
21             slow.next = prev
22             prev = slow
23             slow = next_node
24
25         # 3. 对比前半部分和反转后的后半部分
26         left, right = head, prev
27         while right: # right 是反转后的链表的头
28             if left.val != right.val:
29                 return False
30             left = left.next
31             right = right.next
32
33         return True
34
```

4 其他示例

示例20.删除链表元素

<http://dsbpython.openjudge.cn/dspythonbook/P0020/>

程序填空，删除链表元素

```
1 class Node:
2     def __init__(self, data, next=None):
3         self.data, self.next = data, next
4 class LinkedList: #循环链表
5     def __init__(self):
6         self.tail = None
7         self.size = 0
8     def isEmpty(self):
9         return self.size == 0
10    def pushFront(self, data):
11        nd = Node(data)
12        if self.tail == None:
13            self.tail = nd
14            nd.next = self.tail
15        else:
16            nd.next = self.tail.next
17            self.tail.next = nd
18        self.size += 1
19    def pushBack(self, data):
20        self.pushFront(data)
21        self.tail = self.tail.next
22    def popFront(self):
23        if self.size == 0:
24            return None
25        else:
26            nd = self.tail.next
27            self.size -= 1
28            if self.size == 0:
29                self.tail = None
30            else:
31                self.tail.next = nd.next
32        return nd.data
33    def printList(self):
34        if self.size > 0:
35            ptr = self.tail.next
36            while True:
37                print(ptr.data, end = " ")
38                if ptr == self.tail:
39                    break
40                ptr = ptr.next
41            print("")
42
43    def remove(self, data):
44        // 在此处补充你的代码
45        t = int(input())
```

```

46 for i in range(t):
47     lst = list(map(int, input().split()))
48     lkList = LinkList()
49     for x in lst:
50         lkList.pushBack(x)
51     lst = list(map(int, input().split()))
52     for a in lst:
53         result = lkList.remove(a)
54         if result == True:
55             lkList.printList()
56         elif result == False:
57             print("NOT FOUND")
58         else:
59             print("EMPTY")
60     print("-----")

```

输入

第一行为整数t，表示有t组数据。

每组数据2行

第一行是若干个整数，构成了一张链表

第二行是若干整数，是要从链表中删除的数。

输出

对每组数据第二行中的每个整数x:

1. 如果链表已经为空，则输出 "EMPTY"
2. 如果x在链表中，则将其删除，并且输出删除后的链表。如果删除后链表为空，则没输出。
如果有重复元素，则删前面的。
- 3) 如果链表不为空且x不在链表中，则输出"NOT FOUND"

样例输入

```

1 | 2
2 | 1 2 3
3 | 3 2 2 9 5 1 1 4
4 | 1
5 | 9 88 1 23

```

样例输出

```

1 1 2
2 1
3 NOT FOUND
4 NOT FOUND
5 NOT FOUND
6 EMPTY
7 EMPTY
8 -----
9 NOT FOUND
10 NOT FOUND
11 EMPTY
12 -----

```

来源

郭炜

程序填空题，需要掌握“补充代码”题型，例如写出某个函数的实现代码，如 def remove(self,data):

```

1 class Node:
2     def __init__(self, data, next=None):
3         self.data, self.next = data, next
4
5
6 class LinkList: # 循环链表
7     def __init__(self):
8         self.tail = None
9         self.size = 0
10
11     def isEmpty(self):
12         return self.size == 0
13
14     def pushFront(self, data):
15         nd = Node(data)
16         if self.tail == None:
17             self.tail = nd
18             nd.next = self.tail
19         else:
20             nd.next = self.tail.next
21             self.tail.next = nd
22             self.size += 1
23
24     def pushBack(self, data):
25         self.pushFront(data)
26         self.tail = self.tail.next
27
28     def popFront(self):
29         if self.size == 0:
30             return None
31         else:
32             nd = self.tail.next
33             self.size -= 1

```

```

34         if self.size == 0:
35             self.tail = None
36         else:
37             self.tail.next = nd.next
38     return nd.data
39
40     def printList(self):
41         if self.size > 0:
42             ptr = self.tail.next
43             while True:
44                 print(ptr.data, end=" ")
45                 if ptr == self.tail:
46                     break
47                 ptr = ptr.next
48             print("")
49
50     def remove(self, data): # 填空: 实现函数
51         if self.size == 0:
52             return None
53         else:
54             ptr = self.tail
55             while ptr.next.data != data:
56                 ptr = ptr.next
57             if ptr == self.tail:
58                 return False
59             self.size -= 1
60             if ptr.next == self.tail:
61                 self.tail = ptr
62             ptr.next = ptr.next.next
63             return True
64
65
66 t = int(input())
67 for i in range(t):
68     lst = list(map(int, input().split()))
69     lkList = LinkList()
70     for x in lst:
71         lkList.pushBack(x)
72     lst = list(map(int, input().split()))
73     for a in lst:
74         result = lkList.remove(a)
75         if result == True:
76             lkList.printList()
77         elif result == False:
78             print("NOT FOUND")
79         else:
80             print("EMPTY")
81     print("-----")
82
83 """
84 样例输入
85 2
86 1 2 3
87 3 2 2 9 5 1 1 4
88 1

```

```

89 9 88 1 23
90
91 样例输出
92 1 2
93 1
94 NOT FOUND
95 NOT FOUND
96 NOT FOUND
97 EMPTY
98 EMPTY
99 -----
100 NOT FOUND
101 NOT FOUND
102 EMPTY
103 -----
104 " "

```

示例4.插入链表元素

<http://dsbpython.openjudge.cn/2024allhw/004/>

很遗憾，一意孤行的Y君没有理会你告诉他的饮食计划并很快吃完了他的粮食储备。但好在他捡到了一张校园卡，凭这个他可以偷偷混入领取物资的队伍。为了不被志愿者察觉自己是只猫，他想要插到队伍的最中央。（插入后若有偶数个元素则选取靠后的位置）

于是他又找到了你，希望你能帮他修改志愿者写好的代码，在发放顺序的中间加上他的学号6。你虽然不理解志愿者为什么要用链表来写这份代码，但为了不被发现只得在此基础上进行修改：

```

1  class Node:
2      def __init__(self, data, next=None):
3          self.data, self.next = data, next
4
5  class LinkList:
6      def __init__(self):
7          self.head = None
8
9      def initList(self, data):
10         self.head = Node(data[0])
11         p = self.head
12         for i in data[1:]:
13             node = Node(i)
14             p.next = node
15             p = p.next
16
17         def insertCat(self):
18             // 在此处补充你的代码
19             #####
20         def printLk(self):
21             p = self.head
22             while p:
23                 print(p.data, end=" ")
24                 p = p.next
25             print()

```

```

26
27 lst = list(map(int,input().split()))
28 lkList = LinkList()
29 lkList.initList(lst)
30 lkList.insertCat()
31 lkList.printLk()

```

输入

一行，若干个整数，组成一个链表。

输出

一行，在链表中间位置插入数字6后得到的新链表

样例输入

```

1 ### 样例输入1
2 8 1 0 9 7 5
3 ### 样例输入2
4 1 2 3

```

样例输出

```

1 ### 样例输出1
2 8 1 0 6 9 7 5
3 ### 样例输出2
4 1 2 6 3

```

来源

Lou Yuke

程序填空题，需要掌握“补充代码”题型，例如写出某个函数的实现代码，如 `def insertCat(self):`

```

1 class Node:
2     def __init__(self, data, next=None):
3         self.data, self.next = data, next
4
5 class LinkList:
6     def __init__(self):
7         self.head = None
8
9     def initList(self, data):
10        self.head = Node(data[0])
11        p = self.head
12        for i in data[1:]:
13            node = Node(i)
14            p.next = node
15            p = p.next
16
17    def insertCat(self):
18        # 计算链表的长度

```

```

19     length = 0
20     p = self.head
21     while p:
22         length += 1
23         p = p.next
24
25     # 找到插入位置
26     position = length // 2 if length % 2 == 0 else (length // 2) + 1
27     p = self.head
28     for _ in range(position - 1):
29         p = p.next
30
31     # 在插入位置处插入数字6
32     node = Node(6)
33     node.next = p.next
34     p.next = node
35
36     def printLk(self):
37         p = self.head
38         while p:
39             print(p.data, end=" ")
40             p = p.next
41         print()
42
43     lst = list(map(int, input().split()))
44     lkList = LinkList()
45     lkList.initList(lst)
46     lkList.insertCat()
47     lkList.printLk()
48
49     """
50     ### 样例输入1
51     8 1 0 9 7 5
52     ### 样例输入2
53     1 2 3
54
55     ### 样例输出1
56     8 1 0 6 9 7 5
57     ### 样例输出2
58     1 2 6 3
59     """

```

4 栈 (Stack) - LIFO (后进先出)

栈是常用的一种线性数据结构。栈的修改与访问是按照后进先出的原则进行的，因此栈通常被称为是后进先出（last in first out）表，简称 LIFO 表。

4.1 实现栈

1 使用数组来模拟栈

```

1  st = [0] * N
2  # 这里使用 st[0] 代表栈中元素数量，同时也是栈顶下标
3
4  # 压栈：
5  st[st[0] + 1] = var1
6  st[0] = st[0] + 1
7  # 取栈顶：
8  u = st[st[0]]
9  # 弹栈：注意越界问题，*st == 0 时不能继续弹出
10 if st[0]:
11     st[0] = st[0] - 1
12 # 清空栈
13 st[0] = 0

```

2 使用 Python 中的 list 模拟栈

```

1  st = [5, 1, 4]
2
3  # 使用 append() 向栈顶添加元素
4  st.append(2)
5  st.append(3)
6  # >>> st
7  # [5, 1, 4, 2, 3]
8
9  # 使用 pop 取出栈顶元素
10 st.pop()
11 # >>> st
12 # [5, 1, 4, 2]
13
14 # 使用 clear 清空栈
15 st.clear()

```

3 用类实现栈

栈抽象数据类型通过以下结构和操作来定义。下面给出了栈的操作。

- `push(item)`: 入栈
- `pop()`: 出栈并返回（修改栈）
- `peek()`: 返回栈顶元素（不修改栈）
- `isEmpty()`: 判断是否为空
- `size()`: 返回元素个数

For example, if `s` is a stack that has been created and starts out empty, then Table 1 shows the results of a sequence of stack operations. Under stack contents, the top item is listed at the far right.

Table 1: Sample Stack Operations

Stack Operation	Stack Contents	Return Value
s.isEmpty()	[]	True
s.push(4)	[4]	
s.push('dog')	[4, 'dog']	
s.peek()	[4, 'dog']	'dog'
s.push(True)	[4, 'dog', True]	
s.size()	[4, 'dog', True]	3
s.isEmpty()	[4, 'dog', True]	False
s.push(8.4)	[4, 'dog', True, 8.4]	
s.pop()	[4, 'dog', True]	8.4
s.pop()	[4, 'dog']	True
s.size()	[4, 'dog']	2

现在已经明确定义了栈作为一种抽象数据类型，接下来我们将注意力转向使用Python来实现栈。回想一下，当我们为抽象数据类型提供物理实现时，称这种实现为数据结构。

在Python中，就像在任何面向对象编程语言中一样，实现诸如栈这样的抽象数据类型的首选方法是创建一个新类。栈操作被实现为方法。此外，为了实现栈（它是一个元素的集合），利用Python提供的简单而强大的基本集合是很合理的。我们将使用列表来实现。

Stack
- items: List
+ isEmpty() : bool + push(item: T) : None + pop() : T + peek() : T + size() : int

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4
```

```

5     def is_empty(self):
6         return self.items == []
7
8     def push(self, item):
9         self.items.append(item)
10
11    def pop(self):
12        return self.items.pop()
13
14    def peek(self):
15        return self.items[len(self.items)-1]
16
17    def size(self):
18        return len(self.items)
19
20    s = Stack()
21
22    print(s.is_empty())
23    s.push(4)
24    s.push('dog')
25
26    print(s.peek())
27    s.push(True)
28    print(s.size())
29    print(s.is_empty())
30    s.push(8.4)
31    print(s.pop())
32    print(s.pop())
33    print(s.size())
34
35    """
36    True
37    dog
38    3
39    False
40    8.4
41    True
42    2
43    """

```

要求自己会用类实现Stack，但是实际编程时候，直接使用list模拟栈更好。

4.2 应用一：括号匹配 (Parentheses Matching)

我们现在将注意力转向使用栈来解决真正的计算机科学问题。毫无疑问，你已经写过诸如

`(5+6)*(7+8)/(4+3)` 这样的算术表达式，其中使用了括号来安排操作的执行顺序。

括号必须以平衡的方式出现。**平衡的括号**意味着每个开符号都有一个对应的闭符号，并且括号对是正确嵌套的。考虑以下正确平衡的括号字符串：

```

1 | ((())())
2 |
3 | (((())))
4 |
5 | ((((())))())

```

Compare those with the following, which are not balanced:

```

1 | ((((((()))
2 |
3 | ()))
4 |
5 | ((())())

```

区分括号是否正确平衡是识别许多编程语言结构的重要部分。

接下来的挑战是编写一个算法，该算法能够从左到右读取一串括号，并判断这些符号是否平衡。为了解决这个问题，我们需要做一个重要的观察。当你从左到右处理符号时，最近的开括号必须与下一个闭括号匹配（见图4）。同时，第一个被处理的开括号可能需要等到最后一个符号才能找到它的匹配项。闭括号与开括号的匹配顺序与其出现顺序相反，它们从内到外进行匹配。这一点提示我们可以使用栈来解决这个问题。

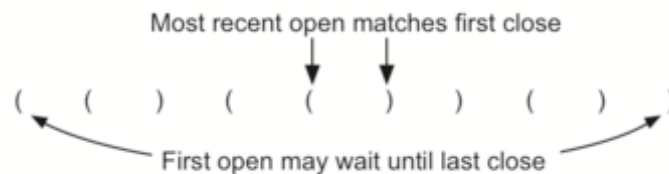


Figure 4: Matching Parentheses

```

1 | #returns a boolean result as to whether the string of parentheses is
   | balanced
2 | def par_checker(symbol_string):
3 |     s = [] # Stack()
4 |     balanced = True
5 |     index = 0
6 |     while index < len(symbol_string) and balanced:
7 |         symbol = symbol_string[index]
8 |         if symbol == "(":
9 |             s.append(symbol) # push(symbol)
10 |        else:
11 |            #if s.is_empty():
12 |                if not s:
13 |                    balanced = False
14 |            else:
15 |                s.pop()
16 |            index = index + 1
17 |
18 |        #if balanced and s.is_empty():
19 |            if balanced and not s:
20 |                return True

```

```

21         else:
22             return False
23
24     print(par_checker('((()))'))
25     print(par_checker('(()))'))
26
27     # True
28     # False

```

示例E20.有效的括号

stack, <https://leetcode.cn/problems/valid-parentheses/>

给定一个只包括 '(' , ')' , '{' , '}' , '[' , ']' 的字符串 `s` , 判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

示例 1:

输入: `s = "()"`

输出: `true`

示例 2:

输入: `s = "()[]{}"`

输出: `true`

示例 3:

输入: `s = "()["`

输出: `false`

示例 4:

输入: `s = "([])"`

输出: `true`

提示:

- `1 <= s.length <= 10^4`
- `s` 仅由括号 `'()[]{}'` 组成

```

1 from typing import List

```

```

2 class Solution:
3     def isValid(self, s: str) -> bool:
4         stack = []
5         for c in s:
6             if c == '(' or c == '[' or c == '{':
7                 stack.append(c)
8             else:
9                 if not stack:
10                    return False
11                 if c == ')' and stack[-1] != '(':
12                    return False
13                 if c == ']' and stack[-1] != '[':
14                    return False
15                 if c == '}' and stack[-1] != '{':
16                    return False
17                 stack.pop()
18         return not stack

```

1 Balanced Symbols (A General Case)

上述的平衡括号问题是出现在许多编程语言中的一种更普遍情况的具体案例。平衡和嵌套不同类型的开符号和闭符号的一般问题频繁出现。例如，在Python中，方括号 `[` 和 `]` 用于列表；花括号 `{` 和 `}` 用于字典；圆括号 `(` 和 `)` 用于元组和算术表达式。只要每种符号都保持自身的开和关关系，就可以混合使用这些符号。例如，如下所示的符号字符串：

看大括号 `{}` 内部是否有冒号 `:`。有冒号的是字典，没有冒号、只有逗号分隔的元素的是集合。而创建空集合时，必须使用 `set()` 函数。

```

1 { { ( [ ] [ ] ) } ( ) }
2
3 [ [ { { ( ( ) ) } } ] ]
4
5 [ ] [ ] [ ] ( ) { }

```

are properly balanced in that not only does each opening symbol have a corresponding closing symbol, but the types of symbols match as well.

Compare those with the following strings that are not balanced:

```

1 ( [ ] )
2
3 ( ( ( ) ] ) )
4
5 [ { ( ) ]

```

从前一节的简单括号检查器可以很容易地扩展来处理这些新的符号类型。回想一下，每个开符号只是简单地压入栈中，等待匹配的闭符号稍后在序列中出现。当一个闭符号确实出现时，唯一的区别是我们必须检查它是否正确匹配栈顶的开符号类型。如果这两个符号不匹配，那么字符串就不平衡。再次强调，如果整个字符串都被处理且栈中没有剩下任何未匹配的符号，那么该字符串就是正确平衡的。

```

1  def par_checker(symbol_string):
2      s = [] # Stack()
3      balanced = True
4      index = 0
5      while index < len(symbol_string) and balanced:
6          symbol = symbol_string[index]
7          if symbol in "([{":
8              s.append(symbol) # push(symbol)
9          else:
10             top = s.pop()
11             if not matches(top, symbol):
12                 balanced = False
13             index += 1
14             #if balanced and s.is_empty():
15             if balanced and not s:
16                 return True
17             else:
18                 return False
19
20  def matches(open, close):
21      opens = "([{"
22      closes = ")]}"
23      return opens.index(open) == closes.index(close)
24
25  print(par_checker('{{}}{{}}'))
26
27  # output: False

```

示例OJ03704: 括号匹配问题

stack, <http://cs101.openjudge.cn/practice/03704>

在某个字符串（长度不超过100）中有左括号、右括号和大小写字母；规定（与常见的算数式子一样）任何一个左括号都从内到外与在它右边且距离最近的右括号匹配。写一个程序，找到无法匹配的左括号和右括号，输出原来字符串，并在下一行标出不能匹配的括号。不能匹配的左括号用"\$"标注，不能匹配的右括号用"?"标注。

输入

输入包括多组数据，每组数据一行，包含一个字符串，只包含左右括号和大小写字母，字符串长度不超过100

注意：cin.getline(str,100)最多只能输入99个字符！

输出

对每组输出数据，输出两行，第一行包含原始输入字符，第二行由"\$","?"和空格组成，"\$"和"?"表示与之对应的左括号和右括号不能匹配。

样例输入

```

1  ((ABCD(x)
2  )(rttyy())sss(

```

样例输出

```
1  ((ABCD(x)
2  $$
3  )(rttyy())sss)(
4  ?           ?$
```

```
1  # https://www.cnblogs.com/huashanqingzhu/p/6546598.html
2
3  lines = []
4  while True:
5      try:
6          lines.append(input())
7      except EOFError:
8          break
9
10 ans = []
11 for s in lines:
12     stack = []
13     Mark = []
14     for i in range(len(s)):
15         if s[i] == '(':
16             stack.append(i)
17             Mark += ' '
18         elif s[i] == ')':
19             if len(stack) == 0:
20                 Mark += '?'
21             else:
22                 Mark += ' '
23                 stack.pop()
24         else:
25             Mark += ' '
26
27     while(len(stack)):
28         Mark[stack[-1]] = '$'
29         stack.pop()
30
31     print(s)
32     print(''.join(map(str, Mark)))
```

练习20140:今日化学论文

<http://cs101.openjudge.cn/practice/20140/>

4.3 进制转换

1 将十进制数转换成二进制数

在你学习计算机科学的过程中，可能已经以这样或那样的方式接触到二进制数的概念。二进制表示在计算机科学中非常重要，因为计算机中存储的所有值都以一串二进制数字的形式存在，即由0和1组成的字符串。如果没有能力在常见表示法和二进制数之间来回转换，我们将需要以非常笨拙的方式与计算机进行交互。

整数值是常见的数据项，在计算机程序和计算中无时无刻不在使用。我们在数学课上学习它们，并且当然使用十进制数系统或基数为10的方式来表示它们。十进制数 233_{10} 及其对应的二进制等价形式 11101001_2 分别被解释为：

- 十进制数 233_{10} 意味着这是一个基于10的数值，计算方式为 $2 * 10^2 + 3 * 10^1 + 3 * 10^0$ 。
- 二进制数 11101001_2 则是一个基于2的数值，计算方式为 $1 * 2^7 + 1 * 2^6 + 1 * 2^5 + 0 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0$ 。

这种转换对于理解计算机如何处理和存储数值数据至关重要。

但是，我们如何轻松地将整数值转换为二进制数呢？答案是一种称为“除以2”的算法，它使用栈来跟踪二进制结果的数字。

“除以2”算法假设我们从一个大于0的整数开始。然后通过一个简单的迭代过程不断将十进制数除以2并记录余数。第一次除以2可以告诉我们该值是奇数还是偶数。偶数值的余数为0，意味着在个位上将是数字0。奇数值的余数为1，在个位上将是数字1。我们可以认为构建二进制数是一个数字序列的过程；我们计算的第一个余数实际上会是这个序列中的最后一个数字。如图5所示，我们再次看到了这种**反转特性**，这表明栈可能是解决问题的合适数据结构。

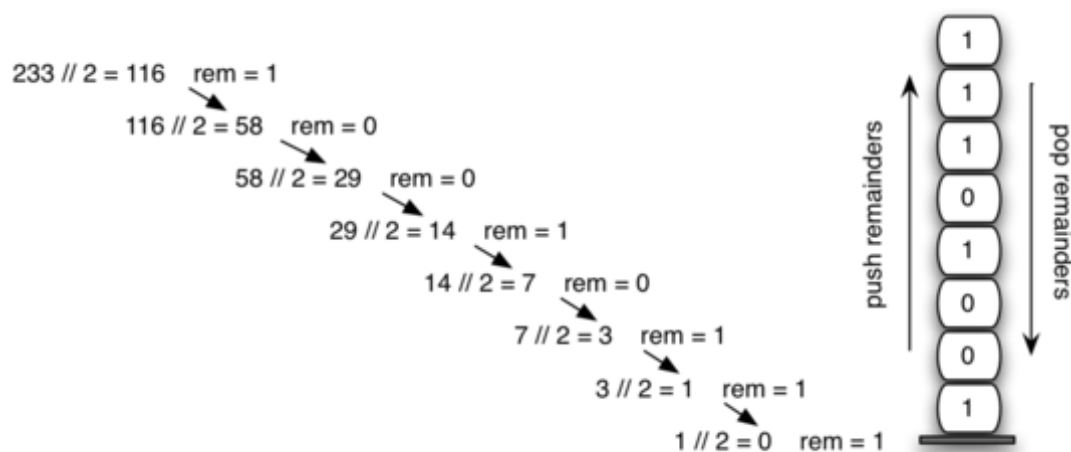


Figure 5: Decimal-to-Binary Conversion

```
1 def divide_by_2(dec_num):
2     rem_stack = [] # Stack()
3
4     while dec_num > 0:
5         rem = dec_num % 2
6         rem_stack.append(rem) # push(rem)
7         dec_num = dec_num // 2
8
9     bin_string = ""
10    #while not rem_stack.is_empty():
11    while rem_stack:
12        bin_string = bin_string + str(rem_stack.pop())
```

```

13
14     return bin_string
15
16 print(divide_by_2(233))
17
18 # output: 11101001

```

```

1 def base_converter(dec_num, base):
2     digits = "0123456789ABCDEF"
3
4     rem_stack = [] # Stack()
5
6     while dec_num > 0:
7         rem = dec_num % base
8         #rem_stack.push(rem)
9         rem_stack.append(rem)
10        dec_num = dec_num // base
11
12    new_string = ""
13    #while not rem_stack.is_empty():
14    while rem_stack:
15        new_string = new_string + digits[rem_stack.pop()]
16
17    return new_string
18
19 print(base_converter(25, 2))
20 print(base_converter(2555, 16))
21
22 # 11001
23 # 9FB

```

练习OJ02734: 十进制到八进制

<http://cs101.openjudge.cn/practice/02734/>

把一个十进制正整数转化成八进制。

输入

一行，仅含一个十进制表示的整数a($0 < a < 65536$)。

输出

一行，a的八进制表示。

样例输入

9

样例输出

11

使用栈来实现十进制到八进制的转换可以通过不断除以8并将余数压入栈中的方式来实现。然后，将栈中的元素依次出栈，构成八进制数的各个位。

```
1 decimal = int(input()) # 读取十进制数
2
3 # 创建一个空栈
4 stack = []
5
6 # 特殊情况：如果输入的数为0，直接输出0
7 if decimal == 0:
8     print(0)
9 else:
10    # 不断除以8，并将余数压入栈中
11    while decimal > 0:
12        remainder = decimal % 8
13        stack.append(remainder)
14        decimal = decimal // 8
15
16    # 依次出栈，构成八进制数的各个位
17    octal = ""
18    while stack:
19        octal += str(stack.pop())
20
21    print(octal)
```

4.4 中序、前序和后序表达式

当你写一个算术表达式，如 $B * C$ 时，表达式的形式为你提供了可以正确解释它的信息。在这种情况下，我们知道变量 B 正在乘以变量 C ，因为乘法操作符 $*$ 出现在它们之间的表达式中。这种类型的表示法被称为**中缀**表示法，因为操作符位于它所操作的两个操作数之间。

考虑另一个中缀的例子， $A + B * C$ 。操作符 $+$ 和 $*$ 仍然出现在操作数之间，但现在有一个问题：它们各自作用于哪些操作数？是 $+$ 作用于 A 和 B ，还是 $*$ 作用于 B 和 C ？这个表达式似乎有歧义。

实际上，你已经阅读和书写这类表达式很长时间了，并且它们并不会给你造成任何问题。原因是你了解关于操作符 $+$ 和 $*$ 的一些事情。每个操作符都有一个**优先级**级别。优先级较高的操作符先于优先级较低的操作符使用。唯一能改变该顺序的是括号的存在。对于算术操作符的优先级顺序将乘除放在加减之上。如果出现相同优先级的操作符，则按照从左到右的顺序或结合性来决定。

让我们使用操作符优先级来解释令人困惑的表达式 $A + B * C$ 。首先对 B 和 C 进行乘法运算，然后将 A 加到那个结果上。 $(A + B) * C$ 将强制先执行 A 和 B 的加法运算，然后再进行乘法运算。在表达式 $A + B + C$ 中，根据优先级（通过结合性），最左边的 $+$ 会首先被执行。

尽管这一切对你来说可能是显而易见的，请记住计算机需要确切知道要执行什么操作以及它们的顺序。一种确保不会因操作顺序引起混淆的方式是创建所谓的**完全括号化**表达式。这种类型的表达式为每个操作符使用一对括号。括号规定了操作的顺序；没有歧义。也不需要记忆任何优先级规则。

表达式 $A + B * C + D$ 可以重写为 $((A + (B * C)) + D)$ ，以显示首先进行乘法，随后是最左边的加法。 $A + B + C + D$ 可以写作 $((((A + B) + C) + D))$ ，因为加法操作从左向右结合。

还有两种其他非常重要的表达式格式，一开始可能并不明显。考虑中缀表达式 $A + B$ 。如果我们把操作符移到两个操作数之前会发生什么？生成的表达式将是 $+ A B$ 。同样，我们可以把操作符移到最后。我们得到 $A B +$ 。这些看起来有点奇怪。

操作符相对于操作数位置的这些变化创造了两种新的表达式格式，**前缀**和**后缀**。**前缀表达式要求所有操作符都在其工作的两个操作数之前**。而后缀则要求其操作符在其对应的操作数之后。更多的例子应该有助于更清晰地理解这一点。

$A + B * C$ 在前缀中会被写作 $+ A * B C$ 。乘法操作符直接出现在操作数 B 和 C 之前，表示 $*$ 的优先级高于 $+$ 。然后加法操作符出现在 A 和乘法的结果之前。

在后缀中，表达式会是 $A B C * +$ 。再次，操作顺序被保留，因为 $*$ 紧接在 B 和 C 之后出现，表明 $*$ 有更高的优先级，随后是 $+$ 。虽然操作符移动了，现在要么出现在各自操作数之前，要么出现在之后，但**操作数之间的相对顺序保持不变**。

When you write an arithmetic expression such as $B * C$, the form of the expression provides you with information so that you can interpret it correctly. In this case we know that the variable B is being multiplied by the variable C since the multiplication operator $*$ appears between them in the expression. This type of notation is referred to as **infix** since the operator is *in between* the two operands that it is working on.

Consider another infix example, $A + B * C$. The operators $+$ and $*$ still appear between the operands, but there is a problem. Which operands do they work on? Does the $+$ work on A and B or does the $*$ take B and C ? The expression seems ambiguous.

In fact, you have been reading and writing these types of expressions for a long time and they do not cause you any problem. The reason for this is that you know something about the operators $+$ and $*$. Each operator has a **precedence** level. Operators of higher precedence are used before operators of lower precedence. The only thing that can change that order is the presence of parentheses. The precedence order for arithmetic operators places multiplication and division above addition and subtraction. If two operators of equal precedence appear, then a left-to-right ordering or associativity is used.

Let's interpret the troublesome expression $A + B * C$ using operator precedence. B and C are multiplied first, and A is then added to that result. $(A + B) * C$ would force the addition of A and B to be done first before the multiplication. In expression $A + B + C$, by precedence (via associativity), the leftmost $+$ would be done first.

Although all this may be obvious to you, remember that computers need to know exactly what operators to perform and in what order. One way to write an expression that guarantees there will be no confusion with respect to the order of operations is to create what is called a **fully parenthesized** expression. This type of expression uses one pair of parentheses for each operator. The parentheses dictate the order of operations; there is no ambiguity. There is also no need to remember any precedence rules.

The expression $A + B * C + D$ can be rewritten as $((A + (B * C)) + D)$ to show that the multiplication happens first, followed by the leftmost addition. $A + B + C + D$ can be written as $((((A + B) + C) + D))$ since the addition operations associate from left to right.

There are two other very important expression formats that may not seem obvious to you at first. Consider the infix expression $A + B$. What would happen if we moved the operator before the two operands? The resulting expression would be $+ A B$. Likewise, we could move the operator to the end. We would get $A B +$. These look a bit strange.

These changes to the position of the operator with respect to the operands create two new expression formats, **prefix** and **postfix**. Prefix expression notation requires that all operators precede the two operands that they work on. Postfix, on the other hand, requires that its operators come after the corresponding operands. A few more examples should help to make this a bit clearer (see Table 2).

$A + B * C$ would be written as $+ A * B C$ in prefix. The multiplication operator comes immediately before the operands B and C , denoting that $*$ has precedence over $+$. The addition operator then appears before the A and the result of the multiplication.

In postfix, the expression would be $A B C * +$. Again, the order of operations is preserved since the $*$ appears immediately after the B and the C , denoting that $*$ has precedence, with $+$ coming after. Although the operators moved and now appear either before or after their respective operands, the order of the operands stayed exactly the same relative to one another.

Table 2: Exmples of Infix, Prefix, and Postfix

Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ A B$	$A B +$
$A + B * C$	$+ A * B C$	$A B C * +$

现在考虑中缀表达式 $(A + B) * C$ 。回想一下，在这种情况下，中缀要求使用括号以强制在乘法之前执行加法运算。然而，当我们将 $A + B$ 写成前缀时，只需将加法操作符移到操作数之前，即 $+ A B$ 。此操作的结果成为乘法的第一个操作数。然后将乘法操作符移到整个表达式的前面，得到 $* + A B C$ 。同样，在后缀表达式中， $A B +$ 强制先进行加法运算。然后将乘法应用于该结果和剩余的操作数 C 。因此，正确的后缀表达式是 $A B + C *$ 。

再次考虑这三个表达式（见表3）。这里发生了一件非常重要的事情。括号去哪了？为什么我们在前缀和后缀中不需要它们？答案是，操作符相对于它们所操作的操作数不再有歧义。只有中缀表示法需要额外的符号。前缀和后缀表达式中的操作顺序完全由操作符的位置决定，而不受其他因素影响。在很多方面，这使得中缀成为最不理想的表示法。

Now consider the infix expression $(A + B) * C$. Recall that in this case, infix requires the parentheses to force the performance of the addition before the multiplication. However, when $A + B$ was written in prefix, the addition operator was simply moved before the operands, $+ A B$. The result of this operation becomes the first operand for the multiplication. The multiplication operator is moved in front of the entire expression, giving us $* + A B C$. Likewise, in postfix $A B +$ forces the addition to happen first. The multiplication can be done to that result and the remaining operand C . The proper postfix expression is then $A B + C *$.

Consider these three expressions again (see Table 3). Something very important has happened. Where did the parentheses go? Why don't we need them in prefix and postfix? The answer is that the operators are no longer ambiguous with respect to the operands that they work on. Only infix notation requires the additional symbols. The order of operations within prefix and postfix expressions is completely determined by the position of the operator and nothing else. In many ways, this makes infix the least desirable notation to use.

Table 3: An Expression with Parentheses

Infix Expression	Prefix Expression	Postfix Expression
$(A + B) * C$	$* + A B C$	$A B + C *$

Table 4 shows some additional examples of infix expressions and the equivalent prefix and postfix expressions. Be sure that you understand how they are equivalent in terms of the order of the operations being performed.

Table 4: Additional Examples of Infix, Prefix and Postfix

Infix Expression	Prefix Expression	Postfix Expression
$A + B * C + D$	$+ + A * B C D$	$A B C * + D +$
$(A + B) * (C + D)$	$* + A B + C D$	$A B + C D + *$
$A * B + C * D$	$+ * A B * C D$	$A B * C D * +$
$A + B + C + D$	$+ + + A B C D$	$A B + C + D +$

1 Conversion of Infix Expressions to Prefix and Postfix

到目前为止，我们使用了临时的方法在中缀表达式和等价的前缀及后缀表达式表示法之间进行转换。正如你可能预期的那样，存在算法方法可以执行这种转换，使得任何复杂度的表达式都能被正确地变换。

我们将首先考虑的技术使用了之前讨论过的完全括号化表达式的概念。回想一下， $A + B * C$ 可以写成 $(A + (B * C))$ 来明确显示乘法优先于加法。然而，仔细观察后你可以看到，**每对括号也标明了一个操作数对的开始和结束**，其中间是相应的操作符。

看看上面子表达式 $(B * C)$ 中的右括号。**如果我们把乘法符号移到该位置并移除匹配的左括号**，得到 $B C *$ ，实际上我们就将子表达式转换为了后缀表示法。如果也将加法操作符移动到其对应的右括号位置，并移除匹配的左括号，就会得到完整的后缀表达式（见图6）。

通过这种方法，我们可以系统地将包含任意复杂度的中缀表达式转换为后缀形式，确保了转换过程的准确性和一致性，而无需依赖记忆操作符优先级规则。同样的原则也可应用于创建前缀表达式，只是操作符的位置相对于操作数有所不同。

So far, we have used ad hoc methods to convert between infix expressions and the equivalent prefix and postfix expression notations. As you might expect, there are algorithmic ways to perform the conversion that allow any expression of any complexity to be correctly transformed.

The first technique that we will consider uses the notion of a fully parenthesized expression that was discussed earlier. Recall that $A + B * C$ can be written as $(A + (B * C))$ to show explicitly that the multiplication has precedence over the addition. On closer observation, however, you can see that each parenthesis pair also denotes the beginning and the end of an operand pair with the corresponding operator in the middle.

Look at the right parenthesis in the subexpression $(B * C)$ above. If we were to move the multiplication symbol to that position and remove the matching left parenthesis, giving us $B C *$, we would in effect have converted the subexpression to postfix notation. If the addition operator were also moved to its corresponding right parenthesis position and the matching left parenthesis were removed, the complete postfix expression would result (see Figure 6).



Figure 6: Moving Operators to the Right for Postfix Notation

If we do the same thing but instead of moving the symbol to the position of the right parenthesis, we move it to the left, we get prefix notation (see Figure 7). The position of the parenthesis pair is actually a clue to the final position of the enclosed operator.

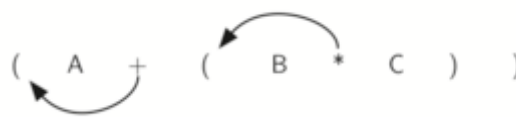


Figure 7: Moving Operators to the Left for Prefix Notation

因此，为了将一个表达式（无论多么复杂）转换为前缀或后缀表示法，首先使用运算顺序完全加上括号。然后，根据你想要得到前缀还是后缀表示法，将括号内的操作符移动到左括号或右括号的位置。

So in order to convert an expression, no matter how complex, to either prefix or postfix notation, fully parenthesize the expression using the order of operations. Then move the enclosed operator to the position of either the left or the right parenthesis depending on whether you want prefix or postfix notation.

Here is a more complex expression: $(A + B) * C - (D - E) * (F + G)$. Figure 8 shows the conversion to postfix and prefix notations.

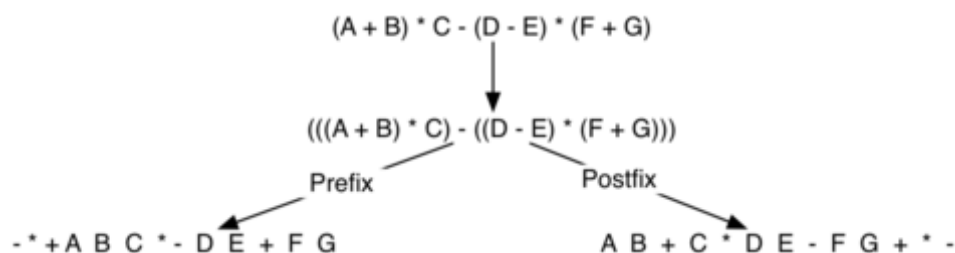


Figure 8: Converting a Complex Expression to Prefix and Postfix Notations

2 应用二：表达式转换 (调度场算法 Shunting Yard)

我们需要开发一种算法，将中缀表达式 (Infix) 转换为后缀表达式 (Postfix/逆波兰式)。为此，我们将更仔细地观察转换过程。

再次考虑表达式 $A + B * C$ 。如上所示，其对应的后缀表达式是 $A B C * +$ 。我们已经注意到操作数 A 、 B 和 C 保持它们的相对位置不变。只有操作符改变了位置。让我们再次查看中缀表达式中的操作符。从左到右首先出现的操作符是 $+$ 。然而，在后缀表达式中，由于下一个操作符 $*$ 的优先级高于加法， $+$ 被放在了最后。原始表达式中的操作符顺序在得到的后缀表达式中被反转了。

当我们处理表达式时，由于相应的右操作数还未出现，操作符需要暂时存储在某处。此外，这些已保存的操作符的顺序可能需要根据它们的优先级进行反转。在这个例子中的加法和乘法就是这种情况。由于加法操作符出现在乘法操作符之前且优先级较低，它需要在乘法操作符之后出现。由于这种顺序的反转，使用栈来保存操作符直到需要它们为止是有意义的。

那么对于 $(A + B) * C$ 怎么办呢？回想一下，其对应的后缀表达式是 $AB + C *$ 。同样，从左到右处理这个中缀表达式，我们首先看到的是 $+$ 。在这种情况下，当我们看到 $*$ 时，由于括号的作用， $+$ 已经被放置在结果表达式中，因为它对 $*$ 具有优先权。现在我们可以开始了解转换算法的工作原理了。当我们看到一个左括号时，我们会将其保存以指示即将出现一个高优先级的操作符。那个操作符需要等待直到出现相应的右括号来标明它的位置（回忆完全括号化的方法）。当右括号出现时，操作符可以从栈中弹出。

当我们从左到右扫描中缀表达式时，我们将使用一个栈来保存操作符。这提供了我们在第一个例子中提到的反转。栈顶总是最近保存的操作符。每当我们读取一个新的操作符时，我们需要考虑该操作符与已经在栈上的操作符（如果有的话）相比，其优先级如何。

假设中缀表达式是由空格分隔的标记字符串。操作符标记包括 $*$, $/$, $+$ 和 $-$ ，以及左右括号 $($ 和 $)$ 。操作数标记是单字符标识符 A , B , C 等等。遵循以下步骤可以产生按后缀顺序排列的标记字符串：

1. 创建一个名为 `opstack` 的空栈用于存放操作符，并创建一个空列表用于输出。
2. 使用字符串方法 `split` 将输入的中缀字符串转换成列表。
3. 从左到右扫描标记列表：
 - 如果标记是操作数，则将其附加到输出列表的末尾。
 - 如果标记是左括号，则将其压入 `opstack` 栈中。
 - 如果标记是右括号，则从 `opstack` 中弹出元素直到移除相应的左括号为止，并将每个操作符附加到输出列表的末尾。
 - 如果标记是操作符 $*$, $/$, $+$ 或 $-$ ，则将其压入 `opstack` 栈中。但是，首先移除已在 `opstack` 上且具有更高或相同优先级的所有操作符，并将它们附加到输出列表的末尾。
4. 当输入表达式被完全处理后，检查 `opstack`。栈上任何剩余的操作符都可以被移除并附加到输出列表的末尾。

将中缀表达式 (Infix) 转换为后缀表达式 (Postfix/逆波兰式)。

- 优先级管理： $*$, $/$ $>$ $+$, $-$ $>$ $($ 。
- 处理规则：
 1. 操作数直接输出。
 2. 遇到 $($ 入栈。
 3. 遇到 $)$ 弹出栈顶直至遇到 $($ 。
 4. 遇到运算符，弹出栈中优先级更高或相等的运算符，再将当前操作符入栈。

```
1 def infixToPostfix(infixexpr):
2     prec = {}
3     prec["*"] = 3
4     prec["/"] = 3
5     prec["+"] = 2
6     prec["-"] = 2
```

```

7     prec["("] = 1
8     opStack = [] # Stack()
9     postfixList = []
10    tokenList = infixexpr.split()
11
12    for token in tokenList:
13        if token in "ABCDEFGHIJKLMNOPQRSTUVWXYZ" or token in "0123456789":
14            postfixList.append(token)
15        elif token == '(':
16            #opStack.push(token)
17            opStack.append(token)
18        elif token == ')':
19            topToken = opStack.pop()
20            while topToken != '(':
21                postfixList.append(topToken)
22                topToken = opStack.pop()
23        else:
24            #while (not opStack.is_empty()) and (prec[opStack.peek()] >=
prec[token]):
25                while opStack and (prec[opStack[-1]] >= prec[token]):
26                    postfixList.append(opStack.pop())
27                #opStack.push(token)
28                opStack.append(token)
29
30    #while not opStack.is_empty():
31    while opStack:
32        postfixList.append(opStack.pop())
33    return " ".join(postfixList)
34
35    print(infixToPostfix("A * B + C * D"))
36    print(infixToPostfix("( A + B ) * C - ( D - E ) * ( F + G )"))
37
38    print(infixToPostfix("( A + B ) * ( C + D )"))
39    print(infixToPostfix("( A + B ) * C"))
40    print(infixToPostfix("A + B * C"))
41
42    """
43    A B * C D * +
44    A B + C * D E - F G + * -
45    A B + C D + *
46    A B + C *
47    A B C * +
48    """

```

关于栈使用的重要原则

【李冠黎 24工学院】

1. 栈的基本用途是用于“等待”。具体来说，当遇到一条指令，而这条指令的操作依赖于后续的指令时，可以选择将这条指令先入栈，“稍作等待”，直到遍历到后面部分了解到具体如何操作后再利用 `pop` 方法回过头来处理该指令。这是栈使用的总体原则。
2. 栈与括号匹配的关系密切。很多涉及栈的问题都涉及到括号匹配问题，其核心在于识别一对括号

内所包含的内容。这时同样可以应用第一条原则。当我们遇到左括号时，由于不清楚它对应的右括号在哪里，所以先将其入栈。等到遇到右括号时，再回头找到匹配的左括号，并进行相应的处理。

3. 理解栈相关问题时，研究给定示例非常重要。在解决栈相关的问题之前，先大致研究一下给出的示例，分析答案和输入数据（或称“密文”）之间的关系非常有帮助。例如，在中序表达式转后序表达式的题目中，通过观察转换后的结果可以发现数字的顺序保持不变，而对于每一个运算单元，符号总是位于最后。这表明：数字不需要入栈，因为它们的操作不依赖于后续的指令，可以直接添加到结果中。相反，运算符需要入栈，因为我们尚未确定该运算符所属的运算单元将在何处结束。一旦根据某些条件判断出运算单元结束，就可以有序地将运算符从栈中弹出，并加入到结果中。

Shunting yard alorighitm

Shunting yard alorighitm（调度场算法）是一种用于将中缀表达式转换为后缀表达式的算法。它由荷兰计算机科学家 Edsger Dijkstra 在1960年代提出，用于解析和计算数学表达式。

Edsger W. Dijkstra

ArticleTalk

ReadEditView historyTools

From Wikipedia, the free encyclopedia

Edsger Wybe Dijkstra (/ˈdaɪkstrə/ *DYKE-strə*; Dutch: [ˈɛtsxər ˈvɪbə ˈdeikstra] ⓘ; 11 May 1930 – 6 August 2002) was a Dutch computer scientist, programmer, software engineer, and science essayist.^{[1][2]}

Born in Rotterdam, the Netherlands, Dijkstra studied mathematics and physics and then theoretical physics at the University of Leiden. Adriaan van Wijngaarden offered him a job as the first computer programmer in the Netherlands at the Mathematical Center in Amsterdam, where he worked from 1952 until 1962. He formulated and solved the shortest path problem in 1956, and in 1960 developed the first compiler for the programming language ALGOL 60 in conjunction with colleague Jaap A. Zonneveld. In 1962 he moved to Eindhoven, and later to Nuenen, where he became a professor in the Mathematics Department at the Technische Hogeschool Eindhoven. In the late 1960s he built the THE multiprogramming system, which influenced the designs of subsequent systems through its use of software-based paged virtual memory. Dijkstra joined Burroughs Corporation as its sole research fellow in August 1973. The Burroughs years saw him at his most prolific in output of research articles. He wrote nearly 500 documents in the "EWD" series, most of them technical reports, for private circulation within a select group.

Dijkstra accepted the Schlumberger Centennial Chair in the Computer Science Department at the University of Texas at Austin in 1984, working in Austin, Texas until his retirement in November 1999. He and his wife returned from Austin to his original house in Nuenen, where he died on 6 August 2002 after a long struggle with cancer.^[3]

He received the 1972 Turing Award for fundamental contributions to developing structured programming languages. Shortly before his death, he received the ACM PODC Influential Paper Award in distributed computing for his work on self-stabilization of program computation. This annual award was renamed the Dijkstra Prize the following year, in his honor.

Edsger W. Dijkstra

Dijkstra in 2002

Born	11 May 1930 Rotterdam, Netherlands
Died	6 August 2002 (aged 72) Nuenen, Netherlands
Citizenship	Netherlands
Education	Leiden University (B.S., M.S.) University of Amsterdam (Ph.D.)
Spouse	Maria (Ria) C. Debets
Awards	Turing Award (1972)

Shunting Yard 算法的主要思想是使用两个栈（运算符栈和输出栈）来处理表达式的符号。算法按照运算符的优先级和结合性，将符号逐个处理并放置到正确的位置。最终，输出栈中的元素就是转换后的后缀表达式。

以下是 Shunting Yard 算法的基本步骤：

- 1. 初始化运算符栈和输出栈为空。
- 2. 从左到右遍历中缀表达式的每个符号。
 - 如果是操作数（数字），则将其添加到输出栈。
 - 如果是左括号，则将其推入运算符栈。

- 如果是运算符：
 - 如果运算符的优先级大于运算符栈顶的运算符，或者运算符栈顶是左括号，则将当前运算符推入运算符栈。
 - 否则，将运算符栈顶的运算符弹出并添加到输出栈中，直到满足上述条件（或者运算符栈为空）。
 - 将当前运算符推入运算符栈。
 - 如果是右括号，则将运算符栈顶的运算符弹出并添加到输出栈中，直到遇到左括号。将左括号弹出但不添加到输出栈中。
3. 如果还有剩余的运算符在运算符栈中，将它们依次弹出并添加到输出栈中。
 4. 输出栈中的元素就是转换后的后缀表达式。

练习OJ24591:中序表达式转后序表达式

<http://cs101.openjudge.cn/practice/24591/>

中序表达式是运算符放在两个数中间的表达式。乘、除运算优先级高于加减。可以用"()"来提升优先级 --- 就是小学生写的四则算术运算表达式。中序表达式可用如下方式递归定义：

- 1) 一个数是一个中序表达式。该表达式的值就是数的值。
2. 若a是中序表达式，则"(a)"也是中序表达式(引号不算)，值为a的值。
3. 若a,b是中序表达式，c是运算符，则"acb"是中序表达式。"acb"的值是对a和b做c运算的结果，且a是左操作数，b是右操作数。

输入一个中序表达式，要求转换成一个后序表达式输出。

输入

第一行是整数n($n < 100$)。接下来n行，每行一个中序表达式，数和运算符之间没有空格，长度不超过700。

输出

对每个中序表达式，输出转成后序表达式后的结果。后序表达式的数之间、数和运算符之间用一个空格分开。

样例输入

```
1 | 3
2 | 7+8.3
3 | 3+4.5*(7+2)
4 | (3)*((3+4)*(2+3.5)/(4+5))
```

样例输出

```
1 | 7 8.3 +
2 | 3 4.5 7 2 + * +
3 | 3 3 4 + 2 3.5 + * 4 5 + / *
```

来源: Guo wei

接收浮点数，是number buffer技巧。

```
1 def infix_to_postfix(expression):
2     precedence = {'+':1, '-':1, '*':2, '/':2}
3     stack = []
4     postfix = []
5     number = ''
6
7     for char in expression:
8         if char.isnumeric() or char == '.':
9             number += char
10        else:
11            if number:
12                num = float(number)
13                postfix.append(int(num) if num.is_integer() else num)
14                number = ''
15            if char in '+-*/':
16                while stack and stack[-1] in '+-*/' and
precedence[char] <= precedence[stack[-1]]:
17                    postfix.append(stack.pop())
18                    stack.append(char)
19            elif char == '(':
20                stack.append(char)
21            elif char == ')':
22                while stack and stack[-1] != '(':
23                    postfix.append(stack.pop())
24                stack.pop()
25
26        if number:
27            num = float(number)
28            postfix.append(int(num) if num.is_integer() else num)
29
30        while stack:
31            postfix.append(stack.pop())
32
33        return ' '.join(str(x) for x in postfix)
34
35 n = int(input())
36 for _ in range(n):
37     expression = input()
38     print(infix_to_postfix(expression))
```

接收数据，还可以用re处理。

```

1  # 24591:中序表达式转后序表达式
2  # http://cs101.openjudge.cn/practice/24591/
3
4  def inp(s):
5      #s=input().strip()
6      import re
7      s=re.split(r'([\(\)\+\-\*\\/])',s)
8      s=[item for item in s if item.strip()]
9      return s
10
11 exp = "(3)*((3+4)*(2+3.5)/(4+5)) "
12 print(inp(exp))

```

3 应用三：后缀表达式求值 (Postfix Evaluation)

作为栈的最后一个示例，我们将考虑已经处于后缀表示法中的表达式的求值。在这种情况下，栈再次成为首选的数据结构。然而，当你扫描后缀表达式时，需要等待的是操作数，而不是像在上述转换算法中那样等待操作符。另一种思考解决方案的方式是，每当在输入中看到一个操作符时，将会使用最近的两个操作数进行求值。

这意味着，在读取后缀表达式的过程中，每当你遇到一个操作数，就将其压入栈中。一旦遇到操作符，就从栈中弹出适当数量的操作数（对于二元操作符来说是两个），执行相应的计算，并将结果压回到栈中。这个过程会一直持续到表达式的末尾，最终栈顶元素就是整个表达式的计算结果。这种方法简洁而有效地解决了后缀表达式的求值问题，因为它充分利用了栈的后进先出特性来管理操作数和操作符的顺序。

核心逻辑：遇到操作数入栈；遇到运算符，弹出两个数进行运算，结果重新入栈。最终栈底即为结果。

```

1  def postfixEval(postfixExpr):
2      operandStack = []
3      tokenList = postfixExpr.split()
4
5      for token in tokenList:
6          if token in "0123456789":
7              operandStack.append(int(token))
8          else:
9              operand2 = operandStack.pop()
10             operand1 = operandStack.pop()
11             result = doMath(token,operand1,operand2)
12             operandStack.append(result)
13     return operandStack.pop()
14
15 def doMath(op, op1, op2):
16     if op == "*":
17         return op1 * op2
18     elif op == "/":
19         return op1 / op2
20     elif op == "+":
21         return op1 + op2

```

```

22         else:
23             return op1 - op2
24
25 print(postfixEval('7 8 + 3 2 + /'))
26
27 # output: 3.0

```

示例OJ24588: 后序表达式求值

<http://cs101.openjudge.cn/practice/24588/>

后序表达式由操作数和运算符构成。操作数是整数或小数，运算符有 $+$ $-$ $*$ $/$ 四种，其中 $*$ $/$ 优先级高于 $+$ $-$ 。后序表达式可用如下方式递归定义：

1. 一个操作数是一个后序表达式。该表达式的值就是操作数的值。
2. 若 a, b 是后序表达式， c 是运算符，则 " $a\ b\ c$ " 是后序表达式。" $a\ b\ c$ " 的值是 $(a)\ c\ (b)$ ，即对 a 和 b 做 c 运算，且 a 是第一个操作数， b 是第二个操作数。下面是一些后序表达式及其值的例子（操作数、运算符之间用空格分隔）：

3.4 值为：3.4

5 值为：5

5 3.4 + 值为：5 + 3.4

5 3.4 + 6 / 值为：(5+3.4)/6

5 3.4 + 6 * 3 + 值为：(5+3.4)*6+3

输入

第一行是整数 n ($n < 100$)，接下来有 n 行，每行是一个后序表达式，长度不超过1000个字符

输出

对每个后序表达式，输出其值，保留小数点后面2位

样例输入

```

1 | 3
2 | 5 3.4 +
3 | 5 3.4 + 6 /
4 | 5 3.4 + 6 * 3 +

```

样例输出

```

1 | 8.40
2 | 1.40
3 | 53.40

```

来源: Guo wei

要解决这个问题，需要理解如何计算后序表达式。后序表达式的计算可以通过使用一个栈来完成，按照以下步骤：

1. 从左到右扫描后序表达式。
2. 遇到数字时，将其压入栈中。
3. 遇到运算符时，从栈中弹出两个数字，先弹出的是右操作数，后弹出的是左操作数。将这两个数字进行相应的运算，然后将结果压入栈中。
4. 当表达式扫描完毕时，栈顶的数字就是表达式的结果。

```
1  def evaluate_postfix(expression):
2      stack = []
3      tokens = expression.split()
4
5      for token in tokens:
6          if token in '+-*/':
7              # 弹出栈顶的两个元素
8              right_operand = stack.pop()
9              left_operand = stack.pop()
10             # 执行运算
11             if token == '+':
12                 stack.append(left_operand + right_operand)
13             elif token == '-':
14                 stack.append(left_operand - right_operand)
15             elif token == '*':
16                 stack.append(left_operand * right_operand)
17             elif token == '/':
18                 stack.append(left_operand / right_operand)
19         else:
20             # 将操作数转换为浮点数后入栈
21             stack.append(float(token))
22
23     # 栈顶元素就是表达式的结果
24     return stack[0]
25
26 # 读取输入行数
27 n = int(input())
28
29 # 对每个后序表达式求值
30 for _ in range(n):
31     expression = input()
32     result = evaluate_postfix(expression)
33     # 输出结果，保留两位小数
34     print(f"{result:.2f}")
```

这个程序将读取输入行数，然后对每行输入的后序表达式求值，并按要求保留两位小数输出结果。

4.5 递归与回溯：八皇后问题 (N-Queens)

- **回溯本质：**利用调用栈自动保存状态（递归）或手动使用栈进行迭代。
- **搜索策略：**逐行放置皇后，检查与已放置皇后的列、对角线是否冲突。冲突则“回滚”到上一行尝试下一个位置。

示例OJ02754: 八皇后

dfs and similar, <http://cs101.openjudge.cn/practice/02754>

会下国际象棋的人都很清楚：皇后可以在横、竖、斜线上不限步数地吃掉其他棋子。如何将8个皇后放在棋盘上（有8 * 8个方格），使它们谁也不能被吃掉！这就是著名的八皇后问题。

对于某个满足要求的8皇后的摆放方法，定义一个皇后串a与之对应，即 $a = b_1b_2 \dots b_8$ ，其中 b_i 为相应摆法中第i行皇后所处的列数。已经知道8皇后问题一共有92组解（即92个不同的皇后串）。给出一个数b，要求输出第b个串。串的比较是这样的：皇后串x置于皇后串y之前，当且仅当将x视为整数时比y小。

八皇后是一个古老的经典问题：如何在一张国际象棋的棋盘上，摆放8个皇后，使其任意两个皇后互相不受攻击。该问题由一位德国国际象棋排局家 **Max Bezzel** 于 1848年提出。严格来说，那个年代，还没有“德国”这个国家，彼时称作“普鲁士”。1850年，**Franz Nauck** 给出了第一个解，并将其扩展成了“n皇后”问题，即在一张 $n \times n$ 的棋盘上，如何摆放 n 个皇后，使其两两互不攻击。历史上，八皇后问题曾惊动过“数学王子”高斯(Gauss)，而且正是 Franz Nauck 写信找高斯请教的。

输入

第1行是测试数据的组数n，后面跟着n行输入。每组测试数据占1行，包括一个正整数b($1 \leq b \leq 92$)

输出

输出有n行，每行输出对应一个输入。输出应是一个正整数，是对应于b的皇后串。

样例输入

```
1 | 2
2 | 1
3 | 92
```

样例输出

```
1 | 15863724
2 | 84136275
```

先给出两个dfs回溯实现的八皇后，接着给出两个stack迭代实现的八皇后。

八皇后思路：回溯算法通过尝试不同的选择，逐步构建解决方案，并在达到某个条件时进行回溯，以找到所有的解决方案。从第一行第一列开始放置皇后，然后在每一行的不同列都放置，如果与前面不冲突就继续，有冲突则回到上一行继续下一个可能性。

```
1 def solve_n_queens(n):
2     solutions = [] # 存储所有解决方案的列表
3     queens = [-1] * n # 存储每一行皇后所在的列数
4
5     def backtrack(row):
6         if row == n: # 找到一个合法解决方案
7             solutions.append(queens.copy())
8         else:
```

```

9         for col in range(n):
10             if is_valid(row, col): # 检查当前位置是否合法
11                 queens[row] = col # 在当前行放置皇后
12                 backtrack(row + 1) # 递归处理下一行
13                 queens[row] = -1 # 回溯，撤销当前行的选择
14
15     def is_valid(row, col):
16         for r in range(row):
17             if queens[r] == col or abs(row - r) == abs(col - queens[r]):
18                 return False
19         return True
20
21     backtrack(0) # 从第一行开始回溯
22
23     return solutions
24
25
26 # 获取第 b 个皇后串
27 def get_queen_string(b):
28     solutions = solve_n_queens(8)
29     if b > len(solutions):
30         return None
31     queen_string = ''.join(str(col + 1) for col in solutions[b - 1])
32     return queen_string
33
34
35 test_cases = int(input()) # 输入的测试数据组数
36 for _ in range(test_cases):
37     b = int(input()) # 输入的 b 值
38     queen_string = get_queen_string(b)
39     print(queen_string)

```

```

1 def is_safe(board, row, col):
2     # 检查当前位置是否安全
3     # 检查同一列是否有皇后
4     for i in range(row):
5         if board[i] == col:
6             return False
7     # 检查左上方是否有皇后
8     i = row - 1
9     j = col - 1
10    while i >= 0 and j >= 0:
11        if board[i] == j:
12            return False
13        i -= 1
14        j -= 1
15    # 检查右上方是否有皇后
16    i = row - 1
17    j = col + 1
18    while i >= 0 and j < 8:
19        if board[i] == j:
20            return False

```

```

21     i -= 1
22     j += 1
23     return True
24
25 def queen_dfs(board, row):
26     if row == 8:
27         # 找到第b个解, 将解存储到result列表中
28         ans.append(''.join([str(x+1) for x in board]))
29         return
30     for col in range(8):
31         if is_safe(board, row, col):
32             # 当前位置安全, 放置皇后
33             board[row] = col
34             # 继续递归放置下一行的皇后
35             queen_dfs(board, row + 1)
36             # 回溯, 撤销当前位置的皇后
37             board[row] = 0
38
39 ans = []
40 queen_dfs([None]*8, 0)
41 #print(ans)
42 for _ in range(int(input())):
43     print(ans[int(input()) - 1])

```

如果要使用栈来实现八皇后问题, 可以采用迭代的方式, 模拟递归的过程。在每一步迭代中, 使用栈来保存状态, 并根据规则进行推进和回溯。

```

1 def queen_stack(n):
2     stack = [] # 用于保存状态的栈
3     solutions = [] # 存储所有解决方案的列表
4
5     stack.append((0, [])) # 初始状态为第一行, 所有列都未放置皇后, 栈中的元素是
6     # (row, queens) 的元组
7
8     while stack:
9         row, cols = stack.pop() # 从栈中取出当前处理的行数和已放置的皇后位置
10        if row == n: # 找到一个合法解决方案
11            solutions.append(cols)
12        else:
13            for col in range(n):
14                if is_valid(row, col, cols): # 检查当前位置是否合法
15                    stack.append((row + 1, cols + [col]))
16
17    return solutions
18
19 def is_valid(row, col, queens):
20     for r in range(row):
21         if queens[r] == col or abs(row - r) == abs(col - queens[r]):
22             return False
23     return True
24
25 # 获取第 b 个皇后串

```

```

26 def get_queen_string(b):
27     solutions = queen_stack(8)
28     if b > len(solutions):
29         return None
30     b = len(solutions) + 1 - b
31
32     queen_string = ''.join(str(col + 1) for col in solutions[b - 1])
33     return queen_string
34
35 test_cases = int(input()) # 输入的测试数据组数
36 for _ in range(test_cases):
37     b = int(input()) # 输入的 b 值
38     queen_string = get_queen_string(b)
39     print(queen_string)

```

```

1 def solve_n_queens(n):
2     stack = [] # 用于保存状态的栈
3     solutions = [] # 存储所有解决方案的列表
4
5     stack.append((0, [-1] * n)) # 初始状态为第一行，所有列都未放置皇后
6
7     while stack:
8         row, queens = stack.pop()
9
10        if row == n: # 找到一个合法解决方案
11            solutions.append(queens.copy())
12        else:
13            for col in range(n):
14                if is_valid(row, col, queens): # 检查当前位置是否合法
15                    new_queens = queens.copy()
16                    new_queens[row] = col # 在当前行放置皇后
17                    stack.append((row + 1, new_queens)) # 推进到下一行
18
19    return solutions
20
21
22 def is_valid(row, col, queens):
23     for r in range(row):
24         if queens[r] == col or abs(row - r) == abs(col - queens[r]):
25             return False
26     return True
27
28
29 # 获取第 b 个皇后串
30 def get_queen_string(b):
31     solutions = solve_n_queens(8)
32     if b > len(solutions):
33         return None
34     b = len(solutions) + 1 - b
35
36     queen_string = ''.join(str(col + 1) for col in solutions[b - 1])
37     return queen_string

```

```
38
39
40 test_cases = int(input()) # 输入的测试数据组数
41 for _ in range(test_cases):
42     b = int(input()) # 输入的 b 值
43     queen_string = get_queen_string(b)
44     print(queen_string)
45
```

4.6 单调栈 (Monotonic Stack)

单调栈即满足单调性的栈结构。与单调队列相比，其只在一端进行进出。将一个元素插入单调栈时，为了维护栈的单调性，需要在保证将该元素插入到栈顶后整个栈满足单调性的前提下弹出最少的元素。

单调栈是一种特殊的栈结构，其中的元素按照某种特定的顺序（如递增或递减）排列。在计算机科学中，单调栈常用于解决一类与数组或序列相关的优化问题，比如寻找下一个更大或更小的元素等。

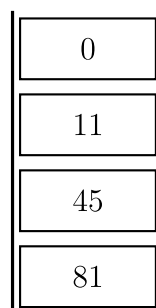
单调栈的应用场景

1. **寻找下一个更大的元素**：给定一个数组，对于每个元素，找到它右边第一个比它大的元素的位置。这类问题可以使用单调递减栈来高效解决。
2. **寻找下一个更小的元素**：类似地，如果需要找到每个元素右边第一个比它小的元素的位置，则可以使用单调递增栈。
3. **直方图中的最大矩形**：这是一个经典的问题，涉及到计算直方图中最大的矩形面积，可以使用单调栈来有效求解。

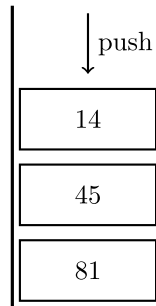
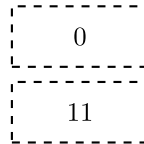
单调栈的工作原理

- **入栈操作**：当一个新的元素需要加入到栈中时，根据栈的性质（递增或递减），将所有不符合条件的栈顶元素弹出，然后再将新元素压入栈中。
- **出栈操作**：通常情况下，出栈操作是自动发生的，即在执行入栈操作时，为了保持栈的单调性，会自动移除不满足条件的栈顶元素。

例如，栈中自顶向下的元素为 {0,11,45,81}.



插入元素 14 时为了保证单调性需要依次弹出元素 0,11，操作后栈变为 {14,45,81}.



用伪代码描述如下：

```
1 insert x
2 while !sta.empty() && sta.top()<x
3     sta.pop()
4 sta.push(x)
```

实现示例

这里以一个简单的例子说明如何使用单调栈来解决问题。假设我们需要找到数组 `[4, 5, 2, 25]` 中每个元素右边第一个更大的数。

```
1 def next_greater_element(nums):
2     stack = []
3     result = [0] * len(nums)
4
5     for i in range(len(nums)):
6         # 当栈不为空且当前考察的元素大于栈顶元素时
7         while stack and nums[i] > nums[stack[-1]]:
8             index = stack.pop()
9             result[index] = nums[i]
10        # 将当前元素的索引压入栈中
11        stack.append(i)
12
13    # 对于栈中剩余的元素，它们没有更大的元素
14    while stack:
15        index = stack.pop()
16        result[index] = -1
17
18    return result
19
20 nums = [4, 5, 2, 25]
21 print(next_greater_element(nums)) # 输出: [5, 25, 25, -1]
```

在这个例子中，我们维护了一个单调递减的栈，当遇到比栈顶元素大的数时，就找到了栈顶元素的“下一个更大的数”，然后将其从栈中弹出，并记录结果。最后，对于那些在栈中没有匹配到更大数的元素，它们的结果设置为 `-1`，表示没有更大的数。

练习04137: 最小新整数

stack, greedy, <http://cs101.openjudge.cn/practice/04137/>

给定一个十进制正整数 n ($0 < n < 1000000000$), 每个数位上数字均不为0。 n 的位数为 m 。
现在从 m 位中删除 k 位 ($0 < k < m$), 求生成的新整数最小为多少?
例如: $n = 9128456$, $k = 2$, 则生成的新整数最小为12456

输入

第一行 t , 表示有 t 组数据;

接下来 t 行, 每一行表示一组测试数据, 每组测试数据包含两个数字 n, k 。

输出

t 行, 每行一个数字, 表示从 n 中删除 k 位后得到的最小整数。

样例输入

```
1 2
2 9128456 2
3 1444 3
```

样例输出

```
1 12456
2 1
```

```
1 # 蒋子轩23工学院
2 def removeKDigits(num, k):
3     stack = []
4     for digit in num:
5         while k and stack and stack[-1] > digit:
6             stack.pop()
7             k -= 1
8         stack.append(digit)
9     # 如果还未删除k位, 从尾部继续删除
10    while k:
11        stack.pop()
12        k -= 1
13    return int(''.join(stack))
14    t = int(input())
15    results = []
16    for _ in range(t):
17        n, k = input().split()
18        results.append(removeKDigits(n, int(k)))
19    for result in results:
20        print(result)
```

练习27205: 护林员盖房子 加强版

monotonic stack, <http://cs101.openjudge.cn/pctbook/T27205/>

在一片保护林中，护林员想要盖一座房子来居住，但他不能砍伐任何树木。
现在请你帮他计算：保护林中所能用来盖房子的矩形空地的最大面积。

输入

保护林用一个二维矩阵来表示，长宽都不超过1000（即 ≤ 1000 ）。
第一行是两个正整数 m, n ，表示矩阵有 m 行 n 列。
然后是 m 行，每行 n 个整数，用1代表树木，用0表示空地。

输出

一个正整数，表示保护林中能用来盖房子的最大矩形空地面积。

练习T26977: 接雨水

stack, dp, math, <http://cs101.openjudge.cn/practice/26977/>

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例：



height = [0,1,0,2,1,0,1,3,2,1,2,1]

由数组表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

输入

第一行包含一个整数 n 。 $1 \leq n \leq 2 * 10^4$

第二行包含 n 个整数，相邻整数间以空格隔开。 $0 \leq ratings[i] \leq 2 * 10^5$

输出

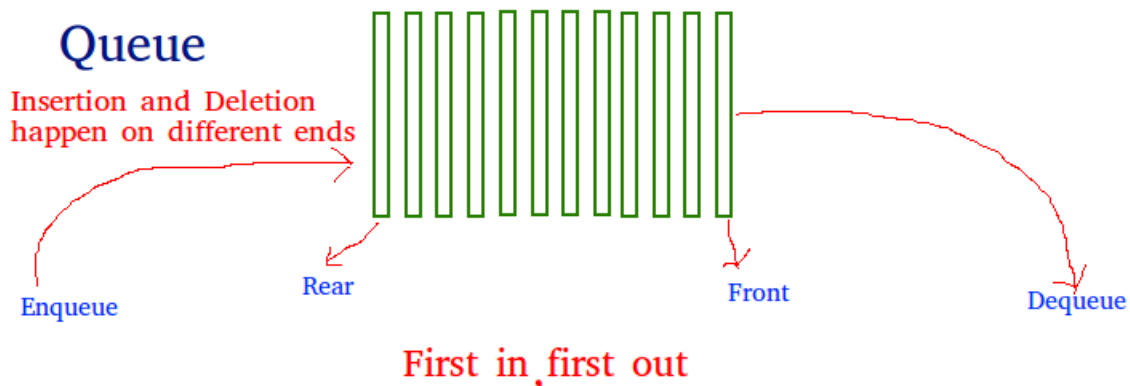
一个整数

5 队列 (Queue) - FIFO (先进先出)

队列 (queue) 是一种具有「先进入队列的元素一定先出队列」性质的表。由于该性质，队列通常也被称为先进先出 (first in first out) 表，简称 FIFO 表。

队列的一个很好的例子是资源的消费者队列，其中最早来的消费者会被优先服务。这意味着，队列中的元素按照它们进入的顺序进行处理，确保了第一个进入队列的元素也是第一个从队列中出来的，体现了“先到先得”的原则。

队列在尾部 (Rear) 添加，从头部 (Front) 移除。



Operations associated with queue are:

- Enqueue: Adds an item to the queue. If the queue is full, then it is said to be an Overflow condition – Time Complexity : $O(1)$
- Dequeue: Removes an item from the queue. The items are popped in the same order in which they are pushed. If the queue is empty, then it is said to be an Underflow condition – Time Complexity : $O(1)$
- Front: Get the front item from queue – Time Complexity : $O(1)$
- Rear: Get the last item from queue – Time Complexity : $O(1)$

The queue abstract data type is defined by the following structure and operations. A queue is structured, as described above, as an ordered collection of items which are added at one end, called the “rear,” and removed from the other end, called the “front.” mQueues maintain a FIFO ordering property. The queue operations are given below.

- `Queue()` creates a new queue that is empty. It needs no parameters and returns an empty queue.
- `enqueue(item)` adds a new item to the rear of the queue. It needs the item and returns nothing.
- `dequeue()` removes the front item from the queue. It needs no parameters and returns the item. The queue is modified.
- `isEmpty()` tests to see whether the queue is empty. It needs no parameters and returns a boolean value.
- `size()` returns the number of items in the queue. It needs no parameters and returns an integer.

As an example, if we assume that `q` is a queue that has been created and is currently empty, then [Table 1](#) shows the results of a sequence of queue operations. The queue contents are shown such that the front is on the right. 4 was the first item enqueued so it is the first item returned by dequeue.

Queue Operation	Queue Contents	Return Value
<code>q.isEmpty()</code>	<code>[]</code>	<code>True</code>
<code>q.enqueue(4)</code>	<code>[4]</code>	
<code>q.enqueue('dog')</code>	<code>['dog', 4]</code>	
<code>q.enqueue(True)</code>	<code>[True, 'dog', 4]</code>	
<code>q.size()</code>	<code>[True, 'dog', 4]</code>	<code>3</code>
<code>q.isEmpty()</code>	<code>[True, 'dog', 4]</code>	<code>False</code>
<code>q.enqueue(8.4)</code>	<code>[8.4, True, 'dog', 4]</code>	
<code>q.dequeue()</code>	<code>[8.4, True, 'dog']</code>	<code>4</code>
<code>q.dequeue()</code>	<code>[8.4, True]</code>	<code>'dog'</code>
<code>q.size()</code>	<code>[8.4, True]</code>	<code>2</code>

5.1 性能陷阱：Python 实现对比

再次创建一个新类来实现队列这种抽象数据类型是合适的。像之前一样，我们将利用列表集合的强大和简洁性来构建队列的内部表示。

- 使用 `list` 实现：若以索引 `0` 为队头，`pop(0)` 的复杂度是 $O(n)$ ，效率极低。
- 使用 `collections.deque`：专门优化的双端队列，`popleft()` 和 `append()` 均为 $O(1)$ 。

我们需要决定使用列表的哪一端作为队列的尾部（rear），哪一端作为前端（front）。清单1中所示的实现假设列表的位置0处为队列的尾部。这允许我们对列表使用 `insert` 函数在队列尾部添加新元素。可以使用 `pop` 操作移除前端的元素（即列表的最后一个元素）。请记住，这也意味着入队（enqueue）操作的时间复杂度为 $O(n)$ ，而出队（dequeue）操作的时间复杂度为 $O(1)$ 。

这种方法下，虽然在队列的尾部添加元素需要移动其他元素以腾出空间，导致了较高的时间复杂度 $O(n)$ ，但是从队列前端移除元素则非常高效，不需要额外的元素移动（时间复杂度为 $O(1)$ ）。因此，选择列表的哪一端作为队列的前端或尾部取决于应用的具体需求以及对于入队和出队操作性能的不同考虑。然而，通常更常见的是将列表的末尾视为队列的尾部，这样可以优化入队操作的效率，尽管这与上述示例中的做法相反。

Queue
- items: list
+ is_empty(self) : : boolean + enqueue(self, item: T) : : void + dequeue(self) : : T + size(self) : : int

Listing 1

```

1  class Queue:
2      def __init__(self):
3          self.items = []
4
5      def is_empty(self):
6          return self.items == []
7
8      def enqueue(self, item):
9          self.items.insert(0, item)
10
11     def dequeue(self):
12         return self.items.pop()
13
14     def size(self):
15         return len(self.items)
16
17
18  q = Queue()
19
20  q.enqueue('hello')
21  q.enqueue('dog')
22  q.enqueue(3)
23  print(q.items)
24
25  q.dequeue()
26  print(q.items)
27  # output:
28  # [3, 'dog', 'hello']
29  # [3, 'dog']

```

pythonthututor.com

Python Tutor: Visualize code in [Python](#), [JavaScript](#), [C](#), [C++](#), and [Java](#)

Python 3.11
[known limitations](#)

```

1 class Queue:
2     def __init__(self):
3         self.items = []
4
5     def is_empty(self):
6         return self.items == []
7
8     def enqueue(self, item):
9         self.items.insert(0, item)
10
11    def dequeue(self):
12        return self.items.pop()
13
14    def size(self):
15        return len(self.items)
16
17 q = Queue()
18 q.enqueue(4)
19 q.enqueue('dog')
20 q.enqueue(True)
21 print(q.size())
22 
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Step 17 of 21

Print output (drag lower right corner to resize)

Frames

Global frame

Queue

q

enqueue

self

item

Return value

None

Objects

Queue class

__init__	function	__init__(self)
dequeue	function	dequeue(self)
enqueue	function	enqueue(self, item)
is_empty	function	is_empty(self)
size	function	size(self)

Queue instance

items

list

0	1	2
True	"dog"	4

Q: Suppose you have the following series of queue operations.

```

1 q = Queue()
2 q.enqueue('hello')
3 q.enqueue('dog')
4 q.enqueue(3)
5 q.dequeue()

```

What items are left on the queue? (B)

- A. 'hello', 'dog'
- B. 'dog', 3**
- C. 'hello', 3
- D. 'hello', 'dog', 3

示例OJ02746: 约瑟夫问题

implementation, <http://cs101.openjudge.cn/practice/02746>

约瑟夫问题：有 n 只猴子，按顺时针方向围成一圈选大王（编号从 1 到 n ），从第 1 号开始报数，一直数到 m ，数到 m 的猴子退出圈外，剩下的猴子再接着从 1 开始报数。就这样，直到圈内只剩下一只猴子时，这个猴子就是猴王，编程求输入 n ， m 后，输出最后猴王的编号。

输入

每行是用空格分开的两个整数，第一个是 n，第二个是 m ($0 < m, n \leq 300$)。最后一行是：

0 0

输出

对于每行输入数据（最后一行除外），输出数据也是一行，即最后猴王的编号

样例输入

```
1 | 6 2
2 | 12 4
3 | 8 3
4 | 0 0
```

样例输出

```
1 | 5
2 | 1
3 | 7
```

说明：使用 队列queue 这种数据结构会方便。它有三种实现方式，我们最常用的 list 就支持，说明，<https://www.geeksforgeeks.org/queue-in-python/>

用list实现队列， $O(n)$

```
1  # 先使用pop从列表中取出，如果不符合要求再append回列表，相当于构成了一个圈
2  def hot_potato(name_list, num):
3      queue = []
4      for name in name_list:
5          queue.append(name)
6
7      while len(queue) > 1:
8          for i in range(num):
9              queue.append(queue.pop(0)) # O(N)
10             queue.pop(0)                # O(N)
11     return queue.pop(0)                 # O(N)
12
13
14 while True:
15     n, m = map(int, input().split())
16     if {n,m} == {0}:
17         break
18     monkey = [i for i in range(1, n+1)]
19     print(hot_potato(monkey, m-1))
20
```

用内置deque， $O(1)$

```

1  from collections import deque
2
3  # 先使用pop从列表中取出，如果不符合要求再append回列表，相当于构成了一个圈
4  def hot_potato(name_list, num):
5      queue = deque()
6      for name in name_list:
7          queue.append(name)
8
9      while len(queue) > 1:
10         for i in range(num):
11             queue.append(queue.popleft()) # O(1)
12         queue.popleft()
13     return queue.popleft()
14
15
16 while True:
17     n, m = map(int, input().split())
18     if {n,m} == {0}:
19         break
20     monkey = [i for i in range(1, n+1)]
21     print(hot_potato(monkey, m-1))

```

5.2 典型应用：系统模拟

打印机模拟：通过时间戳计算平均等待时间，评估系统吞吐量。

一个更有趣的例子是模拟打印任务队列。学生向共享打印机发送打印请求，这些打印任务被存在一个队列中，并且按照先到先得的顺序执行。这样的设定可能导致很多问题。其中最重要的是，打印机能否处理一定量的工作。如果不能，学生可能会由于等待过长时间而错过要上的课。

考虑计算机科学实验室里的这样一个场景：在任何给定的一小时内，实验室里都有约 10 个学生。他们在这小时内最多打印 2 次，并且打印的页数从 1 到 20 不等。实验室的打印机比较老旧，每分钟只能以低质量打印 10 页。可以将打印质量调高，但是这样做会导致打印机每分钟只能打印 5 页。降低打印速度可能导致学生等待过长时间。那么，应该如何设置打印速度呢？

可以通过构建一个实验室模型来解决该问题。我们需要为学生、打印任务和打印机构建对象，如图所示。当学生提交打印任务时，将它们加入等待列表中，该列表是打印机上的**打印任务队列**。当打印机执行完一个任务后，它会检查该队列，看看其中是否还有需要处理的任务。我们感兴趣的是学生**平均需要等待多久**才能拿到打印好的文章。这个时间**等于打印任务在队列中的平均等待时间**。

Lab Computers

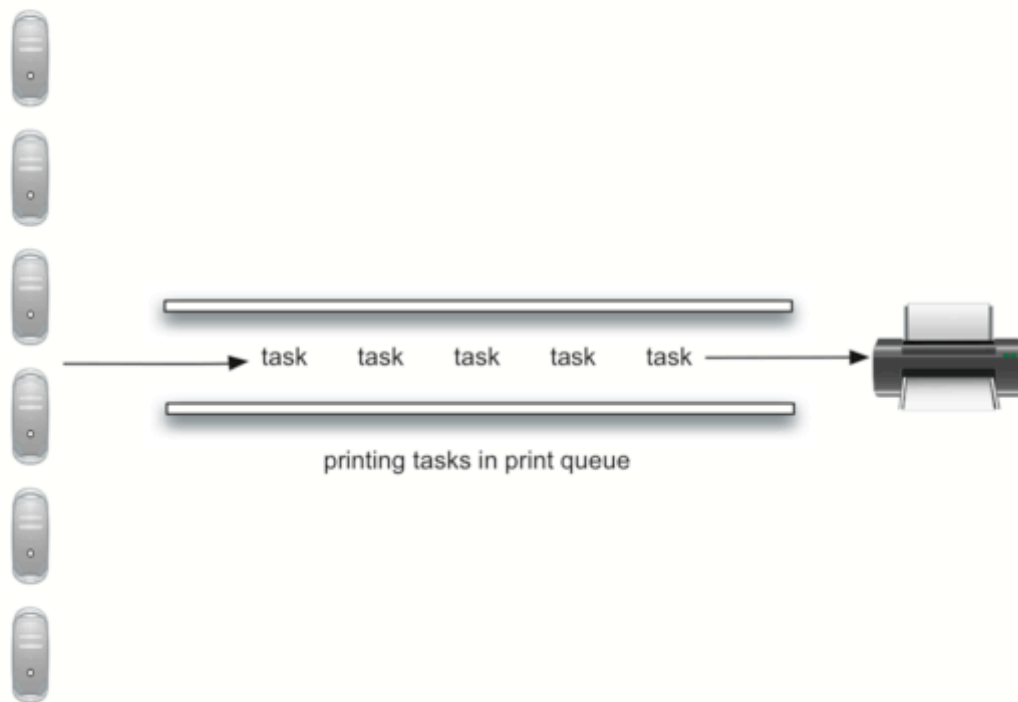


Figure 4: Computer Science Laboratory Printing Queue

在模拟时，需要应用一些概率学知识。举例来说，学生打印的文章可能有 1~20 页。如果各页数出现的概率相等，那么打印任务的实际时长可以通过 1~20 的一个随机数来模拟。

如果实验室里有 10 个学生，并且在一小时内每个人都打印两次，那么每小时平均就有 20 个打印任务。在任意一秒，创建一个打印任务的概率是多少？回答这个问题需要考虑任务与时间的比值。每小时 20 个任务相当于每 180 秒 1 个任务。

$$\frac{20 \text{ tasks}}{1 \text{ hour}} \times \frac{1 \text{ hour}}{60 \text{ minutes}} \times \frac{1 \text{ minute}}{60 \text{ seconds}} = \frac{1 \text{ task}}{180 \text{ seconds}}$$

1.主要模拟步骤

下面是主要的模拟步骤。

(1) 创建一个打印任务队列。每一个任务到来时都会有一个时间戳。一开始，队列是空的。

(2) 针对每一秒（current_second），执行以下操作。

- ❑ 是否有新创建的打印任务？如果是，以current_second作为其时间戳并将该任务加入到队列中。
- ❑ 如果打印机空闲，并且有正在等待执行的任务，执行以下操作：
 - 从队列中取出第一个任务并提交给打印机；
 - 用current_second减去该任务的时间戳，以此计算其等待时间；
 - 将该任务的等待时间存入一个列表，以备后用；
 - 根据该任务的页数，计算执行时间。
- ❑ 打印机进行一秒的打印，同时从该任务的执行时间中减去一秒。
- ❑ 如果打印任务执行完毕，或者说任务需要的时间减为0，则说明打印机回到空闲状态。

(3) 当模拟完成之后，根据等待时间列表中的值计算平均等待时间。

2.Python实现

我们创建3个类：Printer、Task和PrintQueue。它们分别模拟打印机、打印任务和队列。

Printer类需要检查当前是否有待完成的任务。如果有，那么打印机就处于工作状态（busy方法），并且其工作所需的时间可以通过要打印的页数来计算。其构造方法会初始化打印速度，即每分钟打印多少页。tick方法会减量计时，并且在执行完任务之后将打印机设置成空闲状态None。

Task类代表单个打印任务。当任务被创建时，随机数生成器会随机提供页数，取值范围是1~20。我们使用random模块中的randrange函数来生成随机数。

Queue	Printer	Task
- items: list	- page_rate: int - current_task: Task - time_remaining: int	- timestamp: int - pages: int
+ is_empty(self) + enqueue(self, item) + dequeue(self) + size(self) :: int	+ tick(self) + busy(self) + start_next(self, newtask)	+ get_stamp(self) + get_pages(self) + wait_ime(self, currenttime)

代码清单3-11 Printer类

```
1  import random
2
3  class Queue:
4      def __init__(self):
5          self.items = []
6
7      def is_empty(self):
8          return self.items == []
9
10     def enqueue(self, item):
11         self.items.insert(0, item)
12
13     def dequeue(self):
14         return self.items.pop()
15
16     def size(self):
17         return len(self.items)
18
19
20  class Printer:
21     def __init__(self, ppm):
22         self.page_rate = ppm
23         self.current_task = None
24         self.time_remaining = 0
25
26     def tick(self):
27         if self.current_task != None:
28             self.time_remaining = self.time_remaining - 1
29             if self.time_remaining <= 0:
30                 self.current_task = None
31
```

```

32     def busy(self):
33         if self.current_task != None:
34             return True
35         else:
36             return False
37
38     def start_next(self, new_task):
39         self.current_task = new_task
40         self.time_remaining = new_task.get_pages() * 60 / self.page_rate
41
42
43 class Task:
44     def __init__(self, time):
45         self.timestamp = time
46         self.pages = random.randrange(1, 21)
47
48     def get_stamp(self):
49         return self.timestamp
50
51     def get_pages(self):
52         return self.pages
53
54     def wait_time(self, current_time):
55         return current_time - self.timestamp
56
57
58 def simulation(num_seconds, pages_per_minute):
59     lab_printer = Printer(pages_per_minute)
60     print_queue = Queue()
61     waiting_times = []
62
63     for current_second in range(num_seconds):
64
65         if new_print_task():
66             task = Task(current_second)
67             print_queue.enqueue(task)
68
69         if (not lab_printer.busy()) and (not print_queue.is_empty()):
70             nexttask = print_queue.dequeue()
71             waiting_times.append(nexttask.wait_time(current_second))
72             lab_printer.start_next(nexttask)
73
74         lab_printer.tick()
75
76     average_wait = sum(waiting_times) / len(waiting_times)
77     print(f"Average Wait {average_wait:6.2f} secs {print_queue.size():3d}
78 tasks remaining.")
79
80 def new_print_task():
81     num = random.randrange(1, 181)
82     if num == 180:
83         return True
84     else:
85         return False

```

```

86
87
88 for i in range(10):
89     simulation(3600, 10) # 设置总时间和打印机每分钟打印多少页
90
91 """
92 Average Wait  20.05 secs  0 tasks remaining.
93 Average Wait  20.12 secs  0 tasks remaining.
94 Average Wait  28.32 secs  0 tasks remaining.
95 Average Wait   7.65 secs  0 tasks remaining.
96 Average Wait  13.17 secs  1 tasks remaining.
97 Average Wait  45.97 secs  0 tasks remaining.
98 Average Wait  14.94 secs  0 tasks remaining.
99 Average Wait   1.81 secs  0 tasks remaining.
100 Average Wait   0.00 secs  0 tasks remaining.
101 Average Wait   6.71 secs  0 tasks remaining.
102 """

```

每一个任务都需要保存一个时间戳，用于计算等待时间。这个时间戳代表任务被创建并放入打印任务队列的时间。wait_time 方法可以获得任务在队列中等待的时间。

主模拟程序simulation实现了之前描述的算法。print_queue 对象是队列抽象数据类型的实例。布尔辅助函数new_print_task判断是否有新创建的打印任务。我们再一次使用random模块中的randrange 函数来生成随机数，不过这一次的取值范围是 1~180。平均每 180 秒有一个打印任务。通过从随机数中选取 180，可以模拟这个随机事件。

每次模拟的结果不一定相同。对此，我们不需要在意。这是由于随机数的本质导致的。我们感兴趣的是当参数改变时结果出现的趋势。

首先，模拟 60 分钟（3600 秒）内打印速度为每分钟 5 页。并且，我们进行 10 次这样的模拟。由于模拟中使用了随机数，因此每次返回的结果都不同。

在模拟 10 次之后，可以看到平均等待时间是 122.092 秒，并且等待时间的差异较大，从最短的 17.27 秒到最长的 376.05 秒。此外，只有 2 次在给定时间内完成了所有任务。

现在把打印速度改成每分钟 10 页，然后再模拟 10 次。由于加快了打印速度，因此我们希望一小时内能完成更多打印任务。

3.讨论

在之前的内容中，我们试图解答这样一个问题：如果提高打印质量并降低打印速度，打印机能否及时完成所有任务？我们编写了一个程序来模拟随机提交的打印任务，待打印的页数也是随机的。

上面的输出结果显示，按每分钟5页的打印速度，任务的等待时间在31.50秒和292.40秒之间，相差约6分钟。提高打印速度之后，等待时间在1.29秒和28.96秒之间。此外，在每分钟5页的速度下，10次模拟中有2次没有按时完成所有任务。

```
1 Average Wait 45.88 secs 0 tasks remaining.
2 Average Wait 94.42 secs 0 tasks remaining.
3 Average Wait 292.40 secs 2 tasks remaining.
4 Average Wait 49.39 secs 0 tasks remaining.
5 Average Wait 148.27 secs 0 tasks remaining.
6 Average Wait 162.19 secs 0 tasks remaining.
7 Average Wait 71.00 secs 1 tasks remaining.
8 Average Wait 31.50 secs 0 tasks remaining.
9 Average Wait 116.74 secs 0 tasks remaining.
10 Average Wait 90.63 secs 0 tasks remaining.
```

可见，降低打印速度以提高打印质量，并不是明智的做法。学生不能等待太长时间，当 they 要赶去上课时尤其如此。6分钟的等待时间实在是太长了。

这种模拟分析能帮助我们回答很多“如果”问题。只需改变参数，就可以模拟感兴趣的任意行为。以下是几个例子。

- ❑ 如果实验室里的学生增加到20个，会怎么样？
- ❑ 如果是周六，学生不需要上课，他们是否愿意等待？
- ❑ 如果每个任务的页数变少了，会怎么样？

这些问题都能通过修改本例中的模拟程序来解答。但是，模拟的准确度取决于它所基于的假设和参数。真实的打印任务数量和学生数目是准确构建模拟程序必不可缺的数据。

5.3 双端队列 (Deque)

双端队列（Double-ended Queue）允许在两端同时进行添加和删除。

与栈和队列不同的是，双端队列的限制很少。双端队列是与队列类似的有序集合。它有一前、一后两端，元素在其中保持自己的位置。与队列不同的是，双端队列对在哪一端添加和移除元素没有任何限制。新元素既可以被添加到前端，也可以被添加到后端。同理，已有的元素也能从任意一端移除。

1. 在 Python 中，双端队列的容器由 `collections.deque` 提供。

示例如下：

```
1 from collections import deque
2
3 # 新建一个 deque，并初始化内容为 [1, 2, 3]
4 queue = deque([1, 2, 3])
5
6 # 在队尾插入元素 4
7 queue.append(4)
8
9 # 在队首插入元素 0
10 queue.appendleft(0)
11
12 # 访问队列
13 # >>> queue
14 # deque([0, 1, 2, 3, 4])
```

2. 用类实现双端队列

The deque abstract data type is defined by the following structure and operations. A deque is structured, as described above, as an ordered collection of items where items are added and removed from either end, either front or rear. The deque operations are given below.

- `Deque()` creates a new deque that is empty. It needs no parameters and returns an empty deque.
- `addFront(item)` adds a new item to the front of the deque. It needs the item and returns nothing.
- `addRear(item)` adds a new item to the rear of the deque. It needs the item and returns nothing.
- `removeFront()` removes the front item from the deque. It needs no parameters and returns the item. The deque is modified.
- `removeRear()` removes the rear item from the deque. It needs no parameters and returns the item. The deque is modified.
- `isEmpty()` tests to see whether the deque is empty. It needs no parameters and returns a boolean value.
- `size()` returns the number of items in the deque. It needs no parameters and returns an integer.

As an example, if we assume that `d` is a deque that has been created and is currently empty, then Table {dequeoperations} shows the results of a sequence of deque operations. Note that the contents in front are listed on the right. It is very important to keep track of the front and the rear as you move items in and out of the collection as things can get a bit confusing.

Deque Operation	Deque Contents	Return Value
<code>d.isEmpty()</code>	<code>[]</code>	<code>True</code>
<code>d.addRear(4)</code>	<code>[4]</code>	
<code>d.addRear('dog')</code>	<code>['dog', 4,]</code>	
<code>d.addFront('cat')</code>	<code>['dog', 4, 'cat']</code>	
<code>d.addFront(True)</code>	<code>['dog', 4, 'cat', True]</code>	
<code>d.size()</code>	<code>['dog', 4, 'cat', True]</code>	<code>4</code>
<code>d.isEmpty()</code>	<code>['dog', 4, 'cat', True]</code>	<code>False</code>
<code>d.addRear(8.4)</code>	<code>[8.4, 'dog', 4, 'cat', True]</code>	
<code>d.removeRear()</code>	<code>['dog', 4, 'cat', True]</code>	<code>8.4</code>
<code>d.removeFront()</code>	<code>['dog', 4, 'cat']</code>	<code>True</code>

```

1  class Deque:
2      def __init__(self):
3          self.items = []
4
5      def is_empty(self):
6          return self.items == []
7
8      def add_front(self, item):
9          self.items.append(item)
10
11     def add_rear(self, item):
12         self.items.insert(0, item)
13
14     def remove_front(self):
15         return self.items.pop()
16
17     def remove_rear(self):
18         return self.items.pop(0)
19
20     def size(self):
21         return len(self.items)
22
23
24  d = Deque()
25  print(d.is_empty())
26  d.add_rear(4)
27  d.add_rear('dog')
28  d.add_front('cat')
29  d.add_front(True)
30  print(d.size())
31  print(d.is_empty())
32  d.add_rear(8.4)
33  print(d.remove_rear())
34  print(d.remove_front())
35  """
36  True
37  4
38  False
39  8.4
40  True
41  """

```

在双端队列的Python实现中，在前端进行的添加操作和移除操作的时间复杂度是 $O(1)$ ，在后端的则是 $O(n)$ 。

示例05902: 双端队列

<http://cs101.openjudge.cn/practice/05902/>

定义一个双端队列，进队操作与普通队列一样，从队尾进入。出队操作既可以从队头，也可以从队尾。编程实现这个数据结构。

输入

第一行输入一个整数t，代表测试数据的组数。

每组数据的第一行输入一个整数n，表示操作的次数。

接着输入n行，每行对应一个操作，首先输入一个整数type。

当type=1，进队操作，接着输入一个整数x，表示进入队列的元素。

当type=2，出队操作，接着输入一个整数c，c=0代表从队头出队，c=1代表从队尾出队。

n <= 1000

输出

对于每组测试数据，输出执行完所有的操作后队列中剩余的元素,元素之间用空格隔开，按队头到队尾的顺序输出，占一行。如果队列中已经没有任何的元素，输出NULL。

样例输入

```
1 2
2 5
3 1 2
4 1 3
5 1 4
6 2 0
7 2 1
8 6
9 1 1
10 1 2
11 1 3
12 2 0
13 2 1
14 2 0
```

样例输出

```
1 3
2 NULL
```

```
1 from collections import deque
2
3 for _ in range(int(input())):
4     n=int(input())
5     q=deque([])
6     for i in range(n):
7         a,b=map(int,input().split())
8         if a==1:
9             q.append(b)
10        else:
11            if b==0:
12                q.popleft()
13            else:
14                q.pop()
```

```

15     if q:
16         print(*q)
17     else:
18         print('NULL')

```

典型应用：回文检查

```

1  from collections import deque
2
3  def pal_checker(a_string):
4      char_deque = deque(a_string)
5      while len(char_deque) > 1:
6          # 同时从两端弹出
7          if char_deque.popleft() != char_deque.pop():
8              return False
9      return True

```

示例04067: 回文数字 (Palindrome Number)

<http://cs101.openjudge.cn/practice/04067/>

给出一系列非负整数，判断是否是一个回文数。回文数指的是正着写和倒着写相等的数。

输入

若干行，每行是一个非负整数（不超过999999999）

输出

对每行输入，如果其是一个回文数，输出YES。否则输出NO。

样例输入

```

1  11
2  123
3  0
4  14277241
5  67945497

```

样例输出

```

1  YES
2  NO
3  YES
4  YES
5  NO

```

Use the deque from the collections module. The is_palindrome function checks if a number is a palindrome by converting it to a string, storing it in a deque, and then comparing the first and last elements until the deque is empty or only contains one element.

```
1  from collections import deque
2
3  def is_palindrome(num):
4      num_str = str(num)
5      num_deque = deque(num_str)
6      while len(num_deque) > 1:
7          if num_deque.popleft() != num_deque.pop():
8              return "NO"
9      return "YES"
10
11 while True:
12     try:
13         num = int(input())
14         print(is_palindrome(num))
15     except EOFError:
16         break
```

示例04099: 队列和栈

<http://cs101.openjudge.cn/practice/04099/>

队列和栈是两种重要的数据结构，它们具有push k和pop操作。push k是将数字k加入到队列或栈中，pop则是从队列和栈取一个数出来。队列和栈的区别在于取数的位置是不同的。

队列是先进先出的：把队列看成横向的一个通道，则push k是将k放到队列的最右边，而pop则是从队列的最左边取出一个数。

栈是后进先出的：把栈也看成横向的一个通道，则push k是将k放到栈的最右边，而pop也是从栈的最右边取出一个数。

假设队列和栈当前从左至右都含有1和2两个数，则执行push 5和pop操作示例图如下：

push 5 pop

队列 1 2 -----> 1 2 5 -----> 2 5

push 5 pop

栈 1 2 -----> 1 2 5 -----> 1 2

现在，假设队列和栈都是空的。给定一系列push k和pop操作之后，输出队列和栈中存的数字。若队列或栈已经空了，仍然接收到pop操作，则输出error。

输入

第一行为m，表示有m组测试输入， $m < 100$ 。

每组第一行为n，表示下列有n行push k或pop操作。（ $n < 150$ ）

接下来n行，每行是push k或者pop，其中k是一个整数。

（输入保证同时在队列或栈中的数不会超过100个）

输出

对每组测试数据输出两行，正常情况下，第一行是队列中从左到右存的数字，第二行是栈中从左到右存的数字。若操作过程中队列或栈已空仍然收到pop，则输出error。输出应该共 $2*m$ 行。

样例输入

```
1 2
2 4
3 push 1
4 push 3
5 pop
6 push 5
7 1
8 pop
```

样例输出

```
1 3 5
2 1 5
3 error
4 error
```

```
1 from collections import deque
2 for _ in range(int(input())):
3     queue = deque()
4     stack = deque()
5     stop = False
6     for _ in range(int(input())):
7         s = input()
8         if s == 'pop':
9             try:
10                 queue.popleft()
11                 stack.pop()
12             except IndexError:
13                 stop = True
14         else:
15             a = int(s.split()[1])
16             queue.append(a)
17             stack.append(a)
18     if not stop:
19         print(' '.join(list(map(str, queue))))
20         print(' '.join(list(map(str, stack))))
21     elif stop:
22         print('error')
23         print('error')
```

5.4 单调队列

在学习单调队列前，让我们先来看一道例题。

练习T239.滑动窗口最大值

sliding window, monotonic queue, <https://leetcode.cn/problems/sliding-window-maximum/>

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 滑动窗口中的最大值。

示例 1：

```
1 输入: nums = [1,3,-1,-3,5,3,6,7], k = 3
2 输出: [3,3,5,5,6,7]
3 解释:
4 滑动窗口的位置          最大值
5 -----
6 [1  3  -1] -3  5  3  6  7      3
7  1 [3  -1  -3] 5  3  6  7      3
8  1  3 [-1  -3  5] 3  6  7      5
9  1  3  -1 [-3  5  3] 6  7      5
10 1  3  -1  -3 [5  3  6] 7      6
11 1  3  -1  -3  5 [3  6  7]      7
```

示例 2：

```
1 输入: nums = [1], k = 1
2 输出: [1]
```

提示：

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `1 <= k <= nums.length`

最暴力的想法很简单，对于每一段 $i \sim i+k-1$ 的序列，逐个比较来找出最大值，时间复杂度约为 $O(n \times k)$ 。

很显然，这其中进行了大量重复工作，除了开头 $k-1$ 个和结尾 $k-1$ 个数之外，每个数都进行了 k 次比较，而题中 100% 的数据为 $n \leq 10^5$ ，当 k 稍大的情况下，显然会 TLE。

这时所用到的就是单调队列了。

定义

顾名思义，单调队列的重点分为「单调」和「队列」。

「单调」指的是元素的「规律」——递增（或递减）。

「队列」指的是元素只能从队头和队尾进行操作。

Ps. 单调队列中的 "队列" 与正常的队列有一定的区别，稍后会提到

例题分析

解释

有了上面「单调队列」的概念，很容易想到用单调队列进行优化。

要求的是每连续的 k 个数中的最大值，很明显，当一个数进入所要 "寻找" 最大值的范围中时，若这个数比其前面（先进队）的数要大，显然，前面的数会比这个数先出队且不再可能是最大值。

也就是说——当满足以上条件时，可将前面的数 "弹出"，再将该数真正 push 进队尾。

这就相当于维护了一个递减的队列，符合单调队列的定义，减少了重复的比较次数，不仅如此，由于维护出的队伍是查询范围内的且是递减的，队头必定是该查询区域内的最大值，因此输出时只需输出队头即可。

显而易见的是，在这样的算法中，每个数只要进队与出队各一次，因此时间复杂度被降到了 $O(n)$ 。

而由于查询区间长度是固定的，超出查询空间的值再大也不能输出，因此还需要 site 数组记录第 i 个队中的数在原数组中的位置，以弹出越界的队头。

```
1  from collections import deque
2  from typing import List
3
4  class Solution:
5      def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
6          if not nums or k == 0:
7              return []
8
9          n = len(nums)
10         if k == 1:
11             return nums
12
13         deque_index = deque() # 存储索引，保持双端队列中的值递减
14         res = []
15
16         for i in range(n):
17             # 移除滑出窗口的元素（队首元素）
18             if deque_index and deque_index[0] < i - k + 1:
19                 deque_index.popleft()
20
21             # 移除所有小于当前元素的队尾元素
22             while deque_index and nums[deque_index[-1]] < nums[i]:
23                 deque_index.pop()
24
25             # 将当前元素的索引加入队列
26             deque_index.append(i)
27
28             # 从第 k 个元素开始记录结果，队首始终是窗口的最大值
```

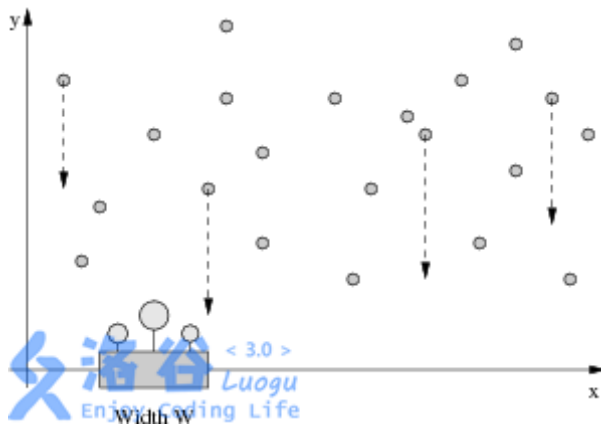
```

29         if i >= k - 1:
30             res.append(nums[deque_index[0]])
31
32     return res
33
34 if __name__ == "__main__":
35     sol = Solution()
36     print(sol.maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3)) # [3,3,5,5,6,7]

```

练习P2698 [USACO12MAR] Flowerpot S

monotonic queue, <https://www.luogu.com.cn/problem/P2698>



老板需要你帮忙浇花。给出 N 滴水的坐标, (x, y) 表示水滴最初的坐标。

每滴水均以每秒 1 个单位长度的速度下落。你需要把花盆放在 x 轴上的某个位置, 使得花盆接到第 1 滴水与最后 1 滴水之间的时间差至少为 D 。

如果水滴落在 x 轴上的位置与花盆的边沿对齐, 也认为被接住。

给出 N 滴水的坐标和时间差 D , 请算出最小的花盆宽度 W 。

输入格式

第一行 2 个整数 N 和 D 。

接下来 N 行, 每行 2 个整数, 表示水滴的坐标 (x, y) 。

输出格式

一行 1 个整数, 表示最小的花盆宽度。如果无法构造出满足题意的花盆, 则输出 -1 。

输入 #1

```

1 4 5
2 6 3
3 2 4
4 4 10
5 12 15

```

输出 #1

说明/提示

【样例解释】

有 4 滴水，初始位置分别在 $(6, 3)$, $(2, 4)$, $(4, 10)$, $(12, 15)$ 。水滴至少用 5 秒时间先后落入花盆。花盆的宽度为 2 是必须且足够的，此时把花盆放在 $x = 4 \dots 6$ 的位置，它可以接到水滴 1 和 3，之间的时间差为 $10 - 3 = 7$ ，满足条件。

【数据范围】

40% 的数据： $1 \leq N \leq 1000$, $1 \leq D \leq 2000$ 。

100% 的数据： $1 \leq N \leq 10^5$, $1 \leq D \leq 10^6$, $0 \leq x, y \leq 10^6$ 。

这是一道经典的双指针（滑动窗口）结合单调队列的题目。

题目分析

1. 物理含义：水滴 (x, y) 表示在 x 坐标处，经过 y 秒后水滴会落地。
2. 核心目标：我们要选定一个区间 $[L, R]$ （即花盆的宽度为 $R - L$ ），使得落在这个区间内的水滴中，落地的最晚时间与最早时间之差 $\geq D$ 。
3. 数学转化：在按 x 坐标排序后的水滴序列中，找到一个子序列 $[i, j]$ ，满足 $\max(y_i \dots y_j) - \min(y_i \dots y_j) \geq D$ ，且最小化 $x_j - x_i$ 。

解题思路

1. 排序：首先将所有水滴按 x 坐标从小到大排序。
2. 双指针：使用左右两个指针 `l` 和 `r` 维护一个窗口。
 - 右指针 `r` 不断向右移动，增加窗口内的水滴。
 - 当窗口内的 $\max(y) - \min(y) \geq D$ 时，尝试收缩左指针 `l`，以减小窗口宽度并更新全局最小值。
3. 单调队列：为了高效地在 $O(1)$ 时间内获取当前窗口内的最大值和最小值，我们维护两个单调队列：
 - `max_q`：单调递减队列，队头存储当前窗口 y 的最大值。
 - `min_q`：单调递增队列，队头存储当前窗口 y 的最小值。

Python 代码实现

```

1  import sys
2  from collections import deque
3
4  def solve():
5      # 使用 fast I/O
6      input_data = sys.stdin.read().split()
7      if not input_data:
8          return
9
10     n = int(input_data[0])
11     d = int(input_data[1])

```

```

12
13     points = []
14     idx = 2
15     for _ in range(n):
16         x = int(input_data[idx])
17         y = int(input_data[idx+1])
18         points.append((x, y))
19         idx += 2
20
21     # 按 x 坐标排序
22     points.sort()
23
24     # 单调队列存储的是 points 的索引
25     max_q = deque() # y 值单调递减, 队头为最大值
26     min_q = deque() # y 值单调递增, 队头为最小值
27
28     l = 0
29     ans = float('inf')
30
31     for r in range(n):
32         # 维护最大值队列
33         while max_q and points[max_q[-1]][1] <= points[r][1]:
34             max_q.pop()
35         max_q.append(r)
36
37         # 维护最小值队列
38         while min_q and points[min_q[-1]][1] >= points[r][1]:
39             min_q.pop()
40         min_q.append(r)
41
42         # 当满足条件: 最大 y - 最小 y >= d 时
43         while l <= r and points[max_q[0]][1] - points[min_q[0]][1] >=
d:
44             # 更新答案: 花盆宽度为当前两个端点的 x 差值
45             ans = min(ans, points[r][0] - points[l][0])
46
47             # 左指针右移, 尝试收缩宽度
48             l += 1
49             # 移除不在窗口内的索引
50             if max_q and max_q[0] < l:
51                 max_q.popleft()
52             if min_q and min_q[0] < l:
53                 min_q.popleft()
54
55     if ans == float('inf'):
56         print("-1")
57     else:
58         print(ans)
59
60 if __name__ == "__main__":
61     solve()

```

关键点详解

- 排序的重要性：排序后，随着 `r` 的增加，`x[r]` 是递增的。这使得我们可以利用双指针在

$O(N)$ 的时间内扫描整个数组，而不需要暴力枚举所有区间。

- 队列维护：
 - `max_q` 队头始终是当前窗口 $[l, r]$ 中 y 最大的那个点的下标。
 - `min_q` 队头始终是当前窗口 $[l, r]$ 中 y 最小的那个点的下标。
- 时间复杂度：排序耗时 $O(N \log N)$ ，双指针扫描每个元素进出队列各一次，耗时 $O(N)$ 。总复杂度 $O(N \log N)$ ，完全可以满足 $N = 10^5$ 的数据范围。
- 空间复杂度：需要 $O(N)$ 存储点的数据和队列。

样例运行

对于输入：

```
1 | 4 5
2 | 6 3
3 | 2 4
4 | 4 10
5 | 12 15
```

1. 排序后： `(2,4), (4,10), (6,3), (12,15)`
2. 当 `r=1` (点 `(4,10)`) 时： `max=10, min=4`，差值 `6 ≥ 5`。 `ans = 4-2 = 2`。
3. 当 `r=2` (点 `(6,3)`) 时：窗口内最大 $y = 10$ (来自点 `(4,10)`)，最小 $y = 3$ (来自点 `(6,3)`)，差值 `7 ≥ 5`。尝试收缩 `l`， `ans = min(2, 6-4) = 2`。
4. 最终输出 `2`。

附录

Python数据结构与算法分析（第3版），<https://runestone.academy/ns/books/published/pythonds3/index.html>

算法导论 第三版 (Thmos.H.Cormen ,Charles E. Leiserson etc.)